

Mémoire de fin de cycle

Expert en informatique et systèmes d'information, option DevOps

Amir REDJEM

Supervisé par Joris VERGULDEZOONE, référent de formation ESI

Problématique

Comment concevoir et déployer une infrastructure DevOps entièrement automatisée, sécurisée et éco-responsable, permettant la livraison continue d'une plateforme SaaS de partage de médias tout en garantissant la qualité, l'observabilité et la maîtrise des coûts ?

Référence RNCP40573BC06, option DevOps

Titre « Expert en Informatique et Systèmes d'Information », niveau 7 (Bac+5)

Année de formation 2024 / 2026

Soutenance du 03/08/2026

École LaPlateforme, Marseille. Entreprise d'accueil, FDJ United, Vitrolles.

Remerciements

Ce mémoire clôt un cycle de formation d'Expert en Informatique et Systèmes d'Information. Les remerciements s'adressent en premier lieu à l'École LaPlateforme et à son équipe pédagogique pour la qualité de l'encadrement dispensé tout au long du parcours, ainsi qu'à Joris VERGULDEZOONE, référent de formation, pour son accompagnement dans la définition de la problématique et la structuration du présent dossier.

Ils s'adressent également à l'entreprise FDJ United, structure d'accueil en alternance, et à Philippe AMERIO, référent en entreprise, dont l'expérience opérationnelle en ingénierie DevOps a nourri la réflexion menée dans ce projet.

La reconnaissance va enfin à l'ensemble des personnes ayant contribué à la réussite de ce travail.

Résumé

Le partage de photographies et de vidéos en ligne impose des exigences fortes de confidentialité, de disponibilité et de performance, tout en confrontant les organisations à la complexité et au coût de l'exploitation d'un service en ligne. Le présent mémoire traite de la conception et du déploiement d'une infrastructure DevOps entièrement automatisée pour dropicture, une plateforme SaaS de partage de médias où chaque contenu demeure privé par défaut. La démarche mobilise l'infrastructure as code (Terraform et Ansible), une chaîne d'intégration et de livraison continues (GitHub Actions), l'orchestration de conteneurs (Docker Swarm derrière un reverse proxy Traefik) et une stack d'observabilité complète (Prometheus, Loki, Grafana, Alertmanager). Une attention particulière est portée à la sécurité par défense en profondeur, à la conformité RGPD et à la cartographie ISO/IEC 27001, à l'éco-responsabilité par le dimensionnement au plus juste, la gestion du cycle de vie du stockage et la supervision de la consommation énergétique estimée, ainsi qu'à la maîtrise des coûts. Les résultats démontrent qu'une automatisation de bout en bout, reproductible et observable, fiabilise et accélère les déploiements tout en conservant un niveau de sécurité et de sobriété élevé. Des perspectives d'amélioration continue sont enfin formulées.

Mots-clés. DevOps, infrastructure as code, CI/CD, conteneurisation, orchestration, observabilité, sécurité, green IT, SaaS.

Abstract

Sharing photos and videos online demands strong guarantees of privacy, availability and performance, while confronting organisations with the complexity and cost of operating an online service. This dissertation addresses the design and deployment of a fully automated DevOps infrastructure for dropicture, a media-sharing SaaS platform where every item is private by default. The approach relies on infrastructure as code (Terraform and Ansible), a continuous integration and delivery pipeline (GitHub Actions), container orchestration (Docker Swarm behind a Traefik reverse proxy) and a complete observability stack (Prometheus, Loki, Grafana, Alertmanager). Particular emphasis is placed on security through defence in depth, on GDPR compliance and ISO/IEC 27001 mapping, on eco-responsibility through right-sizing, storage lifecycle management and monitoring of estimated energy consumption, and on cost control. The results show that end-to-end, reproducible and observable automation makes deployments more reliable and faster while preserving a high level of security and sobriety. Avenues for continuous improvement are finally outlined.

Keywords. DevOps, infrastructure as code, CI/CD, containerisation, orchestration, observability, security, green IT, SaaS.

Sommaire

Mémoire de fin de cycle	1
Expert en informatique et systèmes d'information, option DevOps.....	1
Résumé	3
Abstract	4
Sommaire	5
Liste des abréviations et définition des termes	10
Glossaire	11
9. Présentation personnelle et Introduction.....	1
9.1 Introduction	1
9.2 Parcours académique et professionnel	2
9.3 Objectifs de la présentation	3
10. Présentation de la problématique	4
10.1 Contexte du projet	4
10.2 Description de la problématique	4
10.3 Objectifs du projet.....	5
10.4 Importance et impact de la problématique	6
10.5 Questions de recherche et hypothèses	6
11. Analyse et Définition de la Stratégie du Système d'Information.....	8
11.1 Veille technologique	8
11.1.1 Introduction	8
11.1.2 Méthodologie de la veille technologique	9
11.1.3 Résultats de la veille technologique.....	11
11.1.4 Recommandations et perspectives	13
11.1.5 Conclusion	14
11.2 Besoins fonctionnels et cahier des charges fonctionnel	14
11.2.1 Description des fonctionnalités requises.....	15

11.2.2 Collecte et analyse des besoins	16
11.3 Analyse stratégique	18
11.3.1 Introduction	18
11.3.2 Diagnostic du système d'information (SI).....	18
11.3.3 Gestion des risques	21
11.3.4 Élaboration de la stratégie informatique	22
11.3.5 Présentation des préconisations.....	26
11.3.6 Conclusion	28
12. Implémentations techniques	30
12.1 Infrastructure et environnement de déploiement.....	30
12.1.1 Infrastructure matérielle	30
12.1.2 Environnement de déploiement (cloud, on-premise, hybride)	32
12.2 Technologies et outils utilisés.....	33
12.2.1 Algorithmes et frameworks	33
12.2.2 Outils de développement et de gestion de projet.....	34
12.3 Surveillance et monitoring de la sécurité.....	35
12.3.1 Outils de monitoring	36
12.3.2 Techniques de détection et de réponse aux incidents.....	36
12.4 Plans de sauvegarde et de récupération.....	37
12.4.1 Stratégies de sauvegarde des données	38
12.4.2 Plans de reprise après sinistre.....	39
13. Gestion du projet et management.....	41
13.1 Élaboration du planning et gestion des ressources.....	41
13.1.1 Création d'un planning détaillé.....	41
13.1.2 Allocation des ressources	43
13.2 Suivi de l'avancement et gestion des risques.....	43
13.2.1 Indicateurs de performance	43

13.2.2 Gestion des risques	44
13.3 Management de l'équipe projet	45
13.3.1 Encadrement et motivation	45
13.3.2 Développement des compétences.....	46
13.4 Communication et coordination avec les parties prenantes	46
13.4.1 Plan de communication.....	46
13.4.2 Engagement des parties prenantes	47
13.5 Planification	48
13.5.1 Introduction	48
13.5.2 Méthodologie de projet	48
13.5.3 Phases du projet	49
13.5.4 Planning détaillé.....	49
13.5.5 Ressources et budget	49
14. Conception et Développement de l'Application Informatique.....	52
14.1 Conception	52
14.1.1 Introduction	52
14.1.2 Décomposition de l'application.....	53
14.1.3 Fonctionnement de l'application.....	54
14.1.4 Données utilisées.....	55
14.1.5 Environnement informatique proposé	56
14.1.6 Spécifications techniques	57
14.2 Développement et Stratégie de Tests	58
14.2.1 Objectifs de la stratégie de tests.....	58
14.2.2 Types de tests et outils utilisés	59
14.2.3 Plan de test détaillé.....	59
14.2.4 Environnement de test	61
14.3 Modalités de livraison	62

14.3.1 Mise en service opérationnelle et vérification de service régulier (VSR) .	62
14.3.2 Conclusion	63
17. Concevoir et déployer des infrastructures DevOps automatisées (Option DevOps)	65
17.1 Analyse de l'infrastructure et des processus de développement.....	65
17.2 Conception et implémentation des pipelines	69
17.3 Orchestration des environnements conteneurisés	72
17.4 Mise en œuvre d'une stratégie de monitoring et d'observabilité	75
18. Analyses et Perspectives	79
18.1 Analyse des résultats et des performances.....	79
18.1.1 Évaluation des objectifs atteints.....	79
18.1.2 Analyse des performances	79
18.2 Retour d'expérience et leçons apprises.....	80
18.2.1 Identification des succès et des défis	80
18.2.2 Capitalisation des connaissances.....	81
18.3 Perspectives et recommandations	81
19. Conclusion et remerciements.....	83
19.1 Synthèse des principaux résultats et impacts	83
19.2 Remerciements	83
20. Bibliographie / Webographie	85
Référentiel et cadre de certification	85
Ouvrages et publications de référence.....	85
Webographie technique.....	85
Cadre normatif et réglementaire	86
21. Annexes	87
Justification des choix de représentation.....	87
Annexes illustrées	88

Annexe A, Cartographie du SI, 4 niveaux (métier, fonctionnel, applicatif, infrastructure) (BC1)	89
Annexe B, Topologie de l'infrastructure et du déploiement (BC4C)	90
Annexe C, Conteneurisation, services et segmentation réseau (Docker Swarm) (BC4C).....	91
Annexe D, Modèle Conceptuel de Données (MCD) (BC3)	92
Annexe E, Modèle Logique de Données (MLD) (BC3).....	93
Annexe G, Diagramme de cas d'utilisation (BC3)	94
Annexe H, Diagramme de classes (backend) (BC3).....	95
Annexe I.1, Séquence, téléversement d'un média (BC3).....	96
Annexe I.2, Séquence, publication / partage (BC3)	97
Annexe I.3, Séquence, session sécurisée (connexion) (BC3)	98
Annexe J, Journal des décisions d'architecture (ADR) (BC1)	99
Annexe K, Pipeline CI/CD (chaîne de déploiement SaaS) (BC4C).....	100
Annexe L, Work Breakdown Structure (WBS) (BC2)	101
Annexe M, Diagramme PERT et chemin critique (BC2).....	102
Annexe N, Planning (diagramme de Gantt) (BC2)	103
Annexe O, Matrice des risques (probabilité / impact) (BC2)	104
Annexe P, Architecture de la supervision et de l'observabilité (BC4C)	105
Annexe Q, Contrat d'interface (OpenAPI / Swagger) (BC3).....	106
Annexe R, Cartographie ISO/IEC 27001 & RGPD (BC4C)	107
Annexe S, Chaîne DevSecOps et plan de reprise d'activité (BC4C).....	108
Annexe F, Maquettes des interfaces et flux de données (BC3)	109
Annexe L bis, Matrice RACI (BC2)	110
Annexe O bis, Matrice d'engagement des parties prenantes (BC2).....	111
Annexe T, Outils de gestion de projet ()	112

Liste des abréviations et définition des termes

Abréviation	Signification
ACM	AWS Certificate Manager
ADR	Architecture Decision Record
API	Application Programming Interface
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Delivery
CORS	Cross-Origin Resource Sharing
DORA	DevOps Research and Assessment
DTO	Data Transfer Object
GHCR	GitHub Container Registry
HSTS	HTTP Strict Transport Security
IaC	Infrastructure as Code
MCD / MLD	Modèle Conceptuel / Logique de Données
MTTR	Mean Time To Recovery
OAC	Origin Access Control
ORM	Object-Relational Mapping
PRA	Plan de Reprise d'Activité
RGPD	Règlement Général sur la Protection des Données
SaaS	Software as a Service
SSM	AWS Systems Manager
SSR	Server-Side Rendering
TLS	Transport Layer Security

VSR	Vérification de Service Régulier
WAF	Web Application Firewall
WBS	Work Breakdown Structure

Glossaire

- **DevOps** désigne un ensemble de pratiques rapprochant le développement et l'exploitation afin d'automatiser et de fiabiliser la livraison logicielle.
- **GitOps** désigne une approche où l'état désiré du système est décrit dans Git, qui devient la source de vérité déclenchant les déploiements.
- **Infrastructure as Code** désigne la description de l'infrastructure sous forme de code versionné et reproductible.
- **Conteneurisation** désigne l'encapsulation d'une application et de ses dépendances dans une image exécutable isolée.
- **Orchestration** désigne l'automatisation du déploiement, du placement, de la mise à l'échelle et de la résilience des conteneurs.
- **Observabilité** désigne la capacité à comprendre l'état interne d'un système à partir de ses métriques, de ses journaux et de ses traces.
- **Green IT** désigne l'ensemble des pratiques visant à réduire l'empreinte environnementale du numérique.
- **Rolling update** désigne la mise à jour progressive d'un service sans interruption, avec repli automatique en cas d'anomalie.
- **Session opaque** désigne un jeton de session sans contenu exploitable côté client, dont la validité est vérifiée côté serveur.

9. Présentation personnelle et Introduction

9.1 Introduction

La photographie et la vidéo numériques occupent une place centrale dans les usages contemporains. Chaque utilisateur accumule des milliers de contenus personnels dont il souhaite, tour à tour, préserver la confidentialité et partager une sélection choisie. Les services existants imposent souvent un compromis inconfortable entre commodité et maîtrise, car les contenus déposés deviennent difficilement contrôlables, la frontière entre privé et public reste floue et l'utilisateur perd la visibilité sur l'usage réel de ses données. C'est à cette tension que répond dropicture, une plateforme SaaS de partage de médias fondée sur un principe simple selon lequel tout demeure privé par défaut et seule une action explicite rend un contenu public.

Derrière cette promesse produit se trouve un enjeu d'ingénierie considérable. Offrir un service en ligne fiable, sécurisé et performant suppose une infrastructure capable d'absorber la variabilité de la charge, de résister aux pannes, de protéger des données personnelles et de se déployer sans interruption ni régression. La construction et l'exploitation d'une telle infrastructure constituent, pour beaucoup d'organisations, une source majeure de complexité, de risque et de coût. La discipline DevOps est précisément née de la volonté de maîtriser cette complexité, en automatisant la chaîne qui relie le code à la production et en instaurant une culture de mesure et d'amélioration continue.

Le présent mémoire s'inscrit dans cette perspective. Il documente la conception et le déploiement d'une infrastructure DevOps entièrement automatisée pour la plateforme dropicture, en traitant conjointement quatre exigences qui structurent l'ensemble du raisonnement, à savoir l'automatisation de bout en bout, la sécurité, l'éco-responsabilité et la maîtrise des coûts. Ces exigences ne sont pas abordées isolément mais comme un système d'arbitrages, où chaque décision technique, qu'il s'agisse du choix d'un orchestrateur, de la conception d'un pipeline ou de la stratégie d'observabilité, se justifie au regard de la problématique d'ensemble.

Rédigé selon la posture d'un consultant chargé d'analyser et d'outiller une mission, ce dossier reprend, de manière synthétique et appliquée au projet, les différents

blocs de compétences du cycle. Après la présentation de la problématique en section 10, l'analyse et la définition de la stratégie du système d'information sont exposées en section 11, suivies des implémentations techniques en section 12, de la gestion de projet en section 13 et de la conception de l'application en section 14. La section 17 constitue le cœur de l'option DevOps et détaille la conception et le déploiement de l'infrastructure automatisée. Les analyses, le retour d'expérience et les perspectives en section 18 précèdent la conclusion. L'ensemble des schémas est reporté en annexe afin de préserver la lisibilité du raisonnement.

9.2 Parcours académique et professionnel

Ce mémoire est présenté dans le cadre du titre « Expert en Informatique et Systèmes d'Information » (RNCP40573, niveau 7) préparé à l'École LaPlateforme, à Marseille, de 2024 à 2026. Ce cursus fait suite à une Licence en Informatique obtenue à l'Université d'Aix-Marseille de 2020 à 2024, qui a constitué le socle théorique en algorithmique, en bases de données et en génie logiciel.

Le parcours professionnel se caractérise par une progression du développement applicatif vers l'ingénierie des systèmes et de l'automatisation, cumulant quatre années d'expérience. Une première expérience de développeur full-stack, menée chez IT Remote de 2022 à 2023 dans un cadre agile, a porté sur le développement d'une console d'administration et d'agents Windows en ASP.NET Core et TypeScript. Une seconde expérience, chez Altares Dun & Bradstreet de 2023 à 2024, a élargi le périmètre au développement front-end en Next.js et back-end en Java Spring Boot, avec la mise en œuvre de la recherche à grande échelle via Elasticsearch et l'optimisation de bases Oracle et MySQL.

Depuis septembre 2024, le poste d'ingénieur DevOps occupé en alternance chez FDJ United, à Vitrolles, a consolidé une expertise directement mobilisée dans le présent projet. Ces missions couvrent la conception d'une plateforme interne self-service, l'orchestration de conteneurs sur Kubernetes, l'exploitation en production à fort trafic, l'infrastructure as code avec Terraform, les déploiements applicatifs avec Helm, la gestion des artefacts, les pipelines d'intégration et de livraison continues sur GitLab CI, l'automatisation en Bash et en Go, la supervision des métriques avec Prometheus et la centralisation des journaux avec Loki. Cette double culture,

développement et exploitation, irrigue l'ensemble des choix présentés dans ce dossier.

9.3 Objectifs de la présentation

L'objectif de ce mémoire est de démontrer la capacité à concevoir, automatiser, sécuriser et exploiter une infrastructure DevOps complète, du code à la production, en apportant une réflexion argumentée plutôt qu'une simple description technique. Il s'agit de mettre en évidence les relations entre les besoins fonctionnels, l'architecture, les patterns techniques, les ressources et les contraintes d'exploitation, et de montrer en quoi les décisions prises répondent à la problématique posée.

Ce dossier restitue un cheminement d'ingénieur maîtrisé, où chaque choix, de l'orchestrateur au modèle d'estimation énergétique, est explicité, comparé à ses alternatives et évalué à l'aune de ses conséquences. Il valorise les résultats obtenus et prolonge chaque réalisation par des perspectives d'évolution réalistes, dans une logique d'amélioration continue qui témoigne de la maturité de la démarche.

10. Présentation de la problématique

10.1 Contexte du projet

Le marché du stockage et du partage de médias en ligne est dominé par quelques acteurs majeurs qui proposent des services aboutis mais reposant sur des modèles où la confidentialité et la portabilité des données ne constituent pas la priorité. Dans ce paysage, un besoin persiste, celui de disposer d'un espace où les contenus personnels demeurent privés par défaut, où la publication procède d'un choix explicite et révocable, et où l'utilisateur conserve la maîtrise de son profil public. C'est le positionnement de dropicture, plateforme conçue autour de la confidentialité, de la publication sélective et d'un fil social léger.

Ce positionnement produit engage des exigences techniques directes. La séparation stricte entre contenus privés et publics doit être garantie de bout en bout, du modèle de données jusqu'à la diffusion par le réseau de distribution de contenu. Le service doit rester disponible et performant à mesure que la charge augmente, se déployer sans interruption pour préserver l'expérience, et protéger des données à caractère personnel dans le respect du cadre réglementaire européen. À ces exigences s'ajoute une contrainte transversale de plus en plus prégnante, la sobriété numérique, qui invite à mesurer et à maîtriser l'empreinte environnementale et financière de l'exploitation.

La mission, telle qu'un consultant la recevrait, consiste donc moins à développer une application qu'à bâtir la chaîne industrielle capable de la produire, de la livrer et de l'exploiter de façon fiable, sécurisée et sobre. C'est cette chaîne, l'infrastructure DevOps, qui constitue l'objet central du présent dossier.

10.2 Description de la problématique

Construire un service en ligne moderne confronte l'organisation à une difficulté structurelle, la distance entre le code produit par les développeurs et son exécution en production. Chaque franchissement manuel de cette distance, qu'il s'agisse de provisionner un serveur, de configurer un environnement, de déployer une version ou de surveiller son comportement, représente une source d'erreur, de lenteur et de divergence entre environnements. Multipliées, ces opérations manuelles engendrent

des déploiements risqués, des régressions difficiles à tracer, une exploitation opaque et des coûts mal maîtrisés.

La problématique retenue synthétise cet enjeu sous la forme d'une question centrale.

« Comment concevoir et déployer une infrastructure DevOps entièrement automatisée, sécurisée et éco-responsable, permettant la livraison continue d'une plateforme SaaS de partage de médias tout en garantissant la qualité, l'observabilité et la maîtrise des coûts ? »

Cette question articule plusieurs exigences qu'il convient de traiter conjointement. L'automatisation accélère la livraison tout en préservant la sécurité. L'observabilité soutient le pilotage tout en s'inscrivant dans une consommation de ressources maîtrisée. L'éco-responsabilité et la maîtrise des coûts appellent la sobriété sans renoncer à la résilience ni à la performance. La valeur de la démarche réside dans la conciliation cohérente de ces exigences au sein d'un même système, au-delà des solutions éprouvées qui existent pour chacune prise isolément.

10.3 Objectifs du projet

De la problématique découlent des objectifs opérationnels, qui servent de fil conducteur et de grille d'évaluation. Le premier objectif consiste à décrire l'intégralité de l'infrastructure sous forme de code versionné et reproductible, afin d'éliminer les opérations manuelles et de garantir l'identité entre les environnements. Le deuxième vise à mettre en place une chaîne d'intégration et de livraison continues qui automatise les tests, la construction des artefacts et le déploiement, tout en incorporant des portes de qualité et de sécurité.

Le troisième objectif porte sur l'orchestration d'environnements conteneurisés assurant la portabilité, la scalabilité et la résilience du service, avec des mises à jour sans interruption et un repli automatique en cas d'anomalie. Le quatrième instaure une stratégie d'observabilité couvrant les métriques, les journaux et l'alerting, afin d'anticiper les incidents et d'optimiser les performances. Le cinquième objectif, transversal, inscrit l'ensemble dans une démarche de sécurité par défense en profondeur, de conformité réglementaire et d'éco-responsabilité mesurée, à coût maîtrisé.

10.4 Importance et impact de la problématique

L'enjeu dépasse le cadre du seul projet dropicture. La capacité à livrer vite, souvent et sans risque est devenue un facteur déterminant de compétitivité pour toute organisation produisant du logiciel, et les travaux de référence sur la performance des équipes d'ingénierie établissent une corrélation forte entre la maturité des pratiques d'automatisation et la performance globale. Une infrastructure DevOps bien conçue réduit le délai de mise sur le marché, diminue le taux d'échec des changements et raccourcit le temps de rétablissement après incident.

L'impact est également économique et environnemental. L'automatisation et le dimensionnement au plus juste réduisent le gaspillage de ressources, tandis que la supervision de la consommation permet d'objectiver et de piloter l'empreinte du service. Sur le plan de la sécurité et de la conformité, l'intégration des contrôles dès la conception limite l'exposition aux risques juridiques et réputationnels liés à la protection des données personnelles. La problématique traitée se situe ainsi à l'intersection de préoccupations techniques, économiques, environnementales et réglementaires, ce qui en fait un sujet représentatif des responsabilités d'un expert en systèmes d'information.

10.5 Questions de recherche et hypothèses

La problématique générale se décline en plusieurs questions de recherche qui guident l'investigation. La première interroge la mesure dans laquelle la description intégrale de l'infrastructure en code garantit la reproductibilité et réduit les incidents liés à la configuration. La deuxième examine la façon dont une chaîne de livraison continue concilie rapidité de déploiement et exigence de qualité et de sécurité. La troisième porte sur l'orchestrateur offrant le meilleur compromis entre capacités et simplicité d'exploitation pour un service de cette taille. La quatrième concerne la mesure et la maîtrise de la consommation de ressources d'une infrastructure hébergée dans le cloud, où les compteurs matériels ne sont pas exposés.

À ces questions correspondent des hypothèses de travail. La première pose qu'une automatisation de bout en bout, fondée sur l'infrastructure as code et le GitOps, améliore simultanément la fiabilité et la vitesse des déploiements. La deuxième avance qu'une orchestration adaptée à la taille du cluster garantit portabilité, scalabilité et résilience avec un excellent rapport entre les capacités et la simplicité

d'exploitation. La troisième soutient qu'une stratégie d'observabilité intégrée, complétée d'une estimation de la consommation énergétique, rend l'écoresponsabilité pilotable et compatible avec la maîtrise des coûts. La validation de ces hypothèses est examinée tout au long du dossier et discutée dans la section consacrée aux analyses et perspectives.

11. Analyse et Définition de la Stratégie du Système d'Information

La présente section établit le socle décisionnel de la plateforme dropicture. Elle articule trois démarches complémentaires. La première est une veille technologique structurée, dont l'objet est d'éclairer les choix techniques structurants. La deuxième est une formalisation des besoins fonctionnels aboutissant à un cahier des charges fonctionnel. La troisième est une analyse stratégique du système d'information, comprenant un diagnostic, une lecture des risques et un ensemble de préconisations. L'ensemble répond directement à la problématique retenue, qui consiste à concevoir et déployer une infrastructure DevOps entièrement automatisée, sécurisée et éco-responsable, permettant la livraison continue d'une plateforme SaaS de partage de médias tout en garantissant la qualité, l'observabilité et la maîtrise des coûts. Chaque orientation présentée ci-après est justifiée au regard de six exigences transverses, à savoir l'automatisation, la sécurité, l'éco-responsabilité, la qualité, l'observabilité et la maîtrise des coûts.

11.1 Veille technologique

11.1.1 Introduction

La veille technologique constitue le préalable indispensable à toute décision d'architecture durable. L'écosystème DevOps évolue rapidement, sous l'effet des nouvelles versions de moteurs applicatifs, des mutations des modèles de licence, de la montée en puissance de l'éco-conception numérique et du durcissement du cadre réglementaire. Dans ce contexte, une plateforme conçue sans dispositif de veille s'expose à l'obsolescence, à la dette technique et à des coûts de reprise élevés. Pour la plateforme dropicture, la veille poursuit un double objectif. Il s'agit d'une part de sélectionner, pour chaque brique du système d'information, la technologie la plus adaptée au périmètre réel du projet, et d'autre part d'anticiper les évolutions susceptibles de remettre en cause les choix initiaux. L'enjeu n'est pas d'adopter les outils les plus récents, mais ceux qui offrent le meilleur équilibre entre pérennité, sobriété, sécurité et coût total de possession. Cette démarche irrigue l'intégralité de la problématique. L'automatisation dépend du choix des outils d'infrastructure as code, la sécurité dépend de la maturité des composants retenus, l'éco-responsabilité

repose sur la disponibilité d'outillage de mesure énergétique, et la maîtrise des coûts découle de la sélection d'un hébergeur souverain et abordable. La veille alimente en continu le journal des décisions d'architecture consigné pour le projet (cf. Annexe J).

11.1.2 Méthodologie de la veille technologique

La méthodologie mise en œuvre repose sur trois piliers, à savoir des sources qualifiées, une grille de critères explicites et un dispositif de tri permettant de séparer le signal du bruit. Cette formalisation garantit que les conclusions de la veille sont reproductibles et argumentées, et non le fruit d'un choix intuitif.

Les sources mobilisées sont volontairement diversifiées afin de recouper l'information. En premier lieu figurent les sources primaires, qui regroupent les documentations officielles, les notes de version et les feuilles de route publiques des projets étudiés (NestJS, Next.js, PostgreSQL, Redis, Docker, Terraform, Prometheus). Ces sources permettent de suivre la cadence de publication, les ruptures de compatibilité et l'état de maintenance. En second lieu, les dépôts publics fournissent des indicateurs objectifs de vitalité, comme la fréquence des contributions, le nombre de mainteneurs actifs, le délai de traitement des incidents de sécurité et l'ampleur de la communauté. En troisième lieu, les radars technologiques sectoriels, à l'image du Technology Radar de ThoughtWorks ou de la cartographie de la Cloud Native Computing Foundation, offrent une lecture prospective des tendances et de leur degré de maturité. Les enquêtes annuelles auprès des développeurs complètent cette vision par des données d'adoption. Enfin, un corpus normatif et institutionnel encadre l'analyse. Il comprend les recommandations de l'ANSSI et de la CNIL pour la sécurité et la protection des données, les publications de l'OWASP pour les vulnérabilités applicatives, la norme ISO/IEC 27001 dans sa version 2022 pour le management de la sécurité de l'information, la norme ISO/IEC 25010 pour la qualité logicielle, le Référentiel général d'écoconception de services numériques (RGESN) pour la sobriété, et les référentiels WCAG 2.2 et RGAA 4 pour l'accessibilité.

Sur le plan strictement réglementaire, la veille surveille en continu les grandes orientations susceptibles d'affecter le système d'information. Le Règlement européen sur l'intelligence artificielle (IA Act), adopté en 2024 et d'application progressive, est suivi au titre de l'usage d'outils d'IA dans la chaîne d'ingénierie, car il impose une

vigilance sur la classification des systèmes et leur transparence. Le Cloud Act américain est identifié comme un risque de souveraineté, dans la mesure où il autorise un accès extraterritorial aux données détenues par des fournisseurs de droit américain ; ce constat justifie le choix d'un hébergement de calcul européen (Hetzner) et la localisation des services AWS en région eu-west-3 (Paris), afin de circonscrire l'exposition. S'y ajoutent le RGPD, la directive NIS 2 sur la sécurité des réseaux et des systèmes d'information, et la norme ISO/IEC 27001 version 2022, qui structurent les exigences de sécurité et de conformité du projet.

La grille de critères appliquée à chaque technologie candidate comporte huit axes pondérés. La pérennité et la gouvernance évaluent la solidité du modèle de licence, la santé de la communauté et la lisibilité de la feuille de route. L'adéquation fonctionnelle mesure la couverture du besoin réel sans surdimensionnement. Le coût total de possession intègre non seulement les licences, mais aussi l'exploitation, la formation et la maintenance. La sécurité apprécie la conformité aux exigences ISO/IEC 27001 version 2022 et au Règlement général sur la protection des données (RGPD), ainsi que la surface d'attaque introduite. La sobriété examine la consommation de ressources et la disponibilité d'outils de mesure, en cohérence avec le RGESN. L'accessibilité vérifie la compatibilité avec les référentiels WCAG et RGAA pour les surfaces exposées aux utilisateurs. La courbe d'apprentissage tient compte des compétences déjà maîtrisées, afin de limiter le coût d'appropriation. La réversibilité, enfin, évalue le degré de verrouillage fournisseur et la capacité à migrer sans reprise majeure.

Le tri de l'information obéit à une logique de recoupement. Une tendance n'est retenue que si elle est confirmée par plusieurs sources indépendantes et si sa maturité la place hors de la phase spéculative du cycle d'adoption. Une attention particulière est portée au critère d'obsolescence et de risque de licence, devenu déterminant dans l'écosystème des logiciels d'infrastructure. Deux évolutions récentes illustrent l'importance de cette vigilance. La première est le passage de Terraform sous licence BSL (Business Source License) en 2023, qui a donné naissance au fork communautaire OpenTofu placé sous l'égide de la Linux Foundation. La seconde est la modification du modèle de licence de Redis en 2024, à l'origine du fork Valkey soutenu par plusieurs acteurs majeurs. Ces événements, sans effet opérationnel immédiat pour la plateforme, sont tracés comme points de

vigilance et conditionnent des scénarios de repli documentés. La veille est enfin capitalisée, puisque chaque conclusion structurante donne lieu à une décision formalisée au format Contexte / Décision / Alternatives / Conséquences (cf. Annexe J), ce qui garantit la traçabilité du raisonnement et facilite la réévaluation ultérieure.

11.1.3 Résultats de la veille technologique

Les résultats de la veille sont présentés par domaine du système d'information, du socle applicatif jusqu'à l'hébergement et l'éco-conception. Chaque conclusion est mise en regard des exigences de la problématique.

Concernant le socle applicatif, la veille a confirmé la pertinence d'une pile unifiée autour de TypeScript, ce qui limite la fragmentation des compétences et favorise la réutilisation. Pour les surfaces web, Next.js s'est imposé par sa capacité à couvrir deux besoins distincts avec une seule technologie, à savoir le rendu serveur pour l'application authentifiée, riche et dynamique, et l'export statique pour le site vitrine et les profils publics, hébergeable à très faible coût sur un réseau de diffusion de contenu. Cette dualité sert directement la maîtrise des coûts et la disponibilité. Pour l'interface de programmation, NestJS a été retenu pour son ossature structurante (injection de dépendances, modularité par domaine, pipes de validation), supérieure à celle d'un socle Express nu ou de Fastify seul, tout en conservant d'excellentes performances lorsqu'il est exécuté sous le moteur Bun. Le choix d'une base relationnelle PostgreSQL, exploitée via un mapping objet-relationnel à migrations explicites, a été préféré à une solution à schéma implicite afin de conserver un contrôle fin sur les index, notamment les index partiels qui participent à la sobriété de la base. Redis a été confirmé pour la gestion des sessions opaques, du cache, des verrous et du throttling. La veille sur son évolution de licence a néanmoins conduit à documenter Valkey comme alternative de repli à interface compatible. Ces orientations sont détaillées dans les décisions d'architecture correspondantes (cf. Annexe J).

Sur le plan de l'orchestration, la question centrale opposait Docker Swarm à Kubernetes. La veille a mis en évidence que Kubernetes, bien que devenu le standard de l'industrie, introduit une complexité opérationnelle et un coût, tant humain que financier, disproportionnés pour un cluster de quelques nœuds opéré par une équipe réduite. Les distributions allégées telles que k3s, ou des

ordonnanceurs alternatifs comme Nomad, ont également été examinées. L'analyse conclut au choix de Docker Swarm, intégré nativement au moteur Docker. Celui-ci fournit l'essentiel du besoin (placement par étiquettes de rôle, mises à jour progressives avec retour arrière automatique, secrets et configurations natifs, réseaux overlay segmentés, contrôles de santé et mise à l'échelle horizontale) au prix d'une surcharge opérationnelle minimale. Ce choix sert l'automatisation et la maîtrise des coûts, tout en demeurant réversible, car la veille conserve Kubernetes et k3s comme cibles d'évolution si le trafic ou la taille de l'équipe venaient à croître. Le détail de cette orchestration figure en annexe (cf. Annexe C).

Pour l'infrastructure as code, la veille a confirmé le couple Terraform / Ansible comme référence pour, respectivement, le provisionnement déclaratif et la configuration procédurale idempotente. La séparation nette entre l'approvisionnement des ressources et leur configuration constitue une bonne pratique établie, qui améliore la lisibilité et la reproductibilité de l'infrastructure. Le suivi de l'évolution de licence de Terraform a conduit à identifier OpenTofu comme trajectoire de repli crédible, à interface de configuration compatible, ce qui réduit le risque de verrouillage. Cette architecture entièrement décrite en code est le premier levier de l'automatisation attendue par la problématique (cf. Annexe B).

En matière d'observabilité, la veille a comparé les solutions auto-hébergées aux offres en mode service. Les plateformes commerciales telles que Datadog ou New Relic offrent une intégration rapide, mais induisent un coût récurrent croissant avec le volume et impliquent une sortie des données hors de l'Union européenne, en tension avec les exigences de souveraineté et de RGPD. La pile ELK/EFK a été jugée trop consommatrice de ressources au regard du besoin. La combinaison Prometheus, Alertmanager, Loki, Alloy et Grafana s'est révélée la plus adaptée, car elle couvre métriques, journaux et alerting dans une interface unique, sans coût de licence, avec des configurations et des tableaux de bord entièrement versionnés. Cette stack d'observabilité répond aux exigences d'observabilité et de qualité, tout en maintenant la souveraineté des données (cf. Annexe P).

Concernant l'hébergement, la veille a écarté un recours exclusif à un hyperscaler, coûteux et à la souveraineté floue, ainsi qu'une approche entièrement en plateforme managée, moins riche sur le plan de l'apprentissage DevOps et moins maîtrisable en coût. L'architecture retenue combine trois fournisseurs selon leurs forces respectives.

Hetzner Cloud, hébergeur européen au rapport performance-prix élevé, assure le calcul. Cloudflare prend en charge la périphérie (DNS, pare-feu applicatif, TLS strict). Amazon Web Services, en région européenne, héberge le stockage objet des médias et des sauvegardes ainsi que leur diffusion. Ce montage multi-fournisseurs sert la maîtrise des coûts, la souveraineté et la résilience, et il s'accompagne d'une gestion de secrets répartie entre les fournisseurs, suivie comme un point de vigilance maîtrisé.

Enfin, la veille sur l'éco-conception a constitué un axe à part entière, en réponse directe à l'exigence d'éco-responsabilité. Elle a porté sur les référentiels applicables, dont le RGEN et les travaux du collectif GreenIT, ainsi que sur l'outillage de mesure énergétique. Il en ressort que la mesure directe par les compteurs matériels RAPL demeure propre aux serveurs physiques, sur lesquels des outils tels que Scaphandre ou Kepler fournissent une mesure fiable. Ce constat a orienté la plateforme vers un modèle d'estimation de la puissance dérivé de l'utilisation processeur, documenté et intégré à la stack d'observabilité, qui offre une base pragmatique ouvrant la perspective d'une mesure réelle sur matériel dédié. La veille a par ailleurs conforté plusieurs leviers de sobriété éprouvés, à savoir le dimensionnement au plus juste des ressources, le cycle de vie du stockage vers des paliers moins énergivores, le cache immuable et la compression pour réduire les transferts, et le choix d'une région d'hébergement au mix électrique favorable.

11.1.4 Recommandations et perspectives

Au terme de cette veille, plusieurs recommandations se dégagent pour consolider et faire évoluer le système d'information. Sur le plan de la sécurité de la chaîne logicielle, les analyses automatisées préconisées par la veille ont été intégrées au pipeline et sont aujourd'hui opérationnelles. La chaîne exécute, en portes bloquantes sur les vulnérabilités critiques et élevées, une analyse des dépendances et des secrets du dépôt, une analyse statique des plans d'infrastructure (Terraform, Ansible, Dockerfile) et une analyse des images de conteneurs, complétées par la production d'une nomenclature logicielle (SBOM) au format CycloneDX pour chaque image publiée. Cette réalisation matérialise le passage d'une sécurité par conception à une démarche DevSecOps effective (cf. Annexe S). En matière d'éco-responsabilité, la trajectoire recommandée consiste à substituer, sur un éventuel déploiement sur matériel dédié, une mesure énergétique réelle à l'estimation actuelle, afin d'affiner le

pilotage de l'empreinte carbone. Sur le plan de la résilience, les sauvegardes et leur restauration sont désormais automatisées et rejouables, la veille invitant à poursuivre par la mise en haute disponibilité de la stack d'observabilité. Concernant la livraison, des stratégies de déploiement plus fines, comme le déploiement canari ou bleu-vert, constituent une perspective d'amélioration de la qualité de service. Enfin, l'exploitation assistée par intelligence artificielle, pour la prédiction d'incidents et l'optimisation du dimensionnement, représente une piste prospective à évaluer. L'ensemble de ces orientations est priorisé dans une feuille de route d'amélioration continue, articulée autour d'une boucle mesurer-identifier-prioriser-implémenter-mesurer.

11.1.5 Conclusion

La veille technologique a permis d'ancrer chaque choix structurant de la plateforme dropicture dans une analyse argumentée, recoupée et tracée. Loin d'une simple sélection d'outils, elle a fourni une grille de lecture pérenne, sensible aux risques de licence, aux exigences réglementaires et aux impératifs de sobriété. Les conclusions retenues, à savoir une pile TypeScript unifiée, une orchestration par Docker Swarm, une infrastructure as code Terraform et Ansible, une observabilité auto-hébergée et un hébergement multi-fournisseurs souverain, convergent vers les exigences de la problématique et fournissent une base stable, tout en préservant des trajectoires d'évolution explicites. Cette veille irrigue naturellement l'expression des besoins, objet de la section suivante.

11.2 Besoins fonctionnels et cahier des charges fonctionnel

Cette section traduit la vision produit en exigences fonctionnelles vérifiables, condition première d'un système d'information utile et livrable en continu. La formalisation des besoins conditionne en effet la qualité attendue par la problématique, car sans expression précise, aucune stratégie de tests ni aucun critère d'acceptation ne peut être défini de manière objective. Les fonctionnalités décrites ci-après constituent le périmètre du produit minimum viable et de ses extensions, et se retrouvent modélisées dans les cas d'utilisation du projet (cf. Annexe G) et dans le contrat d'interface exposé par l'interface de programmation (cf. Annexe Q).

11.2.1 Description des fonctionnalités requises

La plateforme dropicture répond à un besoin clairement identifié, celui d'offrir un partage de photographies et de vidéos maîtrisé, fondé sur le principe de confidentialité par défaut. Toute donnée déposée est privée, et l'utilisateur décide explicitement de ce qu'il rend public. Ce positionnement, différenciant vis-à-vis des plateformes grand public, structure l'ensemble des fonctionnalités.

Les fonctionnalités essentielles constituent le cœur du produit minimum viable. La première est l'authentification et la gestion de compte, qui couvre l'inscription, la connexion, le maintien de session et la déconnexion. Cette fonction répond à une exigence de sécurité forte, matérialisée par un hachage robuste des mots de passe, des sessions opaques adossées à un cache, une rotation des jetons et une détection de rejeu. La deuxième fonctionnalité essentielle est le téléversement de médias en flux continu, où l'utilisateur dépose des fichiers volumineux qui sont acheminés directement vers le stockage objet sans saturer la mémoire du serveur, avec un contrôle de taille et de quota. La troisième est la bibliothèque privée, espace personnel où l'utilisateur consulte, liste et gère l'intégralité de ses médias, publiés ou non. La quatrième est l'organisation en albums, qui permet de regrouper et d'ordonner les médias, avec la notion de placement ordonné et de média de couverture. La cinquième, centrale au positionnement du produit, est la publication sélective, par laquelle l'utilisateur choisit explicitement de publier ou de dépublier un média, opération qui date ou réinitialise son statut de publication et déclenche, le cas échéant, l'invalidation du cache de diffusion. La sixième est le profil public, accessible via une adresse dédiée, exposant la sélection publiée, la biographie et l'avatar de l'utilisateur. La septième est le fil social léger, articulé autour des abonnements, où l'utilisateur suit d'autres créateurs et consulte un fil, public ou restreint à ses abonnements. La huitième, enfin, est la recherche de profils, qui assure la découvrabilité au sein de la plateforme. Ces huit fonctions, modélisées dans le diagramme de cas d'utilisation (cf. Annexe G), constituent le contrat fonctionnel minimal et sont couvertes par la couverture de tests automatisés du projet.

Les fonctionnalités secondaires enrichissent l'expérience sans conditionner la viabilité du produit. Parmi elles figure la gestion d'une clé d'interface de programmation, émise dès l'inscription, qui ouvre un accès programmatique

complémentaire à l'authentification par session. S'y ajoutent la gestion fine des paramètres de compte, incluant la modification de l'identifiant, de l'adresse électronique et du mot de passe, ainsi que la gestion de l'avatar et de la biographie. Viennent également le téléchargement de ses propres médias, l'exposition de statistiques publiques et la suppression complète du compte. Cette dernière fonction, bien que secondaire du point de vue de l'usage courant, revêt une importance juridique majeure, car elle matérialise le droit à l'effacement prévu par le RGPD, en supprimant le compte, ses médias par cascade en base et sur le stockage objet, et en invalidant les sessions associées. La distinction entre fonctionnalités essentielles et secondaires a guidé la priorisation du développement et le séquençement des livraisons, dans une logique de livraison continue de valeur.

11.2.2 Collecte et analyse des besoins

La formalisation des besoins s'inscrit dans une situation professionnelle reconstituée, conforme au référentiel de certification, où le projet dropicture sert de support à une mission de conseil. La démarche de collecte, d'analyse, d'identification des contraintes et de validation est décrite ci-après.

La méthodologie de collecte a combiné plusieurs techniques complémentaires. Une analyse du positionnement produit a d'abord permis de circonscrire la proposition de valeur, le partage maîtrisé et confidentiel, et de la distinguer des plateformes existantes. Cette vision a été déclinée en scénarios d'usage, formalisés sous forme de cas d'utilisation distinguant deux acteurs. Le visiteur voit son périmètre couvrir les fonctions publiques d'inscription, de connexion, de consultation et de recherche. Le membre hérite des capacités du visiteur et accède aux fonctions authentifiées de téléversement, d'organisation, de publication, de suivi et de gestion de compte (cf. Annexe G). La collecte s'est appuyée sur des critères d'acceptation explicites, à savoir les préconditions et les résultats attendus pour chaque cas, qui constituent la base de la vérifiabilité ultérieure des exigences.

L'analyse approfondie des besoins a ensuite consisté à traduire ces scénarios en exigences fonctionnelles et non fonctionnelles. Les exigences fonctionnelles ont été rattachées à des domaines cohérents (authentification, bibliothèque, profil, découverte, paramètres et interface publique) qui structurent également l'organisation de l'interface de programmation (cf. Annexe Q). Cette décomposition

par domaine assure une correspondance directe entre le besoin exprimé, l'exigence formalisée et le composant logiciel qui la réalise, ce qui facilite la traçabilité et la couverture de tests. Les exigences non fonctionnelles ont fait l'objet d'une attention équivalente, car elles portent l'essentiel de la problématique, notamment la performance du téléversement en flux continu, la sécurité de l'authentification, la sobriété des ressources, la disponibilité du service et le respect de la vie privée.

L'identification des contraintes et exigences a mis en évidence plusieurs invariants structurants. La contrainte de confidentialité par défaut impose que tout média soit privé jusqu'à publication explicite, ce qui se répercute sur le modèle de données, la conception des points d'accès et la gestion du cache de diffusion. La contrainte réglementaire, tirée du RGPD, impose la minimisation des données collectées, le droit à l'effacement, le chiffrement et un hébergement au sein de l'Union européenne. La contrainte de sécurité, cartographiée sur la norme ISO/IEC 27001 version 2022, impose une défense en profondeur couvrant la périphérie, le réseau, l'application et les données (cf. Annexe R). La contrainte d'accessibilité impose que les surfaces exposées soient utilisables par les personnes en situation de handicap (PSH), conformément aux référentiels WCAG 2.2 et RGAA 4, ce qui se traduit par une structure sémantique explicite, une hiérarchie de titres cohérente, des champs de formulaire étiquetés, des alternatives textuelles pour les médias, une navigation intégralement possible au clavier avec un focus visible, des contrastes suffisants et la déclaration de la langue du document, cette exigence s'étendant à la documentation technique et aux comptes rendus de projet. La contrainte d'éco-responsabilité impose la sobriété des ressources et la mesurabilité de la consommation. Enfin, une contrainte technique forte, propre au produit, découle de l'hébergement statique du site public. Les profils publics doivent demeurer consultables via une architecture sans serveur applicatif dédié, ce qui a orienté la conception vers une interface publique dédiée consommée par un site exporté statiquement. Ces contraintes, loin d'être subies, ont été érigées en exigences de conception et tracées dans le registre des risques du projet lorsqu'elles portaient un enjeu de criticité (cf. Annexe O).

La validation avec les parties prenantes clôt la démarche. Dans le cadre reconstitué de la mission, les parties prenantes identifiées sont au nombre de quatre. Le porteur du produit est garant de la vision et de la priorisation. Les utilisateurs cibles, créateurs soucieux de la maîtrise de leur contenu, ont vu leurs attentes traduites en

scénarios. Le sponsor du projet est attentif à la maîtrise des coûts et au respect des délais. Le référent en charge de la protection des données est garant de la conformité au RGPD. La validation a pris la forme de revues successives. La revue des cas d'utilisation a confirmé le périmètre fonctionnel, la revue des exigences non fonctionnelles a permis d'arbitrer les priorités, et la revue de conformité a vérifié l'adéquation aux contraintes réglementaires. Les décisions structurantes issues de ces revues ont été consignées dans le journal d'architecture (cf. Annexe J), assurant leur traçabilité. Cette validation itérative a permis de figer un cahier des charges fonctionnel stable, décliné en cas d'utilisation vérifiables, qui sert de référence contractuelle pour la conception, le développement et la stratégie de tests. Le suivi du projet s'appuie sur des référents clairement identifiés, à savoir Joris VERGULDEZOONE, référent de formation, et Philippe AMERIO, référent en entreprise.

Ainsi formalisés, les besoins fournissent une assise solide à l'analyse stratégique, qui confronte cette expression fonctionnelle à l'état réel du système d'information et aux risques qu'il porte.

11.3 Analyse stratégique

11.3.1 Introduction

L'analyse stratégique a pour objet de porter un regard d'ensemble sur le système d'information de la plateforme dropicture, afin de déterminer les orientations les plus à même de servir durablement la problématique. Elle s'appuie sur le diagnostic de l'existant, la lecture des risques et l'expression des besoins pour définir une stratégie informatique cohérente, puis la décliner en préconisations argumentées. Cette démarche adopte la posture d'un conseil externe, car elle met en balance les atouts et les points de vigilance, apprécie les opportunités et les menaces, et hiérarchise les actions au regard de leur valeur et de leur criticité.

11.3.2 Diagnostic du système d'information (SI)

Le diagnostic du système d'information est conduit selon une lecture stratégique de type forces, points de vigilance, opportunités et menaces, appliquée à l'architecture réellement mise en œuvre, dont la cartographie complète est fournie en annexe (cf. Annexe A).

Cette grille correspond à une analyse SWOT (forces, faiblesses, opportunités, menaces), complétée d'une lecture de type PESTEL pour les facteurs d'environnement, qu'il s'agisse des évolutions politiques et réglementaires comme l'IA Act, des tendances économiques pesant sur les coûts du cloud ou des mutations technologiques affectant les modèles de licence des logiciels d'infrastructure. Le croisement de ces deux méthodes garantit que le diagnostic couvre aussi bien les facteurs internes du système d'information que son macro-environnement.

Les forces du système d'information sont substantielles et directement alignées sur la problématique. La première est le degré élevé d'automatisation. L'infrastructure est intégralement décrite en code, du provisionnement des serveurs jusqu'à la configuration du système d'exploitation, de l'orchestration et du déploiement applicatif, selon une séparation claire entre Terraform et Ansible (cf. Annexe B). La chaîne CI/CD enchaîne de manière reproductible les contrôles de qualité, la gestion des versions, l'application des changements d'infrastructure, la construction des images, l'approvisionnement conditionnel et le déploiement avec exécution des migrations (cf. Annexe K). La deuxième force est la robustesse de la sécurité, conçue en défense en profondeur. Elle repose sur un filtrage en périphérie par pare-feu applicatif et TLS strict, une restriction du pare-feu aux seules adresses de la périphérie, un isolement du socket de conteneurisation par un mandataire dédié, une validation systématique des entrées, un contrôle du débit, un hachage fort des mots de passe, des sessions opaques à rotation avec détection de rejeu, et un chiffrement des données au repos et en transit. Cette architecture est cartographiée sur la norme ISO/IEC 27001 version 2022 et sur le RGPD (cf. Annexe R). La troisième force est la maturité de l'observabilité. La stack auto-hébergée fournit métriques, journaux, tableaux de bord et alerting, avec un ensemble de règles couvrant la disponibilité, la charge, la saturation et les taux d'erreur (cf. Annexe P). La quatrième force, plus rare dans des projets de cette taille, est l'intégration native de l'éco-responsabilité, à travers un dimensionnement au plus juste, une gestion du cycle de vie du stockage et un suivi de la consommation énergétique estimée. La cinquième force est la maîtrise des coûts, obtenue par le choix d'un hébergeur européen économique, par la maîtrise du périmètre de diffusion et par la mise en place d'alertes budgétaires.

Les points de vigilance du système d'information sont identifiés avec la même lucidité, gage d'une amélioration continue crédible, et chacun s'accompagne déjà

d'une mesure de maîtrise. Le premier concerne la concentration de fonctions sur le nœud de périphérie, qui réunit le reverse proxy, le cache et la stack d'observabilité, ainsi que l'orchestrateur opéré par un gestionnaire unique. Compte tenu de l'impact opérationnel de ce nœud, sa maîtrise est assurée à ce stade par la persistance des données sur disque et par un plan de reconstruction intégralement piloté par le code, et il constitue un axe prioritaire d'évolution vers la haute disponibilité. Le deuxième porte sur la couverture des tests, dont la mesure systématique en intégration continue et la publication d'un seuil minimal restent à instaurer, alors même que la suite automatisée est déjà exécutée en porte bloquante. Le troisième, inhérent au montage multi-fournisseurs, concerne la gestion répartie des secrets, maîtrisée par leur stockage hors du dépôt de code et par le moindre privilège, et appelée à être centralisée à mesure que la plateforme grandit. Le quatrième, corollaire assumé du choix d'orchestration, tient à un écosystème plus ciblé que celui de Kubernetes, la mise à l'échelle horizontale restant assurée et l'évolution vers une mise à l'échelle automatique demeurant une trajectoire ouverte. Deux points de vigilance antérieurs ont en revanche été levés au cours du projet, à savoir l'absence d'analyses de sécurité automatisées, désormais couverte par une chaîne DevSecOps complète, et l'absence de dispositif de restauration éprouvé, désormais couverte par des sauvegardes automatisées et un mécanisme de restauration outillé (cf. Annexe S).

Les opportunités identifiées ouvrent des marges de progression significatives. La disponibilité d'outils de mesure énergétique réelle sur matériel dédié ouvre la voie à un pilotage plus fin de l'empreinte carbone. L'exploitation des nomenclatures logicielles (SBOM) déjà produites par la chaîne permet d'envisager une veille continue sur les vulnérabilités des composants embarqués, au-delà du seul instant de la construction. La réversibilité préservée des choix, avec des trajectoires vers OpenTofu, vers Valkey et vers une orchestration Kubernetes allégée, offre des évolutions sans reprise majeure. Enfin, l'exploitation assistée par intelligence artificielle constitue une opportunité d'anticipation des incidents et d'optimisation du dimensionnement.

Les menaces relèvent principalement de l'environnement et font l'objet d'une veille active assortie de réponses documentées. Les évolutions de modèles de licence des logiciels d'infrastructure, déjà observées, sont anticipées par des scénarios de repli tels que les trajectoires vers OpenTofu et Valkey. La dépendance à des fournisseurs

tiers est couverte par la reproductibilité de l'infrastructure par le code, qui autorise un redéploiement rapide en cas d'interruption ou de variation tarifaire. Le durcissement continu du cadre réglementaire est pris en charge par une vigilance permanente sur la conformité. La pression constante des menaces de sécurité est traitée par une actualisation régulière des mesures. Ce diagnostic, qui synthétise les atouts, les points de vigilance, les opportunités et les menaces, fournit la base rationnelle de la stratégie et se prolonge naturellement par une analyse des risques.

11.3.3 Gestion des risques

La gestion des risques constitue le pivot entre le diagnostic et la stratégie, en hiérarchisant les incertitudes selon leur probabilité et leur impact. La plateforme dropicture s'appuie sur un registre des risques formalisé, assorti d'une matrice probabilité-impact et de plans de mitigation, dont le détail complet figure en annexe (cf. Annexe O). La présente section en restitue la logique et les enseignements sans en reproduire le tableau.

La démarche d'analyse de risques s'appuie explicitement sur les méthodes de référence du domaine, à savoir EBIOS Risk Manager, publiée par l'ANSSI, et la norme ISO/IEC 27005. Elle en reprend la logique d'appréciation par sources de risque, événements redoutés et scénarios, puis d'estimation du couple vraisemblance-gravité, avant de retenir une option de traitement (éviter, réduire, transférer ou accepter). Appliquée à la cartographie du système d'information (cf. Annexe A), cette méthode permet d'anticiper les besoins de sécurité du projet et d'ancrer le registre des risques (cf. Annexe O) dans un cadre normatif reconnu.

L'analyse retient une échelle croisant la probabilité et l'impact sur trois niveaux, dont le produit détermine une criticité. Deux risques ressortent comme prioritaires. Le premier est la perte du nœud de périphérie, qui concentre plusieurs fonctions critiques. Son impact élevé et sa probabilité non négligeable en font un risque majeur, d'ores et déjà atténué par la persistance des données, un plan de reconstruction et des sauvegardes, et dont le traitement de fond passe par une évolution vers la haute disponibilité. Le second risque prioritaire est le vol ou le rejeu de session, dont la maîtrise repose sur un cookie strictement protégé, une rotation des jetons par valeur unique, une détection de rejeu et un mécanisme de révocation globale. Les autres risques identifiés, à savoir la fuite de secrets, le dépassement

des coûts cloud, la perte de données, l'indisponibilité d'un fournisseur, la régression lors d'un déploiement et la non-conformité au RGPD, présentent une criticité intermédiaire et font l'objet de plans de mitigation adaptés. La fuite de secrets est contrée par leur stockage hors du dépôt de code et par le moindre privilège. Le dépassement de coûts est prévenu par des alertes budgétaires, un dimensionnement au plus juste et une gestion du cycle de vie du stockage. La perte de données est couverte par des sauvegardes versionnées et verrouillées. L'indisponibilité d'un fournisseur est traitée par la reproductibilité de l'infrastructure par le code, qui autorise un redéploiement rapide. La régression de déploiement est maîtrisée par les tests en intégration, l'immutabilité des images et le déploiement progressif avec retour arrière automatique. La non-conformité réglementaire est écartée par la minimisation, le droit à l'effacement, le chiffrement et l'hébergement souverain. Cette lecture des risques irrigue directement la priorisation de la feuille de route et confirme que les axes d'évolution les plus prioritaires, à savoir la haute disponibilité de l'observabilité et le test de restauration, coïncident avec les risques de criticité la plus élevée.

11.3.4 Élaboration de la stratégie informatique

Avant d'arrêter les axes de la stratégie, il convient de justifier le style architectural retenu. Quatre grands types d'architecture ont été comparés au regard des exigences de la problématique, à savoir le monolithe modulaire, l'architecture en microservices, l'architecture sans serveur (serverless) et l'architecture orientée événements. Chacun a été évalué sur ses caractéristiques propres, ses cas d'usage typiques, sa performance, sa scalabilité, sa sécurité, son coût total de possession et son empreinte de ressources, ces deux derniers critères découlant directement des exigences de maîtrise des coûts et d'éco-responsabilité.

Type d'architecture	Cas d'usage typique	Avantages	Inconvénients	Verdict pour dropicture
Monolithe modulaire	Produit à périmètre maîtrisé, équipe réduite, domaine cohérent	Déploiement unitaire simple, transactions	Couplage à surveiller, mise à l'échelle par	Retenu. Le découpage par domaine (un contrôleur

		ACID natives, latence interne nulle, empreinte de ressources minimale, coût d'exploitation faible	bloc, risque d'érosion des frontières sans discipline	par périmètre) préserve la modularité sans payer le coût du distribué
Microservices	Organisations à plusieurs équipes, domaines hétérogènes, cycles de livraison indépendants	Déploiement et mise à l'échelle indépendants, isolation des pannes, liberté technologique par service	Complexité opérationnelle forte, cohérence distribuée, latence réseau, surcoût de calcul permanent, observabilité plus lourde	Écarté. Le surcoût opérationnel et énergétique est disproportionné pour une équipe réduite et cinq entités métier
Serverless (FaaS)	Charges intermittentes, pics imprévisibles, traitements événementiels courts	Aucune infrastructure à opérer, facturation à l'usage, mise à l'échelle automatique	Démarrages à froid, verrouillage fournisseur marqué, difficulté du téléversement en flux, souveraineté	Écarté. Incompatible avec le téléversement de médias volumineux en flux et avec l'objectif d'apprentissage

			et coûts peu prévisibles à charge soutenue	ge et de maîtrise de l'infrastructure
Orientée événements	Flux asynchrones, forte volumétrie, découplage producteur/consommateur	Découplage temporel, absorption des pics, extensibilité par abonnement	Courbe de complexité élevée, cohérence à terme, traçabilité et débogage plus difficiles, courtier supplémentaire à exploiter	Écarté à ce stade. Pertinent en cible pour le post-traitement des médias (vignettes, transcodage), non pour le cœur transactionnel

Cette comparaison éclaire les interactions entre composants autant que le choix lui-même. L'architecture retenue est un monolithe modulaire déployé en conteneurs, dont les parties (proxy d'entrée, applications sans état, base de données et son mandataire de connexions, cache, stockage objet) sont séparées par des réseaux cloisonnés et évaluées individuellement, tant sur leurs aspects techniques (vulnérabilités, performances, limitations) que sur leurs aspects normatifs et budgétaires. La priorisation des inconvénients a conduit à traiter en premier le couplage potentiel, neutralisé par la décomposition par domaine et l'injection de dépendances, puis la mise à l'échelle par bloc, traitée par la réplication horizontale des nœuds applicatifs sans état. Ce choix conserve par ailleurs sa réversibilité, puisque l'extraction ultérieure d'un domaine en service autonome, ou l'ajout d'un traitement événementiel pour le post-traitement des médias, reste possible sans refonte, les frontières de domaine étant déjà matérialisées dans le code (cf. Annexes A, B, C et H).

La stratégie informatique de la plateforme dropicture se construit en réponse au diagnostic et à la lecture des risques, autour de trois axes structurants qui recouvrent l'intégralité de la problématique. Elle vise à transformer les forces constatées en avantages durables et à traiter méthodiquement les points de vigilance identifiés.

Le premier axe est l'automatisation intégrale sous le principe « tout en code ». L'infrastructure, la configuration, les pipelines et l'observabilité sont décrits de manière déclarative et versionnés, faisant du dépôt de code l'unique source de vérité selon une approche GitOps. Cette orientation garantit la reproductibilité, réduit l'écart entre les environnements, et sert de fondation à la livraison continue exigée par la problématique. Elle se matérialise par des images immuables identifiées par empreinte de commit, par un déploiement progressif sans interruption avec retour arrière automatique, et par des migrations exécutées de façon contrôlée après le déploiement (cf. Annexe K).

Le second axe est la sécurité intégrée, dans une démarche DevSecOps désormais effective. La stratégie consiste à déplacer les contrôles au plus tôt dans le cycle (validation d'entrée, gestion des secrets, durcissement du système et du réseau), puis à faire porter par la chaîne elle-même des analyses automatiques positionnées en portes bloquantes, à savoir l'analyse des dépendances et des secrets, l'analyse statique des plans d'infrastructure et l'analyse des images de conteneurs, assortie d'une nomenclature logicielle par artefact publié. Cette orientation s'inscrit dans le cadre de la norme ISO/IEC 27001 version 2022 et du RGPD, dont la couverture est cartographiée (cf. Annexe R), et sa mise en œuvre est détaillée en annexe (cf. Annexe S).

Le troisième axe est l'éco-responsabilité mesurée, ou green IT. La stratégie érige la sobriété en critère de conception permanent, à travers le dimensionnement au plus juste des ressources, la gestion du cycle de vie du stockage vers des paliers moins énergivores, la réduction des transferts par le cache et la compression, et le choix d'une région à mix électrique favorable. Elle assortit ces leviers d'une mesure, à savoir l'estimation de la consommation énergétique intégrée à la stack d'observabilité, appelée à évoluer vers une mesure réelle sur matériel dédié (cf. Annexe P). Cet axe distingue la plateforme en faisant de l'éco-responsabilité non un affichage, mais une donnée pilotée. Ces trois axes sont complétés par une exigence

transverse d'observabilité de premier ordre et de maîtrise des coûts, présentes dès la conception et non ajoutées après coup.

11.3.5 Présentation des préconisations

Les préconisations qui suivent traduisent la stratégie en actions priorisées, hiérarchisées selon leur contribution à la réduction des risques les plus critiques et selon leur effort de mise en œuvre. Chacune est chiffrée en charge et en coût, sur la base du taux journalier moyen de référence de 500 € retenu pour le projet, afin que l'arbitrage puisse être conduit sur le rapport entre la valeur apportée et la dépense consentie. Elles s'inscrivent dans une boucle d'amélioration continue mesurer-identifier-prioriser-implémenter-mesurer.

Préconisation	Exigence servie	Priorité	Charge (j-h)	Coût estimé	Coût récurrent
Mise en haute disponibilité du nœud de périphérie et de la stack d'observabilité	Résilience, observabilité	Haute	8	4 000 €	+ 24 € / mois (nœud supplémentaire)
Mesure de couverture de tests en intégration continue avec seuil bloquant	Qualité	Haute	2	1 000 €	—
Exercice périodique de restauration de bout en bout et suivi d'un objectif de point de	Résilience, conformité	Moyenne	3	1 500 €	négligeable (exécution planifiée)

reprise					
Raccordement effectif des canaux de notification d'alerte	Observabilité	Moyenne	1	500 €	—
Mesure énergétique matérielle (RAPL) sur environnement dédié	Éco-responsabilité	Basse	5	2 500 €	selon offre bare-metal retenue
Stratégies de déploiement canari ou bleu-vert	Qualité de service	Basse	6	3 000 €	—
Évaluation d'une exploitation assistée par intelligence artificielle	Observabilité, coûts	Basse	4	2 000 €	à cadrer selon l'outil
Total	—	—	29	14 500 €	—

La lecture de ce chiffrage confirme la cohérence entre priorité et effort. Les deux préconisations de priorité haute, qui répondent au risque le plus critique du registre (cf. Annexe O) et consolident la garantie de qualité, représentent ensemble dix jours-homme, soit 5 000 €, pour un tiers seulement de l'enveloppe totale. La trajectoire complète d'amélioration continue s'établit à vingt-neuf jours-homme, soit 14 500 €, à rapporter aux soixante-neuf jours-homme et 34 500 € du projet initial, ce qui situe l'effort de consolidation à environ 42 % de l'effort de construction. Le suivi de ces

actions s'appuie sur des indicateurs objectifs, à savoir le taux de réussite des déploiements, le délai moyen de rétablissement, la fréquence de déploiement, le taux d'erreurs, la latence, la puissance et l'énergie estimées, la couverture de tests et le nombre de vulnérabilités critiques ouvertes.

La première préconisation, de priorité haute, est l'évolution du nœud de périphérie vers la haute disponibilité, par une montée en résilience de la stack d'observabilité et de l'orchestration. Elle répond au risque de criticité la plus élevée du registre (cf. Annexe O) et conditionne la robustesse de la visibilité opérationnelle. La deuxième préconisation, également prioritaire, est le renforcement de la chaîne par des analyses de sécurité automatisées en portes bloquantes, qui matérialise la trajectoire DevSecOps et renforce la couverture des vulnérabilités. La troisième préconisation est l'instauration d'une mesure de couverture de tests en intégration continue, assortie d'un seuil minimal, afin de piloter la couverture de test dans la durée et de consolider la garantie de qualité. La quatrième préconisation est la réalisation régulière d'un test de restauration des sauvegardes, qui transforme un dispositif théorique en garantie éprouvée et complète le plan de reprise d'activité. La cinquième préconisation, d'horizon plus lointain, est le raccordement effectif des canaux de notification d'alerte, l'adoption d'une mesure énergétique réelle sur matériel dédié, l'introduction de stratégies de déploiement plus fines et l'évaluation d'une exploitation assistée par intelligence artificielle. Chacune de ces préconisations est argumentée par sa contribution directe à l'une des exigences de la problématique, la résilience et l'observabilité pour la première, la sécurité pour la deuxième, la qualité pour la troisième et la quatrième, l'éco-responsabilité et la qualité de service pour la cinquième, ainsi que par le rapport entre la valeur apportée et l'effort requis. Le suivi de ces actions s'appuie sur des indicateurs objectifs, à savoir le taux de réussite des déploiements, le délai moyen de rétablissement, la fréquence de déploiement, le taux d'erreurs, la latence, la puissance et l'énergie estimées, la couverture de tests et le nombre de vulnérabilités critiques ouvertes.

11.3.6 Conclusion

L'analyse stratégique confirme que le système d'information de la plateforme dropicture repose sur des fondations solides et cohérentes avec la problématique, à savoir une automatisation intégrale par l'infrastructure as code et la chaîne CI/CD, une sécurité en défense en profondeur cartographiée sur les référentiels et

prolongée par des analyses automatisées bloquantes, une observabilité mature, un dispositif de sauvegarde et de restauration outillé et une éco-responsabilité mesurée, le tout dans une enveloppe de coûts maîtrisée. Le diagnostic a par ailleurs mis en évidence des axes d'amélioration continue clairement identifiés et chiffrés, au premier rang desquels l'évolution vers la haute disponibilité et le renforcement des garanties de test, dont le traitement est hiérarchisé par la lecture des risques (cf. Annexe O) et décliné en préconisations argumentées. La stratégie définie, articulée autour de l'automatisation, de la sécurité intégrée et du green IT, trace une trajectoire claire d'amélioration continue, préservant à chaque étape la réversibilité des choix. Cette stratégie fournit le cadre de référence des sections suivantes, consacrées à la mise en œuvre technique de l'infrastructure, à la conduite du projet et à la conception détaillée du système.

12. Implémentations techniques

La présente section décrit les choix d'implémentation qui transforment les orientations stratégiques de la plateforme dropicture en un système exploitable en production. Elle articule quatre volets (l'infrastructure et son environnement de déploiement, les technologies et outils mobilisés, la surveillance de la sécurité, puis les dispositifs de sauvegarde et de reprise) en conservant pour fil conducteur la problématique du projet, à savoir concevoir et déployer une infrastructure DevOps entièrement automatisée, sécurisée et éco-responsable, au service d'une livraison continue maîtrisée en coûts. Chaque décision technique y est ancrée dans l'implémentation réelle et renvoie aux annexes pour les schémas correspondants, le corps du mémoire restant volontairement dépourvu de figures.

12.1 Infrastructure et environnement de déploiement

Le socle d'exécution de dropicture répond à un double impératif. Il doit offrir une capacité de calcul suffisante pour un service de partage de médias, tout en restant intégralement descriptible et reconstituable par le code. L'infrastructure n'est jamais configurée manuellement. Elle est déclarée, versionnée puis appliquée, ce qui conditionne à la fois l'automatisation revendiquée par la problématique et la reproductibilité indispensable en cas de sinistre.

12.1.1 Infrastructure matérielle

L'infrastructure matérielle repose sur un cluster **Docker Swarm** constitué de machines virtuelles louées chez **Hetzner Cloud** et organisées par **rôle fonctionnel**, avec un nœud *proxy*, un nœud *db*, un ou plusieurs nœuds *backend* et un ou plusieurs nœuds *frontend* (cf. Annexe B). Cette spécialisation, matérialisée par des étiquettes Swarm (`node.labels.dropicture.role`), permet d'affecter à chaque machine les seuls services qui lui incombent et d'isoler nettement les responsabilités. La base de données et son mandataire de connexions résident sur le nœud *db*. Le proxy d'entrée, le cache Redis et l'ensemble de la pile d'observabilité sont regroupés sur le nœud *proxy*, tandis que les processus applicatifs sont répartis sur les nœuds *backend* et *frontend*. Une contrainte de placement lie ainsi chaque conteneur à la classe de nœud adéquate, ce qui garantit un comportement déterministe des

ordonnancements et évite qu'un service critique ne migre vers une machine sous-dimensionnée.

L'ensemble des serveurs est provisionné par **Terraform**, qui dérive dynamiquement la liste des machines à créer à partir d'une déclaration paramétrique associant, pour chaque rôle, un type d'instance et un nombre de réplicas. Ce mécanisme rend le dimensionnement du parc pilotable par une simple variable, puisqu'ajouter un nœud *backend* ne suppose aucune intervention manuelle, mais une modification déclarative reprise par la chaîne de déploiement. Les nœuds communiquent sur un **réseau privé Hetzner** dédié, complété d'un sous-réseau, sur lequel s'initialise le plan de contrôle Swarm. L'exposition publique est réduite au strict nécessaire, les ports 80 et 443 n'étant ouverts qu'aux plages d'adresses de Cloudflare et le port d'administration n'acceptant qu'une authentification par clé. Les réseaux overlay applicatifs adoptent une MTU réduite à 1400 octets pour rester compatibles avec l'encapsulation du réseau privé sous-jacent (cf. Annexe C).

Le dimensionnement fin constitue un levier direct de l'éco-responsabilité et de la maîtrise des coûts. Chaque service déclare un couple **réservations / limites**, où les réservations garantissent une allocation minimale et orientent le placement, tandis que les limites protègent le cluster contre l'emballlement d'un conteneur. Le service PostgreSQL, le plus exigeant, réserve un cœur virtuel et 2 Go de mémoire pour un plafond de 3,4 cœurs et 6 Go. Les processus applicatifs *backend* et *frontend* réservent respectivement 384 Mo et 256 Mo pour des limites de 1 280 Mo et 960 Mo. Le proxy Traefik, le mandataire PgBouncer et le cache Redis se contentent d'empreintes modestes. Enfin, les agents d'observabilité déployés en mode global (node-exporter, cAdvisor, Alloy) présentent une empreinte volontairement minimale afin de ne pas grever les nœuds qu'ils instrumentent. Ce *right-sizing* explicite, documenté service par service, traduit une démarche de sobriété, la capacité provisionnée correspondant au besoin réel plutôt qu'à une surallocation de confort, ce qui limite simultanément la facture Hetzner et la consommation énergétique induite. La capacité horizontale, quant à elle, découle du nombre de nœuds déclarés dans Terraform, le calcul des réplicas *backend* et *frontend* étant réalisé au moment du déploiement à partir de l'état de l'infrastructure, de sorte que l'échelle applicative reste cohérente avec l'échelle matérielle, sans réglage manuel susceptible de dériver.

12.1.2 Environnement de déploiement (cloud, on-premise, hybride)

L'environnement de déploiement retenu est résolument **cloud** et repose sur une combinaison de trois fournisseurs aux rôles complémentaires. **Hetzner Cloud** assure le calcul et l'hébergement du cluster. **Amazon Web Services** fournit le stockage objet et la diffusion des médias (S3, CloudFront, WAF, ACM, SSM, Budgets) ainsi que l'état distant de Terraform. **Cloudflare** opère la périphérie réseau (DNS, pare-feu applicatif, terminaison TLS stricte, protection volumétrique). Cette répartition n'est pas fortuite, car elle affecte chaque besoin au fournisseur qui l'adresse au meilleur rapport qualité-prix, plutôt que de concentrer l'ensemble chez un acteur unique. Hetzner offre un coût de calcul très compétitif au sein de l'Union européenne. AWS apporte la durabilité éprouvée du stockage objet et un réseau de diffusion mondial mature. Cloudflare mutualise une protection de périphérie difficile à reconstituer soi-même.

Le choix du cloud a été arbitré au regard des alternatives *on-premise* et hybride. Une infrastructure **on-premise** aurait imposé un investissement matériel initial, la gestion physique des serveurs, de leur alimentation et de leur refroidissement, sans offrir l'élasticité ni l'exposition programmatique attendues. Elle contredit surtout le principe fondateur du projet, puisqu'une infrastructure entièrement pilotée par le code et restructurable à la demande suppose des ressources exposées via des interfaces applicatives, ce que le matériel possédé ne fournit pas nativement. Une approche **hybride** aurait, de son côté, cumulé les contraintes des deux mondes sans justification, faute d'un existant à interconnecter, car le projet démarre sans dette d'infrastructure, si bien que la complexité d'une passerelle entre site privé et cloud n'apporterait aucune valeur. Le cloud s'impose donc par sa nature *opex*, son élasticité, et surtout par son pilotage intégral au travers d'API, condition sine qua non de l'*Infrastructure as Code* qui structure l'ensemble du projet.

Deux exigences transversales ont enfin guidé la sélection des régions et des services. La conformité **RGPD** commande un hébergement au sein de l'Union européenne, les machines Hetzner étant localisées en Allemagne et la région AWS retenue étant eu-west-3 (Paris), de sorte que les données personnelles et les médias ne quittent pas l'espace européen (cf. Annexe R). La **maîtrise des coûts** se traduit par des garde-fous concrets, à savoir une classe de prix restreinte pour la diffusion CloudFront, un cycle de vie du stockage orienté vers les paliers économiques, et une

surveillance budgétaire via AWS Budgets côté site public, qui déclenche une alerte en cas de dérive. L'environnement ainsi constitué n'est pas considéré comme un actif fragile à préserver, mais comme un ensemble jetable et reproductible, toute machine pouvant être détruite puis recréée à l'identique par la chaîne outillée, propriété sur laquelle s'appuient directement les dispositifs de reprise décrits plus loin.

12.2 Technologies et outils utilisés

Les technologies retenues privilégient la cohérence de l'écosystème, la performance et la sécurité par conception. Elles se répartissent entre les briques logicielles qui constituent l'application et les mécanismes algorithmiques qui la sécurisent, d'une part, et la chaîne d'outillage qui industrialise sa production et son déploiement, d'autre part.

12.2.1 Algorithmes et frameworks

Le backend s'appuie sur le framework **NestJS 11** exécuté sous le moteur **Bun**, avec l'ORM **TypeORM** pour la persistance sur **PostgreSQL 18**, accédé au travers du mandataire **PgBouncer** en mode transactionnel, et **Redis 8** pour les sessions, le cache, les verrous et la limitation de débit. Le frontal applicatif et le site public reposent sur **Next.js 16**, ce dernier étant produit en export statique pour être servi depuis S3 et CloudFront. Cette homogénéité autour de TypeScript, du backend au frontend, réduit la charge cognitive et mutualise l'outillage de qualité.

Au-delà des frameworks, plusieurs mécanismes algorithmiques concourent directement à la sécurité et à la performance. Le hachage des mots de passe recourt à **Argon2id**, fonction *memory-hard* réputée résistante aux attaques par force brute accélérées sur processeurs graphiques ou circuits dédiés, conformément aux recommandations cryptographiques actuelles (cf. Annexe R). La gestion de l'authentification met en œuvre des **sessions opaques**, le jeton, de forme `sid.nonce`, ne portant aucune donnée exploitable et servant uniquement de référence vers un état conservé dans Redis. À chaque rafraîchissement, une **rotation par nonce** émet une nouvelle empreinte et invalide la précédente. La réutilisation d'un nonce périmé est interprétée comme une **tentative de replay** et déclenche la révocation de la lignée de sessions concernée. Un compteur de version de jeton (`tokenVersion`) autorise en outre une révocation globale immédiate pour un compte donné. Ce dispositif combine des fenêtres de validité étagées (glissante de trente minutes, absolue de

huit heures, jeton d'accès de cinq minutes) pour concilier confort d'usage et surface d'attaque réduite.

Deux techniques ciblent la performance et la sobriété du traitement des données. La **pagination par curseur** (*keyset*) remplace la pagination par décalage. Au lieu de parcourir puis d'écarter les lignes précédentes, chaque page repart d'un repère stable, ce qui garantit un temps de réponse constant même sur un fil volumineux et évite les incohérences liées aux insertions concurrentes. L'**upload en flux multipart** vers S3, doté d'une garde de taille, transmet les fichiers média par fragments sans jamais charger l'intégralité du contenu en mémoire, de sorte que l'empreinte du processus reste bornée quel que soit le volume déposé, ce qui protège les nœuds *backend* de la saturation et sert l'éco-conception. Enfin, des **index partiels** n'indexent que les lignes effectivement publiées, notamment pour le fil public, si bien que l'index reste compact, les analyses de requêtes plus rapides, et le coût de stockage comme la charge de maintenance s'en trouvent réduits. Ces choix illustrent une constante du projet, à savoir que la performance y est recherchée par la justesse des structures de données plutôt que par le surdimensionnement matériel.

12.2.2 Outils de développement et de gestion de projet

La chaîne d'outillage industrialise chaque étape, du poste de développement à la mise en production. Le moteur **Bun** cumule les rôles de *runtime*, de gestionnaire de paquets et d'exécuteur de tests, ce qui accélère sensiblement les cycles locaux et la phase de vérification en intégration continue. La conteneurisation s'appuie sur **Docker**, avec des images **multi-étapes** (constructeur puis exécution sur base Bun/Alpine) produisant des artefacts finaux légers, orchestrés par **Docker Swarm**. L'infrastructure elle-même est décrite par **Terraform** pour le provisionnement des ressources cloud et par **Ansible** pour la configuration des nœuds, à savoir l'installation de Docker, le durcissement du système, l'initialisation du cluster, la distribution des configurations et des secrets, puis le déploiement de la pile et l'exécution des migrations.

L'orchestration de bout en bout est confiée à **GitHub Actions**, qui héberge trois chaînes distinctes, à savoir le déploiement du SaaS, le déploiement du site public et une chaîne ciblée sur le périmètre de diffusion et de sauvegarde, cette dernière étant assortie de garde-fous refusant toute action hors de son domaine. **Git** tient le rôle de

source de vérité unique, conformément à une logique **GitOps** où l'état désiré est versionné et où les artefacts sont immuables, les images étant taguées par l'empreinte du dernier *commit* les concernant et les configurations Swarm identifiées par le hachage de leur contenu. Le registre d'images utilisé est **GHCR**.

La qualité est garantie au plus tôt par un outillage intégré au flux de développement. **ESLint** et **Prettier** normalisent le style et détectent les erreurs de programmation, tandis que les crochets Git orchestrés par **husky** et **lint-staged** exécutent automatiquement, avant chaque validation, le formatage Terraform et l'analyse statique sur les seuls fichiers modifiés. La validation terraform valide et une base de **218 tests Jest** couvrant contrôleurs, services et limitation de débit complètent ce filet de sécurité, la chaîne d'intégration jouant ces vérifications avant tout déploiement. Sur le plan de la gestion de projet, le dépôt monolithique unique (regroupant backend, frontal et site) centralise le suivi des évolutions, l'historique et la revue des modifications. Les artefacts de planification proprement dits (organigramme des tâches, réseau des dépendances, diagramme de Gantt) sont détaillés dans le chapitre consacré à la conduite du projet et rassemblés en annexe. Cet outillage matérialise directement les axes *automatisation* et *qualité* de la problématique, puisque rien de ce qui peut être vérifié ou déployé mécaniquement n'est laissé à une intervention manuelle.

L'outillage de développement intègre par ailleurs une assistance par intelligence artificielle. GitHub Copilot est mobilisé au quotidien pour accélérer l'écriture du code applicatif, des tests et des manifestes d'infrastructure, l'ingénieur conservant l'entière responsabilité de la revue et de la validation de chaque suggestion. Cette assistance augmente la productivité sans se substituer au jugement, les portes de qualité du pipeline (tests automatisés, analyse statique, revue de code) garantissant que le code produit avec l'aide de l'IA respecte les mêmes exigences que le reste de la base.

12.3 Surveillance et monitoring de la sécurité

La sécurité ne se limite pas à des barrières préventives. Elle suppose de voir ce qui se passe et de réagir vite. La plateforme dropicture s'est donc dotée d'une chaîne d'observabilité qui, au-delà du suivi de performance, alimente la détection des anomalies de sécurité et outille la réponse aux incidents (cf. Annexe P).

12.3.1 Outils de monitoring

La collecte de métriques est assurée par **Prometheus**, qui interroge ses cibles toutes les quinze secondes et conserve les séries sur quinze jours dans une limite de dix gigaoctets. Les cibles instrumentées couvrent le proxy Traefik (via son point d'exposition de métriques détaillant points d'entrée, services et routeurs), chaque nœud par l'intermédiaire de **node-exporter**, et chaque conteneur par **cAdvisor**, ces deux agents étant déployés en mode global pour couvrir l'ensemble du parc. La gestion des alertes est déléguée à **Alertmanager**, qui regroupe, déduplique et route les notifications. Les journaux, quant à eux, sont centralisés par **Loki**, alimenté par l'agent **Alloy** exécuté sur chaque nœud, lequel découvre les conteneurs, ré-étiquette les flux par service, pile et nœud, puis les pousse vers le magasin de logs. L'ensemble est restitué dans **Grafana** au travers de tableaux de bord provisionnés par le code (vue d'ensemble, exploration des journaux, et suivi énergétique) de sorte que l'observabilité elle-même est reconstituée et versionnée.

Du point de vue spécifiquement sécuritaire, le backend émet des **journaux d'accès HTTP structurés au format JSON**, consignnant pour chaque requête la méthode, la route, le code de statut, l'adresse IP, l'identifiant d'utilisateur, le quota et la durée de traitement. Ce format normalisé rend les journaux immédiatement interrogeables dans Loki et permet d'isoler un motif suspect (rafale de requêtes, échecs d'authentification répétés, statuts anormaux) sans analyse ad hoc. À la périphérie, les pare-feu applicatifs constituent une seconde source de télémétrie de sécurité. Le **WAF Cloudflare** filtre le trafic entrant, tandis que le **WAF AWS** protégeant la diffusion des médias applique une limitation de débit par adresse, la liste de réputation d'adresses IP d'AWS et un jeu de règles communes, en publiant des métriques et des échantillons de requêtes exploitables pour l'analyse. La combinaison des journaux applicatifs, des métriques d'infrastructure et de la télémétrie de périphérie offre ainsi une visibilité à plusieurs niveaux, condition de l'axe *observabilité* de la problématique.

12.3.2 Techniques de détection et de réponse aux incidents

La détection s'appuie sur un jeu de **dix règles d'alerte** évaluées en continu par Prometheus (cf. Annexe P). Plusieurs d'entre elles portent une valeur de sécurité et de disponibilité directe. L'indisponibilité d'une cible ou d'un nœud est signalée dès

que l'indicateur de présence tombe à zéro pendant cinq minutes. Un taux d'erreurs 5xx supérieur à cinq pour cent sur dix minutes déclenche une alerte critique, symptôme possible d'un abus, d'une attaque ou d'une régression. Un redémarrage fréquent de conteneur, au-delà de deux occurrences par quart d'heure, révèle une instabilité ou un plantage en boucle. La latence au 95^e centile au-delà d'une seconde, la saturation processeur, la pénurie de mémoire et l'épuisement projeté de l'espace disque complètent la couverture. Une règle dédiée à la consommation énergétique estimée prolonge cette surveillance vers l'éco-responsabilité. Ces seuils transforment des signaux bruts en événements qualifiés, portés à la connaissance des exploitants via Alertmanager (par courriel ou messagerie) et consultables dans Grafana.

La réponse aux incidents combine des mécanismes automatiques et des procédures documentées. Le plus structurant est le **retour arrière automatique** intégré à l'orchestration, chaque mise à jour de service suivant une stratégie de déploiement progressif surveillée pendant une fenêtre définie, avec une action d'échec configurée sur le retour à la version antérieure. Un déploiement défectueux se replie donc de lui-même sur l'état stable précédent, sans intervention humaine, ce qui borne l'impact d'une régression et sert conjointement la disponibilité et la qualité. Sur le plan applicatif, la **révocation de sessions** offre une réponse immédiate à une compromission, l'incrémentation du compteur de version de jeton invalidant en bloc toutes les sessions d'un compte, tandis que la nature opaque et centralisée des sessions dans Redis autorise une révocation ciblée. La détection de rejeu évoquée plus haut agit ici comme un capteur de sécurité autonome, coupant automatiquement une lignée de sessions dès qu'un jeton périmé est rejoué. Ces automatismes sont enfin encadrés par un **runbook** qui documente la marche à suivre pour les situations non automatisées (diagnostic, escalade, remédiation) et par la capacité de reconstruction décrite à la section suivante, qui constitue le dernier recours en cas d'incident majeur.

12.4 Plans de sauvegarde et de récupération

La continuité de service suppose de protéger les données contre la perte comme l'infrastructure contre la destruction. La stratégie de dropicture distingue ces deux plans. La préservation des données, d'une part, est confiée à un stockage objet

durci, tandis que la capacité de reconstruction, d'autre part, est portée par l'*Infrastructure as Code*.

12.4.1 Stratégies de sauvegarde des données

Les sauvegardes de la base de données sont produites automatiquement et déposées dans un bucket S3 dédié, distinct du stockage des médias et durci par plusieurs mécanismes cumulés. Une chaîne planifiée, déclenchée toutes les six heures, réalise un cliché de la base au format natif compressé depuis le conteneur PostgreSQL en fonctionnement, calcule son empreinte SHA-256, refuse tout cliché dont la taille serait manifestement invraisemblable, puis téléverse le fichier et son empreinte avant de vérifier la taille de l'objet effectivement déposé. Cette suite de gardes évite le scénario le plus redouté d'un dispositif de sauvegarde, à savoir l'archivage silencieux d'une sauvegarde vide ou tronquée. Le stockage est ensuite durci. Le versionnage est activé, de sorte qu'aucune écriture n'écrase définitivement une version antérieure. Le verrouillage d'objet en mode GOVERNANCE impose une conservation immuable sur une durée paramétrée, une sauvegarde ne pouvant être ni altérée ni supprimée avant l'échéance, ce qui protège l'historique contre une suppression accidentelle comme contre une action malveillante de type rançongiciel. Un cycle de vie orchestre enfin l'économie du stockage. Les sauvegardes quotidiennes, rangées sous un préfixe dédié, basculent vers une classe d'accès peu fréquent au bout de trente jours avant expiration au terme de leur rétention. Les sauvegardes mensuelles transitent vers un palier d'archivage Glacier selon la même logique, pour une conservation plus longue à moindre coût. Les versions périmées et les téléversements multipart inachevés sont purgés automatiquement, afin de ne pas accumuler de résidus facturables. Ce recours au tiering sert simultanément la maîtrise des coûts et l'éco-responsabilité, l'archivage froid présentant une empreinte bien inférieure au stockage chaud.

La protection des données ne s'arrête pas à la base. Le bucket de diffusion des **médias** est lui-même versionné, ce qui prémunit contre l'écrasement d'un contenu. Le chiffrement au repos en AES-256 s'applique à l'ensemble des buckets, le transport est systématiquement chiffré et une politique explicite refuse toute connexion non sécurisée. Surtout, l'**état de Terraform**, description faisant autorité de l'infrastructure, est conservé dans un bucket S3 chiffré, versionné et protégé par un verrou de concurrence, cet état constituant le point de vérité à partir duquel toute

reconstruction s'opère, et sa sauvegarde est donc aussi critique que celle des données métier. L'ensemble de ces dispositifs relève du chapitre technologique de la cartographie ISO/IEC 27001 relatif à la sauvegarde et à la cryptographie (cf. Annexe R).

12.4.2 Plans de reprise après sinistre

Le plan de reprise après sinistre découle directement du parti pris fondateur. L'infrastructure étant intégralement décrite par le code, sa reconstruction n'est pas une opération exceptionnelle et improvisée, mais l'exécution ordonnée de la même chaîne outillée qui la produit au quotidien. En cas de perte d'un nœud, d'une région ou de l'ensemble du cluster, **Terraform** recrée les serveurs Hetzner, le réseau privé, les règles de pare-feu, les buckets, la distribution de diffusion et les réglages Cloudflare, tandis qu'**Ansible** réinstalle Docker, ré-applique le durcissement, réinitialise le cluster Swarm, redistribue configurations et secrets, redéploie la pile et rejoue les migrations. Le caractère **idempotent et reproductible** de ces descriptions garantit une reconstruction à l'identique, le délai de rétablissement se mesurant à la durée d'exécution de la chaîne, non à une remise en état manuelle et incertaine.

Cette capacité s'appuie sur une architecture délibérément sans état sur les nœuds de calcul, les médias résidant dans S3 et non sur les machines, la base de données étant isolée sur le nœud db et adossée à ses sauvegardes, si bien que la perte d'un nœud applicatif se règle par un simple reprovisionnement plutôt que par une récupération de données. La reprise complète consiste alors à reconstruire l'infrastructure par le code, puis à restaurer la base à partir de la dernière sauvegarde S3, les médias demeurant disponibles depuis leur stockage objet. Cette restauration n'est pas une procédure manuelle décrite sur le papier, elle est elle-même outillée et rejouable. Une chaîne dédiée, déclenchée manuellement et protégée par une confirmation explicite, sélectionne soit le dernier cliché d'un préfixe, soit une sauvegarde précise, met le service applicatif à l'arrêt le temps de l'opération afin qu'aucune écriture concurrente ne survienne, vérifie l'empreinte SHA-256 du fichier téléchargé, réalise un cliché de sécurité de la base courante avant tout écrasement, restaure en une transaction unique de sorte qu'une erreur annule intégralement l'opération, contrôle le schéma restauré, puis rétablit le nombre initial de répliques applicatives. Ce séquençage fait du plan de reprise une capacité vérifiable plutôt qu'une intention (cf. Annexe S).

Le dispositif de reprise atteint ainsi un niveau de maturité opérationnel. Les briques du plan de reprise sont en place et pleinement fonctionnelles, qu'il s'agisse du versionnage, du verrouillage d'objet, de l'état distant, de la reconstruction par le code, de la sauvegarde planifiée ou de la restauration outillée et transactionnelle. La démarche d'amélioration continue porte désormais sur la formalisation d'un exercice de reprise périodique conduit en conditions réelles et sur le suivi documenté d'un objectif de point de reprise, afin de confirmer régulièrement l'objectif de temps de rétablissement et l'intégrité des sauvegardes restaurées. Cette maîtrise inscrit pleinement le dispositif dans le cycle d'amélioration continue qui sous-tend l'ensemble du projet et prépare la revue de sécurité récurrente attendue d'une infrastructure DevOps mûre.

Au terme de cette section, les implémentations techniques apparaissent comme la traduction opérationnelle et cohérente de la problématique, à savoir une infrastructure matérielle sobre et spécialisée, un environnement cloud entièrement piloté par le code, des choix algorithmiques orientés vers la sécurité et la performance, une surveillance capable de détecter et de contenir les incidents, et des dispositifs de sauvegarde et de reprise adossés à la reproductibilité. Ces réalisations fournissent le socle concret sur lequel s'appuient la conduite de projet et l'approfondissement de l'option DevOps développés dans les chapitres suivants.

13. Gestion du projet et management

La problématique du mémoire (concevoir et déployer une infrastructure DevOps automatisée, sécurisée et éco-responsable tout en maîtrisant la qualité, l'observabilité et les coûts) ne relève pas seulement de choix techniques, elle suppose également un pilotage rigoureux. Livrer en continu une plateforme SaaS impose de séquencer des travaux interdépendants (infrastructure, backend, frontend, chaîne CI/CD, observabilité), d'allouer des ressources limitées, d'anticiper les aléas et de coordonner des parties prenantes aux attentes distinctes. La présente section décrit la démarche de gestion de projet et de management retenue pour la plateforme dropicture, transposée dans une situation professionnelle reconstituée de mission de conseil, conforme au référentiel. Les outils de planification et le registre des risques sont détaillés en annexes (L, M, N, O) et seulement appelés ici. Le corps du texte se concentre sur la méthode, les arbitrages et leur justification.

13.1 Élaboration du planning et gestion des ressources

La planification a reposé sur un enchaînement classique et outillé, avec la décomposition du périmètre en lots de travail, le séquençage des tâches par identification du chemin critique, puis la transcription calendaire. Cette rigueur conditionne directement la maîtrise des délais et des coûts, deux axes de la problématique.

13.1.1 Création d'un planning détaillé

La première étape a consisté à décomposer l'ensemble du périmètre en une structure de découpage du projet (Work Breakdown Structure). Cette WBS organise dropicture en huit lots de travail hiérarchisés (cadrage et conception, infrastructure laC, backend API, frontend et site, chaîne CI/CD, observabilité, sécurité et conformité, documentation et maintenance) pour une charge globale estimée à environ soixante-neuf jours-homme (cf. Annexe L). Chaque lot est qualifié de fonctionnel (backend, frontend) ou non fonctionnel (infrastructure, CI/CD, observabilité, sécurité), ce qui met en évidence le poids important des exigences non fonctionnelles propres à un projet à dominante DevOps. La WBS a servi de référentiel unique pour l'estimation de charge, l'affectation des travaux et le suivi du reste-à-faire.

Chaque lot de la WBS est en outre valorisé : la charge exprimée en jours-homme est convertie en coût par application d'un taux journalier moyen de référence, ce qui rattache un coût estimé à chaque tâche et alimente directement le chiffrage budgétaire à trois niveaux (global, conception et déploiement) détaillé en 13.5.5. Le tableau ci-dessous récapitule cette valorisation, en distinguant les lots fonctionnels des lots non fonctionnels.

Lot de travail (WBS)	Type	Charge (j-h)	Coût estimé (TJM 500 €)
Cadrage & conception	Fonctionnel	8	4 000 €
Infrastructure laC	Non fonctionnel	10	5 000 €
Backend API	Fonctionnel	14	7 000 €
Frontend & site	Fonctionnel	9	4 500 €
Chaîne CI/CD	Non fonctionnel	8	4 000 €
Observabilité	Non fonctionnel	7	3 500 €
Sécurité & conformité	Non fonctionnel	7	3 500 €
Documentation & soutien	Support	6	3 000 €
Total	—	69	34 500 €

À partir de cette décomposition, un diagramme PERT a formalisé les dépendances logiques entre lots et calculé le chemin critique (cf. Annexe M). Le séquençage fait apparaître un enchaînement structurant, à savoir cadrage et conception, puis backend API, puis CI/CD, puis observabilité, puis documentation et rétrospective, soit un chemin critique de quarante jours ($A \rightarrow C \rightarrow D \rightarrow F \rightarrow H$). Une attention particulière portée à ces tâches sécurise l'échéance finale et justifie une vigilance renforcée sur ces lots. À l'inverse, l'infrastructure laC, le frontend et la sécurité disposent d'une marge et peuvent absorber un léger retard sans décaler la date de fin, information précieuse pour arbitrer les priorités en cas de tension.

Ce séquençage a enfin été transcrit dans un planning de Gantt (cf. Annexe N) organisé en cinq phases (conception, infrastructure, développement, DevOps et clôture) et jalonné de cinq points de contrôle (M1 à M5). Le planning constitue le support de pilotage, où les dates, indicatives, respectent l'ordre imposé par le chemin critique et sont réactualisées à chaque itération pour refléter l'avancement réel.

13.1.2 Allocation des ressources

L'allocation des ressources a distingué le capital humain des moyens matériels et cloud. Dans la situation reconstituée, l'équipe projet réunit un chef de projet assurant également le rôle d'ingénieur DevOps référent, un ou deux développeurs en charge des lots applicatifs, un référent sécurité mobilisé sur le durcissement et la conformité, ainsi qu'un sponsor porteur des objectifs. La taille resserrée de l'équipe favorise une forte polyvalence et une affectation des personnes en priorité sur les lots du chemin critique. Les rôles et responsabilités sont formalisés par une matrice RACI décrite en 13.4.1, afin d'éviter toute ambiguïté sur qui réalise, qui décide, qui est consulté et qui est informé.

Les ressources matérielles et cloud ont été dimensionnées au plus juste, en cohérence avec l'objectif de maîtrise des coûts et d'éco-responsabilité. L'infrastructure repose sur des nœuds Hetzner Cloud provisionnés par rôle (proxy, base de données, backend, frontend), complétés par les services AWS mobilisés pour la diffusion des médias et les sauvegardes (S3, CloudFront). Le nombre de réplicas applicatifs est dérivé du nombre de nœuds déclaré dans Terraform, et chaque service Docker Swarm porte des réservations et limites calibrées (right-sizing) qui évitent la sur-allocation. Les moyens de production complémentaires (postes de développement, dépôt d'images GHCR, exécuteurs GitHub Actions, outils de collaboration) complètent l'inventaire. Cette approche « au plus près du besoin » relie directement l'allocation des ressources à deux exigences de la problématique, la sobriété énergétique et la maîtrise budgétaire.

13.2 Suivi de l'avancement et gestion des risques

Le pilotage ne se limite pas à établir un plan, il exige de mesurer l'avancement au moyen d'indicateurs objectifs et d'anticiper les événements susceptibles d'affecter les objectifs. L'infrastructure d'observabilité mise en place sert ici de source de vérité pour une partie des mesures.

13.2.1 Indicateurs de performance

Le suivi de l'avancement s'appuie d'abord sur des indicateurs de projet, le reste-à-faire par lot issu de la WBS, le respect des jalons M1 à M5 et la consommation de la marge sur le chemin critique. À ces mesures s'ajoutent des indicateurs de

performance de la chaîne de livraison, alignés sur les métriques DORA reconnues comme références du domaine, à savoir la fréquence de déploiement, le délai de livraison des changements (du commit à la mise en production), le taux d'échec des changements et le temps moyen de rétablissement du service (MTTR). Ces quatre indicateurs traduisent la capacité de la plateforme à livrer vite et sûrement, au cœur de la problématique de livraison continue.

La qualité est suivie au moyen de la couverture assurée par les deux cent dix-huit tests automatisés Jest exécutés en intégration continue et des portes de contrôle bloquantes du pipeline. L'exploitation est mesurée par la disponibilité de la plateforme, le taux d'erreurs 5xx, la latence au 95^e centile et la consommation énergétique estimée, autant de séries exposées dans les tableaux de bord Grafana provisionnés (cf. Annexe P). L'ensemble de ces indicateurs, consolidés dans une boucle « mesurer → identifier → prioriser → implémenter → mesurer », fournit une base factuelle pour décider des actions correctives plutôt que de piloter à l'estime.

La restitution de ces indicateurs est assistée par intelligence artificielle. Un tableau de bord de synthèse, sécurisé et consultable en ligne, agrège les métriques d'avancement, de qualité et d'exploitation, et s'appuie sur des fonctions d'IA générative pour produire automatiquement des résumés en langage naturel et signaler les tendances saillantes, telles qu'une dérive d'indicateur ou une corrélation entre incidents et déploiements. Ce reporting assisté, de même que le rapport de qualité de l'application publié en ligne (tableaux de bord Grafana et documentation OpenAPI), fournit aux parties prenantes des feedbacks pertinents et accessibles sans travail manuel de mise en forme, et alimente la boucle d'amélioration continue.

13.2.2 Gestion des risques

La gestion des risques a été formalisée dans un registre recensant huit risques majeurs, chacun évalué selon une échelle de probabilité et d'impact dont le produit détermine la criticité, puis positionné sur une matrice probabilité/impact (cf. Annexe O). Cette cotation hiérarchise l'effort et met en avant deux points de vigilance prioritaires, tous deux déjà encadrés par un dispositif de maîtrise. Le premier porte sur le nœud proxy qui regroupe Traefik, Redis et le monitoring (R1), dont la résilience est assurée par le caractère reproductible de l'infrastructure as code autorisant une reconstruction rapide, et renforcée par la perspective de haute disponibilité du

monitoring inscrite au plan d'amélioration continue. Le second porte sur la protection des sessions contre le vol ou le rejeu (R3), assurée par les mesures de durcissement de la sécurité mises en œuvre sur la plateforme.

À chaque risque est associé un plan de mitigation ancré dans les dispositifs techniques du projet. Le risque de régression au déploiement (R7) est contenu par les tests en intégration continue, les analyses de sécurité bloquantes, les images immuables et le déploiement progressif avec retour arrière automatique. Le dépassement des coûts cloud (R4) est surveillé par AWS Budgets, le right-sizing et le cycle de vie du stockage S3. La perte de données (R5) est couverte par des sauvegardes automatisées toutes les six heures, versionnées et protégées par Object Lock, dont l'intégrité est vérifiée par empreinte et dont la restauration est elle-même outillée et transactionnelle (cf. Annexe S). La fuite de secrets (R2) est prévenue par leur externalisation hors du dépôt Git et par une analyse automatique de détection de secrets en porte bloquante. La panne d'un fournisseur (R6) est neutralisée par le caractère reproductible de l'infrastructure as code. La non-conformité RGPD (R8) est écartée par la minimisation, le droit à l'effacement et l'hébergement européen. Le registre fait l'objet d'une revue périodique, et les perspectives d'amélioration continue (haute disponibilité du monitoring, exercice de reprise périodique, notifications d'alerte) renforcent encore ce dispositif de maîtrise.

13.3 Management de l'équipe projet

Au-delà des outils, la réussite du projet repose sur l'animation de l'équipe. La situation reconstituée retient une équipe resserrée, ce qui appelle un management de proximité, orienté vers l'autonomie et la montée en compétences.

13.3.1 Encadrement et motivation

L'encadrement s'est organisé autour d'un management par objectifs et d'itérations courtes. Chaque itération ouvre sur des engagements clairs, se clôt par une revue des livrables et une rétrospective, et laisse une large autonomie d'exécution aux contributeurs. Ce cadre, hérité des pratiques agiles, favorise la responsabilisation, chacun disposant d'un périmètre lisible et d'un droit à l'erreur encadré par des filets de sécurité techniques (tests, retour arrière automatique) qui dédramatisent la prise d'initiative.

La motivation a été entretenue par le sens donné au travail et par la reconnaissance des contributions. Le caractère concret du produit (une plateforme réellement déployée et observable) constitue un moteur d'engagement, de même que la célébration des jalons franchis (infrastructure déployée, application fonctionnelle, chaîne de livraison opérationnelle). Le feedback régulier, dans les deux sens, et la transparence sur l'avancement au moyen des indicateurs partagés contribuent à maintenir la mobilisation sur la durée, y compris sur les lots non fonctionnels dont la valeur se confirme dans le temps.

13.3.2 Développement des compétences

Le développement des compétences a été traité comme un objectif de projet à part entière, et non comme un effet secondaire. La revue de code systématique, la pratique ponctuelle de la programmation en binôme et la rédaction de décisions d'architecture documentées (ADR) diffusent les savoir-faire au sein de l'équipe et évitent la concentration des connaissances sur une seule personne. Une veille technologique continue nourrit les choix d'outillage et maintient l'équipe au niveau de l'état de l'art.

La capitalisation prolonge cette dynamique au-delà du projet. La documentation technique structurée, les runbooks d'exploitation et le retour d'expérience consignent les enseignements tirés, tant sur les réussites que sur les points d'amélioration identifiés. Cette mémoire projet réduit la dépendance aux individus, facilite l'intégration de nouveaux contributeurs et alimente le plan d'amélioration continue. Elle transforme l'expérience acquise en actif réutilisable, cohérent avec une démarche DevOps fondée sur l'apprentissage itératif.

13.4 Communication et coordination avec les parties prenantes

La coordination des parties prenantes conditionne l'alignement des attentes et la fluidité des décisions. Un plan de communication explicite et un engagement actif des acteurs ont donc été mis en place.

13.4.1 Plan de communication

Le plan de communication définit la cadence, les canaux et les responsabilités de l'information projet. La cadence combine des points d'avancement réguliers, des revues à l'issue de chaque itération et un reporting synthétique au franchissement

des jalons. Les canaux privilégient les outils collaboratifs déjà au cœur de la chaîne DevOps. Le dépôt Git et sa forge (suivi des tâches, revues de code, tableaux de bord de projet) tiennent lieu de source de vérité, la documentation versionnée et la spécification OpenAPI/Swagger exposée par le backend servent de référence partagée, et les tableaux de bord Grafana rendent l'état d'exploitation visible de tous.

La répartition des responsabilités est formalisée par une matrice RACI qui, pour chaque grand livrable, désigne le responsable de la réalisation, le décideur redevable, les acteurs consultés et ceux simplement informés. À titre d'illustration, sur le lot infrastructure l'ingénieur DevOps est responsable de la réalisation et le sponsor redevable de la validation, tandis que le référent sécurité est consulté et l'équipe applicative informée. Cette formalisation clarifie les responsabilités décisionnelles et fluidifie les arbitrages.

La matrice RACI complète, qui croise les huit lots de la structure de découpage (WBS) et les six acteurs du projet, figure en annexe (cf. Annexe L bis). Elle attribue à chaque lot un unique décideur redevable (Accountable) afin d'éviter la dilution de la décision, et veille à une distribution équilibrée des rôles : le sponsor est redevable du cadrage et de l'infrastructure, l'ingénieur DevOps des lots techniques, le référent sécurité du lot sécurité et conformité, et le référent pédagogique de la documentation et de la soutenance. Les abréviations (R, A, C, I) y sont explicitées et la répartition par acteur est récapitulée.

13.4.2 Engagement des parties prenantes

L'engagement des parties prenantes a débuté par leur identification et la caractérisation de leurs attentes. Le sponsor attend l'atteinte des objectifs dans le cadre convenu, le référent pédagogique Joris VERGULDEZOONE et le référent en entreprise Philippe AMERIO veillent à la conformité au référentiel, les utilisateurs bêta éprouvent l'usage réel de la plateforme, et les fournisseurs cloud constituent des partenaires techniques dont dépend la disponibilité du service.

L'alignement de ces acteurs a été recherché par une communication transparente et par l'objectivation des échanges au moyen des indicateurs et des tableaux de bord. Les demandes de changement sont tracées et arbitrées à l'aune de leur impact sur le chemin critique et sur les coûts, ce qui permet d'associer les parties prenantes aux décisions sans diluer le cap. La visibilité offerte par l'observabilité joue ici un rôle de

confiance et donne à chaque acteur une image partagée et vérifiable de l'état du système, condition d'un engagement durable.

Cette démarche est consolidée dans une matrice d'engagement des parties prenantes (cf. Annexe O bis), qui positionne pour chaque acteur son niveau d'engagement actuel et son niveau cible, en identifie les causes et définit les actions d'amélioration correspondantes. Cette matrice prend explicitement en compte les situations de handicap (PSH), en prévoyant des surfaces et une documentation accessibles, conformes aux référentiels WCAG 2.2 et RGAA 4, ainsi que des formats lisibles par lecteur d'écran et des comptes rendus écrits, afin que la contribution de chaque partie prenante soit recueillie et valorisée sans discrimination.

13.5 Planification

Cette dernière section consolide la démarche de planification en explicitant la méthodologie retenue, le découpage en phases, le planning détaillé et le cadrage des ressources et du budget. Elle articule en un ensemble cohérent les éléments déjà mobilisés en 13.1, sous l'angle du cycle de vie du projet.

13.5.1 Introduction

La planification poursuit un double objectif, garantir la livraison des lots dans un ordre techniquement viable et offrir aux parties prenantes une lecture claire de la trajectoire du projet. Elle sert de contrat de référence, ajusté au fil de l'eau, entre les ambitions du produit et les contraintes de délai, de ressources et de coût. Les instruments qui la matérialisent (WBS, PERT, Gantt) sont renvoyés en annexes et leur logique d'ensemble est exposée ci-après.

13.5.2 Méthodologie de projet

La méthodologie retenue est hybride, adaptée à la nature du projet et à la taille de l'équipe. Un cadrage prédictif structure l'ensemble, la WBS, le PERT et le planning de Gantt fixant le périmètre, les dépendances et les échéances, indispensables au pilotage d'un projet à forte composante infrastructure. L'exécution, en revanche, adopte une approche agile adaptée, organisée en itérations et en sprints courts alignés sur les lots de la WBS, chacun se concluant par une revue et une rétrospective. Cette adaptation assume les particularités du contexte, l'agilité étant allégée de ses cérémonies les plus lourdes, peu pertinentes pour une équipe

resserrée, tout en conservant son esprit (livrer par incréments, mesurer, s'ajuster). Le couplage du cadre prédictif et de l'exécution itérative concilie la prévisibilité attendue d'une certification et la souplesse nécessaire à un projet technique exploratoire.

13.5.3 Phases du projet

Le cycle de vie du projet se décline en quatre phases. L'initiation cadre le besoin, la problématique et les objectifs, et pose les fondations de conception (veille, cartographie, choix d'architecture). La planification établit la WBS, le séquençage PERT, le planning de Gantt et le registre des risques, et arrête l'allocation des ressources. L'exécution constitue le cœur des travaux, avec le provisionnement de l'infrastructure as code, le développement du backend et du frontend, l'industrialisation de la chaîne CI/CD, la mise en place de l'observabilité et le durcissement de la sécurité, le tout piloté au moyen des indicateurs décrits en 13.2. La clôture consolide enfin la documentation technique, formalise le retour d'expérience et prépare la soutenance devant le jury, programmée le 03/08/2026. Chaque transition de phase est matérialisée par un jalon, condition de passage à la suivante et occasion d'un bilan d'avancement.

13.5.4 Planning détaillé

Le planning détaillé, présenté sous forme de diagramme de Gantt (cf. Annexe N), déroule les cinq sections de travail (conception, infrastructure, développement, DevOps et clôture) à partir d'un démarrage fixé au 3 février 2025. Il matérialise cinq jalons de contrôle, à savoir les dossiers de conception validés (M1), l'infrastructure déployée par l'IaC (M2), l'application fonctionnelle (M3), le pipeline et l'observabilité en place (M4), puis le dossier technique et le retour d'expérience (M5). L'ordonnancement respecte scrupuleusement le chemin critique identifié par le PERT (cf. Annexe M), de sorte que les tâches disposant d'une marge (infrastructure, frontend, sécurité) puissent être décalées sans compromettre l'échéance globale. Les dates demeurant indicatives, le planning est réactualisé à chaque jalon pour intégrer l'avancement réel et les éventuels arbitrages de priorité.

13.5.5 Ressources et budget

Le cadrage budgétaire a suivi une logique d'estimation à trois niveaux de granularité croissante, chacun chiffré. Le premier niveau, global, établi en début de projet à partir

de la charge totale d'environ soixante-neuf jours-homme issue de la WBS et d'un taux journalier moyen de 500 €, fixe l'enveloppe de construction à 34 500 €. Le deuxième niveau, dit de conception, affine cette estimation lot par lot une fois les choix d'architecture arrêtés, en pondérant chaque poste selon sa complexité. Il isole les lots de conception et de spécification, à savoir le cadrage et la conception ainsi que la part d'étude des lots techniques, pour un total de vingt-deux jours-homme et 11 000 €. Le troisième niveau, dit de déploiement, distingue la charge de mise en œuvre et d'industrialisation, soit quarante-sept jours-homme et 23 500 €, et la complète par les coûts récurrents d'exploitation de l'infrastructure en fonctionnement, estimés à environ 71 € par mois, soit 852 € sur douze mois. Le tableau ci-dessous récapitule ces trois niveaux.

Niveau d'estimation	Périmètre couvert	Charge (j-h)	Montant
Global	Enveloppe de construction du projet, tous lots WBS confondus	69	34 500 €
Conception	Cadrage, veille, architecture, modélisation, spécification et documentation de conception	22	11 000 €
Déploiement (construction)	Développement, IaC, CI/CD, observabilité, durcissement et mise en production	47	23 500 €
Déploiement (exploitation récurrente)	Calcul Hetzner par rôle, stockage et diffusion AWS (S3, CloudFront),	—	≈ 71 € / mois, soit 852 € / an

	sauvegardes, DNS et périphérie		
Total première année	Construction et douze mois d'exploitation	69	≈ 35 352 €

La maîtrise des coûts s'appuie sur des leviers concrets, directement reliés à la problématique. Le choix d'un hébergement Hetzner à faible coût pour les nœuds de calcul, préféré aux fournisseurs cloud majeurs pour un besoin équivalent, réduit sensiblement la facture d'exploitation. Le recours ciblé à AWS pour la diffusion des médias et les sauvegardes est encadré par AWS Budgets, qui déclenche des alertes de coût par courriel dès qu'un seuil est approché. Le right-sizing des services, la classe de prix restreinte du CDN et le cycle de vie du stockage vers des paliers d'archivage complètent le dispositif de sobriété. Le budget d'ensemble demeure ainsi maîtrisé, adossé à l'hébergement Hetzner à faible coût, à un suivi des dépenses cloud automatisé par AWS Budgets et à une estimation conduite à trois niveaux, global, conception et déploiement. Cette démarche garantit une trajectoire de coûts anticipée, mesurée et alertée, cohérente avec l'exigence d'éco-responsabilité et de maîtrise budgétaire portée par le projet.

14. Conception et Développement de l'Application Informatique

La problématique du présent mémoire interroge la capacité à livrer en continu une plateforme SaaS de partage de médias tout en garantissant la qualité, la sécurité et la maîtrise des coûts. Cette exigence ne se joue pas seulement au niveau de l'infrastructure, elle prend racine dans la conception applicative elle-même. Un logiciel mal découpé, peu testé ou dépourvu de contrats explicites reste impossible à déployer sereinement, quelle que soit la sophistication de la chaîne d'automatisation qui l'entoure. Ce chapitre décrit donc l'application dropicture sous l'angle de sa conception, de sa stratégie de tests et de ses modalités de livraison, en montrant comment chaque décision d'ingénierie sert directement les objectifs d'automatisation, d'observabilité et de sécurité portés par le projet. Les schémas correspondants sont regroupés en annexe et appelés au fil du texte.

14.1 Conception

14.1.1 Introduction

La plateforme dropicture est un service de partage de photographies et de vidéos organisé autour d'un principe simple. La bibliothèque d'un membre est privée par défaut, et seule une publication explicite rend un média visible du public. Cette règle fonctionnelle structure l'ensemble de la conception, car elle impose de séparer nettement le domaine privé (bibliothèque, albums, brouillons) du domaine public (profils, galeries, fil social léger). La conception retenue traduit ce besoin métier en une architecture logicielle claire, testable et automatisable, plutôt qu'en un monolithe indifférencié.

Trois préoccupations ont guidé les choix de conception. La première est la séparation des responsabilités, chaque périmètre fonctionnel disposant de son propre point d'entrée et ne connaissant des autres que ce qui lui est strictement nécessaire. La deuxième est la sécurité par construction, l'authentification, la validation et la limitation de débit étant traitées comme des propriétés transversales appliquées systématiquement, et non comme des ajouts ponctuels. La troisième est l'éco-conception, qui se manifeste par le refus de bufferiser les fichiers volumineux en mémoire, par des index partiels réduisant le volume de données indexées et par

un modèle relationnel dépourvu de colonnes techniques superflues. Les acteurs et cas d'utilisation qui sous-tendent cette conception sont détaillés en annexe (cf. Annexe G), tandis que l'architecture objet du backend est formalisée par le diagramme de classes (cf. Annexe H).

En amont de l'implémentation, les interfaces des principaux modules ont été esquissées sous forme de maquettes basse fidélité (wireframes), reliant chaque écran à ses flux de données et aux points d'accès de l'interface de programmation (cf. Annexe F). Volontairement dépouillées de tout parti pris graphique, ces maquettes ont servi de support de validation fonctionnelle avec les parties prenantes avant tout développement, garantissant que l'agencement des écrans et les parcours utilisateurs (connexion, bibliothèque, téléversement, publication, profil public, fil social) répondaient au besoin exprimé et facilitaient l'implémentation.

14.1.2 Décomposition de l'application

L'application est organisée en monorepo, un dépôt unique regroupant les trois applications qui composent le produit ainsi que le code d'infrastructure. Ce choix structurant permet de versionner conjointement le code applicatif, les définitions d'infrastructure et les pipelines de livraison, condition nécessaire à une démarche GitOps cohérente. Le monorepo distingue trois applications, à savoir un backend applicatif (apps/saas/backend), une application web authentifiée (apps/saas/frontend) et un site vitrine (apps/website). Le backend est une API REST développée avec le framework NestJS 11 exécuté sous le runtime Bun, adossée à l'ORM TypeORM. L'application authentifiée et le site public reposent sur Next.js 16, ce dernier étant produit en export statique afin d'être servi depuis un stockage objet derrière un réseau de diffusion, sans serveur applicatif à maintenir, arbitrage favorable à la fois aux coûts et à la surface d'attaque.

Le backend constitue le cœur de la logique métier et applique une décomposition par domaine. Chaque périmètre fonctionnel est pris en charge par un contrôleur dédié, dont les routes sont préfixées par /api/<domaine>. Le contrôleur d'authentification gère l'inscription, la connexion et le cycle de vie des sessions. Le contrôleur de bibliothèque prend en charge le téléversement, les albums et la publication des médias privés. Le contrôleur de profil expose les informations publiques du membre, sa biographie et son avatar. Le contrôleur de découverte alimente le fil d'actualité et

les abonnements. Le contrôleur de paramètres administre le compte, la sécurité et la clé d'API. Enfin, le contrôleur public sert les profils et galeries ouverts sans authentification. Cette organisation « un contrôleur, un domaine » facilite la lecture, circonscrit l'impact des évolutions et rend chaque périmètre testable de façon isolée.

La logique transverse est factorisée dans trois services injectables. Le service d'authentification (AuthService) encapsule la création, la résolution, la rotation et la révocation des sessions ainsi que la gestion des jetons. Le service de médias (MediaService) centralise l'envoi vers le stockage objet en flux, la publication, la dépublication et la suppression des fichiers, y compris l'invalidation du cache de diffusion. Le service Redis (RedisService) fournit l'accès mutualisé au magasin clé-valeur utilisé pour les sessions, le cache, les verrous et la limitation de débit. À ces éléments s'ajoutent deux stratégies d'authentification Passport (l'une adossée au cookie de session, l'autre à la clé d'API), un intergiciel de journalisation des accès HTTP et une couche de persistance regroupant les cinq entités du modèle. Cette décomposition en contrôleurs, services et stratégies, articulée autour de l'injection de dépendances, est représentée dans le diagramme de classes du backend (cf. Annexe H). La cartographie applicative complète figure en annexe (cf. Annexe A).

14.1.3 Fonctionnement de l'application

Le fonctionnement de la plateforme se comprend à travers trois flux représentatifs, dont les diagrammes de séquence détaillés sont regroupés en annexe (cf. Annexe I).

Le premier flux est le téléversement d'un média. Lorsqu'un membre envoie une photographie ou une vidéo, le corps de la requête est traité comme un flux binaire et non comme un fichier intégralement chargé en mémoire. Le service de médias interpose un transformateur qui compte les octets à la volée et interrompt immédiatement l'envoi dès qu'un seuil est franchi, soit huit méga-octets pour une image ou un avatar et cent méga-octets pour une vidéo. Le contenu est acheminé vers le stockage objet en mode multipart, par tranches de cinq méga-octets, et l'envoi est avorté puis nettoyé en cas de dépassement ou d'échec. Cette conception en flux poursuit un double objectif de robustesse et d'éco-conception, en bornant l'empreinte mémoire du processus quelle que soit la taille du fichier et en évitant toute copie inutile. Le média ainsi enregistré est privé par défaut, sa date de

publication restant nulle, et sa clé de stockage est rendue immuable côté cache de diffusion.

Le deuxième flux est la publication. Rendre un ou plusieurs médias publics revient à dater leur champ de publication au moyen d'une mise à jour atomique et idempotente. La date existante est préservée, de sorte que republier un contenu ne réécrit pas sa date d'origine. Chaque opération est bornée par une clause d'appartenance garantissant qu'un membre n'agit que sur ses propres médias, les éléments non traités étant restitués avec un code d'erreur explicite. Une fois publié, le média apparaît dans la galerie publique, dont la lecture s'appuie sur un index partiel ne référençant que les lignes effectivement publiées. La suppression suit le même principe de cohérence. Elle efface l'objet du stockage, retire la ligne en base, puis déclenche une invalidation du cache de diffusion afin de purger la copie mise en cache. Le partage repose ainsi sur un basculement d'état simple et réversible plutôt que sur une duplication de données.

Le troisième flux concerne les sessions sécurisées, fonctionnalité différenciante du projet. À la connexion, après vérification du mot de passe, une session opaque est créée. Un enregistrement est stocké dans Redis et le client ne reçoit qu'un cookie composé d'un identifiant de session et d'un jeton aléatoire, sans aucune donnée exploitable côté navigateur. Chaque requête protégée résout ce cookie contre l'enregistrement Redis, avec une expiration glissante de trente minutes, une expiration absolue de huit heures et une fenêtre d'accès de cinq minutes. Le rafraîchissement procède par rotation du jeton, protégée par un verrou afin d'éviter les rotations concurrentes. La réutilisation d'un ancien jeton hors d'une brève fenêtre de grâce est interprétée comme un vol présumé et entraîne la révocation globale des sessions du compte. Ce mécanisme, articulé au versionnement des jetons qui invalide instantanément toutes les sessions lors d'un changement de mot de passe, illustre concrètement la posture de défense en profondeur défendue dans la problématique. Le détail de ces échanges est présenté en annexe (cf. Annexe I).

14.1.4 Données utilisées

Le modèle de données a été conçu dans un esprit de sobriété, cinq tables suffisant à couvrir l'ensemble du domaine. La table des comptes conserve l'identité du membre, l'empreinte de son mot de passe et sa clé d'API. La table des médias décrit chaque

fichier, distingue par un rôle les contenus des avatars et porte la date de publication qui commande la visibilité publique. La table des albums matérialise les regroupements créés par le membre. La table des placements réalise l'association ordonnée entre albums et médias en portant la position d'affichage. Enfin, la table des abonnements matérialise une relation réflexive entre comptes. Ce modèle a été formalisé selon le formalisme Merise (cf. Annexe D), traduit en modèle logique (cf. Annexe E).

Plusieurs choix d'intégrité et de performance méritent d'être soulignés, car ils traduisent des préoccupations de conformité et d'éco-responsabilité. La suppression d'un compte propage ses effets en cascade sur les médias, albums, placements et liens d'abonnement associés, tandis que les références dites douces, comme l'avatar d'un compte ou la couverture d'un album, sont ramenées à une valeur nulle plutôt que de bloquer l'opération. Cette sémantique de cascade est le socle technique du droit à l'effacement, la suppression d'un compte entraînant mécaniquement la disparition de l'ensemble de ses données, en base comme dans le stockage objet, ce qui répond directement aux exigences du règlement général sur la protection des données. Les tables d'association utilisent des clés primaires composites, sans identifiant technique superflu, ce qui allège le stockage et l'indexation. La taille des fichiers est stockée sur un entier long afin de supporter des vidéos volumineuses.

L'attention portée aux index illustre la démarche d'éco-conception. La galerie publique s'appuie sur un index partiel qui ne référence que les médias effectivement publiés et de type contenu. Les brouillons privés, majoritaires en volume, n'alourdissent donc pas cet index. Le même principe s'applique à la contrainte d'unicité de la clé d'API, restreinte aux comptes en possédant une. Ces index partiels réduisent l'empreinte de stockage et accélèrent les lectures les plus fréquentes, conciliant performance et maîtrise des ressources. L'ensemble du schéma est administré par une migration TypeORM versionnée, garantissant que la structure de la base est reproductible à l'identique d'un environnement à l'autre et rejouable automatiquement lors des déploiements.

14.1.5 Environnement informatique proposé

L'environnement d'exécution proposé pour l'application a été dimensionné au plus juste, dans une logique de maîtrise des coûts et de sobriété énergétique. La

persistance relationnelle repose sur PostgreSQL 18, système éprouvé et adapté aux besoins transactionnels du domaine. L'accès à la base n'est jamais direct. Il transite par PgBouncer, configuré en mode de mutualisation par transaction, afin de partager un nombre restreint de connexions physiques entre les processus applicatifs. Ce point est essentiel dans un contexte de mise à l'échelle horizontale, où plusieurs instances du backend coexistent. Sans mutualisation, chacune monopoliserait des connexions et saturerait rapidement le serveur de base de données. Le pool applicatif est lui-même borné et assorti de délais de garde explicites sur l'établissement des connexions et l'exécution des requêtes.

Le magasin clé-valeur Redis assume un rôle transversal. Il héberge les sessions opaques, sert de cache, fournit les verrous distribués utilisés lors des rotations de session et stocke les compteurs de limitation de débit. Le stockage et la diffusion des médias sont délégués à un service de stockage objet compatible S3, exposé au public via un réseau de diffusion de contenu et protégé par un pare-feu applicatif, la configuration correspondante (identifiant de distribution, domaine, nom du seau) étant lue au démarrage depuis un magasin de paramètres. Les nœuds d'exécution sont spécialisés par rôle (proxy, base de données, backend, frontend) et communiquent au sein d'un réseau privé, hébergement retenu au sein de l'Union européenne pour des raisons de conformité. Le détail de cette topologie, ainsi que les choix de conteneurisation qui la sous-tendent, sont documentés en annexe (cf. Annexes B et C) et approfondis dans le chapitre consacré à l'option DevOps.

14.1.6 Spécifications techniques

La conception applicative s'appuie sur un ensemble de patrons éprouvés dont la combinaison sert la testabilité, la sécurité et la maintenabilité. L'injection de dépendances, native au framework retenu, découple les contrôleurs des services et de la couche de persistance. Les dépendances sont déclarées et fournies par le conteneur, ce qui permet de les substituer aisément par des doublures lors des tests. La validation des entrées repose sur des objets de transfert de données annotés, contrôlés par un canal de validation appliqué globalement et configuré en liste blanche. Toute propriété non déclarée est rejetée, les types sont convertis et les contraintes de format sont vérifiées avant même que la logique métier ne s'exécute. Cette validation systématique constitue une première ligne de défense contre les entrées malformées ou malveillantes.

L'accès aux données combine le patron Repository et un constructeur de requêtes, ce qui autorise aussi bien des opérations simples que des requêtes analytiques finement optimisées, tout en conservant un typage strict. La pagination des listes s'effectue par curseur opaque, plus robuste et plus économe qu'une pagination par décalage sur de gros volumes. La sécurité applicative suit les recommandations de l'OWASP. Les mots de passe sont hachés avec l'algorithme Argon2id, résistant au calcul massif. Les en-têtes de sécurité, dont le transport strictement chiffré, sont posés par un intergiciel dédié. Les origines autorisées sont restreintes par une liste blanche. Le corps des requêtes est plafonné. Par ailleurs, la limitation de débit, adossée à Redis, s'applique globalement puis se resserre route par route sur les points sensibles. Les données secrètes, telles que l'empreinte du mot de passe et la clé d'API, sont exclues des lectures par défaut au niveau même du modèle. Enfin, l'API est intégralement documentée au format OpenAPI, la spécification étant générée à partir du code et des annotations de validation, ce qui garantit une documentation toujours synchronisée avec l'implémentation et exploitable pour outiller les intégrations (cf. Annexe Q). Les journaux d'accès HTTP, structurés en JSON, complètent ce dispositif en fournissant une matière directement exploitable par la chaîne d'observabilité.

14.2 Développement et Stratégie de Tests

14.2.1 Objectifs de la stratégie de tests

La stratégie de tests répond frontalement à l'exigence de qualité inscrite dans la problématique. Sans filet de sécurité automatisé, la livraison continue ne serait qu'une prise de risque répétée. Le premier objectif est la non-régression. Chaque évolution du code doit pouvoir être validée automatiquement contre un corpus de comportements attendus, afin qu'une correction ou une nouvelle fonctionnalité n'introduise pas silencieusement de défaut ailleurs. Cette garantie est la condition qui autorise à déployer fréquemment et sans crainte.

Le deuxième objectif est la vérification des règles de sécurité et des invariants métier. Les tests ne se cantonnent pas au chemin nominal, ils couvrent explicitement les limitations de débit, la validation des entrées, le contrôle d'appartenance des ressources et le comportement des sessions en situation d'attaque, ces propriétés étant précisément celles dont une défaillance aurait les conséquences les plus

graves. Le troisième objectif est la documentation vivante, un test lisible décrivant sans ambiguïté ce que le système est censé faire et servant de référence à l'équipe. Enfin, la stratégie vise l'intégration au pipeline, les tests étant conçus pour s'exécuter rapidement et de façon déterministe en intégration continue, où ils constituent une porte de qualité bloquante conditionnant tout déploiement.

14.2.2 Types de tests et outils utilisés

Le socle de tests automatisés rassemble deux cent dix-huit cas répartis selon plusieurs natures complémentaires. Les tests unitaires ciblent la logique des services, au premier rang desquels le service d'authentification, dont les mécanismes de rotation de session, de détection de rejeu et d'expiration sont vérifiés indépendamment de tout transport HTTP. Les tests de contrôleurs opèrent au niveau HTTP. Ils sollicitent l'application assemblée comme le ferait un client réel et valident le comportement de bout en bout des routes de chaque domaine (authentification, bibliothèque, profil, découverte, paramètres et périmètre public). Une suite est spécifiquement dédiée à la limitation de débit, propriété transversale que cette approche dédiée permet de couvrir de façon fiable, au-delà du seul niveau unitaire.

L'outillage retenu privilégie la simplicité et le déterminisme. Le cadre de test est Jest, associé au transpileur ts-jest qui permet d'exécuter directement les tests écrits en TypeScript sans étape de compilation intermédiaire. Les tests de niveau HTTP s'appuient sur supertest, qui pilote l'application et formule des assertions sur les réponses, statuts et en-têtes. Les dépendances externes sont neutralisées par des doublures. Le magasin Redis est remplacé par une implémentation en mémoire, la couche de persistance par des dépôts factices reproduisant le contrat attendu, et le service de médias par une doublure évitant tout appel réseau vers le stockage objet. Cette combinaison rend la suite entièrement autonome, sans dépendance à une base de données, à un magasin clé-valeur ou à un service tiers réels, ce qui garantit sa rapidité et sa reproductibilité, y compris en intégration continue.

14.2.3 Plan de test détaillé

Le plan de test couvre chaque domaine fonctionnel par une suite dédiée, en veillant à éprouver systématiquement les cas nominaux, les cas aux bornes et les cas d'erreur. Le tableau suivant en synthétise la structure et les scénarios représentatifs.

Suite de test	Périmètre couvert	Scénarios représentatifs
Authentification (service)	Cycle de vie des sessions	Création, résolution, expiration glissante et absolue, rotation par jeton, détection de rejeu et révocation globale
Authentification (contrôleur)	Inscription et connexion	Validation des identifiants, normalisation des noms, noms d'utilisateur réservés, conflits d'unicité, pose du cookie de session
Bibliothèque	Médias privés et albums	Publication et dépublication idempotentes, contrôle d'appartenance, gestion des albums et des placements, pagination par curseur
Profil	Profil public du membre	Mise à jour de la biographie, gestion de l'avatar, dépublication, restitution des compteurs
Découverte	Fil et abonnements	Fil global et fil des abonnements, suivi et arrêt du suivi, interdiction de se suivre soi-même
Paramètres	Compte et sécurité	Changement de mot de passe avec révocation des sessions, rotation et révocation de la clé d'API,

		suppression du compte
Périmètre public	Accès non authentifié	Recherche de profils, galeries et statistiques publiques, robustesse face aux entrées invalides
Limitation de débit	Protection transverse	Plafonnement des tentatives par route et cloisonnement des compteurs

Les scénarios de limitation de débit sont particulièrement instructifs, car ils vérifient des seuils différenciés selon la sensibilité des routes. La connexion est plafonnée à dix tentatives par minute, l'inscription à cinq par heure et la régénération de la clé d'API à cinq par minute, un compteur distinct étant maintenu pour chaque route afin qu'une route saturée ne pénalise pas les autres. De même, les tests de sécurité des sessions reproduisent explicitement un scénario de vol (réutilisation d'un ancien jeton) et vérifient que la plateforme révoque bien l'intégralité des sessions concernées. Ce plan traduit la volonté de traiter la sécurité et la robustesse comme des exigences testables au même titre que les fonctionnalités.

14.2.4 Environnement de test

L'environnement de test a été conçu pour être totalement hermétique, c'est-à-dire dépourvu de toute dépendance à des services externes réels. Cette propriété est déterminante. Elle rend la suite exécutable de façon identique sur le poste d'un développeur et dans le pipeline d'intégration continue, sans provisionnement préalable ni configuration d'accès. Le magasin Redis, central au fonctionnement des sessions et de la limitation de débit, est substitué par une implémentation en mémoire, ce qui permet d'éprouver fidèlement les comportements dépendants de Redis (expirations, verrous, compteurs) sans qu'un serveur soit démarré.

La couche de persistance est remplacée par des dépôts factices qui reproduisent le contrat des dépôts réels tout en conservant leurs données dans des structures en mémoire, de sorte que les tests de contrôleurs valident la logique applicative sans base de données. Chaque cas de test reconstruit un module applicatif isolé, avec ses doublures propres, garantissant l'absence d'effet de bord entre tests. L'exécution

est configurée pour être déterministe et séquentielle, ce qui écarte les aléas liés à la concurrence et rend les résultats parfaitement reproductibles. Cet environnement constitue le socle sur lequel s'appuie la porte de qualité du pipeline, décrite ci-après. Il constitue d'ores et déjà un dispositif éprouvé, que la démarche d'amélioration continue enrichira par des tests d'intégration exécutés contre des instances réelles éphémères et par la publication de seuils de couverture.

14.3 Modalités de livraison

14.3.1 Mise en service opérationnelle et vérification de service régulier (VSR)

La mise en service de la plateforme est entièrement automatisée par une chaîne d'intégration et de déploiement continu, dont le détail figure en annexe (cf. Annexe K). Le déploiement du service applicatif se déroule selon une séquence maîtrisée. Une étape de contrôles exécute d'abord la totalité des tests automatisés ainsi que la validation du code d'infrastructure, ces vérifications constituant une porte de qualité bloquante. Une étape de versionnement attribue ensuite à chaque service une étiquette dérivée de l'empreinte du dernier commit le concernant. L'infrastructure est alors réconciliée de façon déclarative, un déploiement effectif n'ayant lieu que si un écart avec l'état désiré est détecté. Les images conteneurisées sont construites et publiées, la reconstruction étant évitée lorsque l'image correspondante existe déjà. Enfin, la configuration des nœuds et le déploiement de la stack sont appliqués. Cette conception incarne les principes GitOps posés par le projet, à savoir que le dépôt Git fait autorité sur l'état désiré, que les images sont immuables et que les configurations sont versionnées par empreinte de contenu.

Deux mécanismes garantissent la continuité de service lors des mises à jour. D'une part, le déploiement s'effectue par mise à jour progressive selon une stratégie qui démarre la nouvelle version avant d'arrêter l'ancienne, assortie d'un retour arrière automatique en cas d'échec. Associée aux sondes de santé de chaque service, cette approche évite les interruptions perceptibles par les utilisateurs. D'autre part, les migrations de schéma sont exécutées après le déploiement, au moyen d'un conteneur éphémère raccordé au réseau interne du backend, ce qui découple l'évolution de la structure de données du redémarrage des instances applicatives. La plateforme expose par ailleurs un point de terminaison de santé dédié, sollicité par le

proxy et par l'orchestrateur pour n'acheminer le trafic que vers des instances effectivement disponibles.

La vérification de service régulier prolonge cette mise en service. Au-delà du succès technique du déploiement, il convient de s'assurer que le service rendu est conforme aux attentes dans la durée. Cette vérification s'appuie sur les journaux d'accès structurés et sur la chaîne d'observabilité (métriques, journaux et alertes) qui surveille les taux d'erreur, les temps de réponse et la disponibilité des cibles, jusqu'à la consommation énergétique estimée des nœuds (cf. Annexe P). Une anomalie détectée après mise en production, qu'il s'agisse d'une hausse des erreurs serveur ou d'une dégradation des latences, déclenche une alerte et peut motiver un retour arrière. La vérification de service régulier constitue ainsi le point de bouclage entre la livraison et l'exploitation, en confirmant que la valeur attendue est effectivement délivrée aux utilisateurs.

Au-delà de la mise en service, un plan de maintenance périodique encadre l'exploitation dans la durée et prépare les audits de conformité. Il fixe des échéances explicites : une revue mensuelle des correctifs de sécurité et des dépendances, appuyée sur les rapports d'analyse produits à chaque exécution de la chaîne, un exercice trimestriel de restauration des sauvegardes, une revue semestrielle de la conformité RGPD et de la cartographie ISO/IEC 27001 version 2022, et une revue annuelle complète de sécurité tenant lieu de préparation à un audit externe. Chaque échéance donne lieu à un compte rendu daté, versionné avec le code dans la documentation technique du dépôt, de sorte que la traçabilité exigée en prévision d'un audit soit garantie.

14.3.2 Conclusion

La conception de dropicture montre que la livraison continue d'une plateforme SaaS ne se décrète pas au seul niveau de l'infrastructure, elle se prépare dès l'architecture applicative. La décomposition en monorepo et en contrôleurs de domaine circonscrit les évolutions, l'injection de dépendances et la validation systématique rendent le code testable et sûr, le modèle relationnel sobre porte les exigences de conformité et d'éco-conception, et les patrons de sécurité appliqués par construction matérialisent la défense en profondeur. Cette conception rigoureuse est ce qui rend possible une automatisation sereine.

La stratégie de tests, avec ses deux cent dix-huit cas couvrant services, contrôleurs, validation et limitation de débit, fournit le filet de sécurité sans lequel la fréquence des déploiements serait un facteur de risque plutôt qu'un atout. Les modalités de livraison, appuyées sur une chaîne GitOps, une mise à jour progressive réversible, des migrations post-déploiement et une vérification de service régulier, ferment la boucle entre développement et exploitation. Ainsi, la conception et le développement applicatifs répondent directement à la problématique en conjuguant qualité, sécurité, observabilité et maîtrise des coûts. Le chapitre consacré à l'option DevOps approfondira la manière dont cette application est concrètement automatisée, orchestrée et supervisée en production.

17. Concevoir et déployer des infrastructures DevOps automatisées (Option DevOps)

Cette section constitue le cœur de l'option évaluée. Elle répond directement à la problématique (*concevoir et déployer une infrastructure DevOps entièrement automatisée, sécurisée et éco-responsable, permettant la livraison continue d'une plateforme SaaS de partage de médias tout en garantissant la qualité, l'observabilité et la maîtrise des coûts*) en la déclinant en quatre volets complémentaires. Le premier porte sur l'analyse de l'existant et la formalisation d'une stratégie DevOps (compétence C28), le deuxième sur la conception des pipelines de livraison continue (C29), le troisième sur l'orchestration des environnements conteneurisés (C30) et le quatrième sur la mise en œuvre d'une chaîne d'observabilité (C31). Chaque choix technique est ancré dans la plateforme dropicture, dont l'infrastructure réelle sert de terrain de démonstration. Le fil conducteur reste constant d'un bout à l'autre. Tout ce qui peut être décrit, versionné et rejoué l'est, tout ce qui touche à la donnée ou au réseau est cloisonné, et tout ce qui consomme des ressources est mesuré, y compris son empreinte énergétique.

17.1 Analyse de l'infrastructure et des processus de développement

L'automatisation ne se décrète pas, elle procède d'un diagnostic préalable de l'existant et des frictions à supprimer. Ce premier volet établit ce diagnostic et en déduit une stratégie DevOps alignée à la fois sur les objectifs de la plateforme (livrer vite et sans régression un SaaS de partage de médias) et sur une exigence transverse d'éco-responsabilité. Il fixe le cadre méthodologique dont découlent les trois volets suivants.

Le point de départ est un monorepo structuré en trois livrables aux contraintes d'exploitation distinctes, à savoir une API apps/saas/backend (NestJS 11 exécuté sous Bun), une application authentifiée apps/saas/frontend (Next.js 16) et un site vitrine apps/website (Next.js 16 en export statique). Cette hétérogénéité impose des chaînes de déploiement différenciées plutôt qu'un pipeline unique, car l'API et l'application tournent en conteneurs sur un cluster tandis que le site, purement statique, se distribue via un stockage objet adossé à un CDN. L'infrastructure sous-jacente est par nature multi-fournisseurs, avec Hetzner Cloud pour le calcul, Cloudflare pour la périphérie réseau et le DNS, et AWS pour le stockage des médias,

des sauvegardes et de l'état d'infrastructure. Cette dispersion, dès lors qu'elle est maîtrisée par le code, cesse d'être une source de dérive de configuration et d'erreurs manuelles pour devenir un atout de pilotage unifié.

L'analyse de l'existant fait ressortir six axes de progrès, chacun appelant une réponse d'ingénierie précise. Sur le plan de la livraison, des déploiements manuels et non reproductibles appellent un pipeline CI/CD déterministe. Sur le plan de l'infrastructure, la multiplicité des fournisseurs impose une description déclarative unifiée, autrement dit une approche Infrastructure as Code. Concernant les environnements, la volonté de supprimer tout écart entre développement et production justifie la conteneurisation et le recours à des images immuables. La prévention des régressions fonctionnelles rend nécessaires des tests automatisés adossés à des portes de qualité bloquantes, tandis que la maîtrise de la chaîne d'approvisionnement logicielle appelle des analyses de sécurité automatisées portant à la fois sur les dépendances, sur les plans d'infrastructure et sur les images produites. Le besoin de visibilité en exploitation appelle une chaîne de métriques, de journaux et d'alertes. Enfin, une consommation de ressources jusqu'alors non mesurée motive une démarche de dimensionnement au plus juste assortie d'un indicateur énergétique. Ces constats et les besoins associés sont formalisés dans la stratégie DevOps du projet et repris, sous l'angle des décisions d'architecture, dans le registre correspondant (cf. Annexe J).

Le principe structurant retenu est la séparation nette entre le *provisioning* et la *configuration*. Terraform assume exclusivement l'approvisionnement des ressources, à savoir les serveurs Hetzner déclarés par rôle et par nombre de réplicas, le réseau et le sous-réseau privés, le pare-feu, les réglages Cloudflare, les buckets S3, les distributions CloudFront et les paramètres SSM. Ansible prend le relais pour tout ce qui relève de la configuration d'un système déjà provisionné, à savoir l'installation du moteur Docker depuis un dépôt signé, le durcissement SSH, le chargement de modules noyau et les réglages sysctl, l'initialisation du cluster Swarm sur le réseau privé, la jointure des nœuds, la pose des labels de rôle, la gestion de la PKI d'origine, la distribution des configs et secrets, le déploiement de la pile applicative et l'exécution des migrations. Cette frontière n'est pas cosmétique, car elle garantit que la topologie matérielle et l'état logiciel évoluent selon deux cycles de vie distincts, chacun idempotent, ce qui limite le rayon d'impact d'un changement et clarifie les

responsabilités. Elle sert directement l'axe *automatisation* de la problématique en rendant l'ensemble de la chaîne rejouable à l'identique.

La fiabilité de cette approche repose sur la gestion de l'état Terraform. L'état n'est jamais conservé localement. Il est stocké dans un bucket S3 dédié (dropicture-tfstate-prod), chiffré au repos (encrypt = true) et protégé par un verrou natif (use_lockfile = true) qui interdit deux applications concurrentes. Le bucket est versionné et intégralement soustrait à tout accès public. Ce dispositif adresse simultanément deux axes de la problématique. Il sert la *sécurité*, car l'état contient des sorties sensibles et ne doit jamais fuiter, et il sert la *qualité*, car le verrou élimine toute possibilité de corruption de l'état par des exécutions simultanées. Il constitue par ailleurs le socle du plan de reprise d'activité, l'infrastructure pouvant être reconstruite à l'identique à partir de cet état et des playbooks associés.

La stratégie DevOps qui découle de ce diagnostic tient en quelques principes directeurs. Le premier est le *tout en code*, par lequel infrastructure, configuration, définitions de pipelines, règles d'alerte et tableaux de bord d'observabilité sont versionnés au même titre que le code applicatif. Le deuxième est le modèle *GitOps*, où le dépôt Git fait autorité sur l'état désiré et où la convergence vers cet état est déclenchée par un événement traçable. Le troisième est l'immutabilité des artefacts, chaque image étant identifiée par une empreinte de commit. Le quatrième est le déploiement progressif sans interruption, assorti d'un retour arrière automatique. Le cinquième érige l'observabilité en exigence de premier ordre, présente dès le premier déploiement et non ajoutée après coup. Le sixième intègre la sécurité au plus tôt dans la chaîne, selon une logique *shift-left*.

À ces principes s'ajoute une dimension d'éco-responsabilité assumée comme un objectif d'ingénierie à part entière, et non comme un supplément déclaratif. Elle se traduit par un dimensionnement au plus juste des ressources (des réservations et limites calibrées service par service pour éviter toute sur-allocation), par un cycle de vie du stockage qui bascule les sauvegardes vers des classes de moins en moins coûteuses puis vers l'archivage froid, par un cache immuable et une compression au niveau du CDN qui réduisent les transferts, par des index partiels en base qui n'indexent que les lignes utiles, et par le choix d'un hébergement en Union européenne à la classe de distribution CDN volontairement restreinte. Surtout, elle est instrumentée. La consommation électrique du cluster est estimée en continu et

surveillée, ce qui relie directement les axes *éco-responsabilité*, *observabilité* et *maîtrise des coûts*. Ce lien entre right-sizing, mesure et coût constitue l'originalité de la démarche et sera détaillé au volet 17.4.

À cette dimension environnementale répond une exigence d'accessibilité tout aussi structurante, traitée comme un critère de conception et non comme une mise en conformité tardive. Elle porte d'abord sur les surfaces exposées aux utilisateurs, où les référentiels WCAG 2.2 et RGAA 4 guident la structure sémantique des pages, la hiérarchie des titres, l'étiquetage explicite des champs de formulaire, les alternatives textuelles des médias, la navigation complète au clavier, la visibilité du focus et le respect des contrastes, l'application authentifiée comme le site public déclarant leur langue et recourant aux rôles et attributs ARIA nécessaires à la restitution par lecteur d'écran. Elle porte ensuite sur les artefacts propres à la chaîne DevOps, souvent oubliés des démarches d'accessibilité alors qu'ils conditionnent la capacité d'un exploitant en situation de handicap à opérer le système. La documentation technique et les procédures d'automatisation sont donc rédigées en texte structuré et versionné plutôt qu'en captures d'écran, chaque schéma d'architecture est doublé d'une description textuelle de son contenu, les journaux applicatifs sont émis dans un format structuré exploitable par des outils tiers, et les tableaux de bord de supervision évitent de reposer sur la seule couleur pour transmettre une information, un seuil ou un état d'alerte étant toujours accompagné d'une valeur numérique et d'un intitulé. Les comptes rendus de revue et de jalon sont enfin produits sous forme écrite, afin que la contribution de chaque partie prenante soit recueillie sans discrimination. Cette exigence, portée par la matrice d'engagement des parties prenantes (cf. Annexe O bis), reste perfectible et son prolongement naturel consiste à intégrer un contrôle d'accessibilité automatisé aux portes de qualité de la chaîne, au même titre que les analyses de sécurité, perspective inscrite au plan d'amélioration continue.

En synthèse, l'analyse débouche sur une infrastructure entièrement descriptible, reproductible et audité, où chaque fournisseur est piloté par du code plutôt que par des gestes manuels. Cette fondation conditionne la suite, puisque sans description déclarative de l'existant, ni pipeline reproductible, ni orchestration cohérente, ni observabilité n'auraient de socle stable. Le volet suivant montre comment cette stratégie se matérialise dans des pipelines de livraison continue.

17.2 Conception et implémentation des pipelines

Les pipelines sont le point où la stratégie *tout en code* devient un mécanisme de livraison effectif. Ils répondent aux axes *automatisation* et *qualité* de la problématique en transformant un déclenchement unique en une séquence déterministe d'étapes vérifiées, et à l'axe *sécurité* en confinant les secrets et les permissions. La plateforme dropicture s'appuie sur GitHub Actions, avec un pipeline principal pour le SaaS conteneurisé, un pipeline dédié au site statique et un pipeline ciblé pour le périmètre CDN et sauvegardes. Le détail complet des workflows figure en annexe (cf. Annexe K).

Le pipeline `saas-deploy` orchestre six étapes chaînées selon leurs dépendances réelles, à savoir `checks`, `versions`, `terraform`, `build`, `provision` et `deploy`. L'étape `checks` constitue la première porte de qualité et de sécurité. Elle exécute la suite Jest de la plateforme (218 tests couvrant contrôleurs, services et limitation de débit) via `bun run test --ci`, valide la syntaxe et la mise en forme de l'infrastructure par `terraform fmt -check -recursive` et `terraform validate`, puis enchaîne deux analyses de sécurité bloquantes. La première examine l'arborescence du dépôt à la recherche de vulnérabilités connues dans les dépendances et de secrets accidentellement versionnés. La seconde soumet les plans d'infrastructure eux-mêmes, qu'il s'agisse des fichiers Terraform, des playbooks Ansible ou des Dockerfile, à une analyse statique de configuration. Un échec sur l'un quelconque de ces contrôles bloque l'ensemble, aucune image n'étant construite et aucun serveur n'étant touché tant que le code, l'infrastructure et la chaîne d'approvisionnement logicielle ne sont pas jugés sains. L'étape `versions` calcule ensuite, pour chaque service, une étiquette d'image égale au SHA du dernier commit ayant modifié son répertoire (`git log -1 --format=%H -- apps/saas/<service>`). Cette dérivation est le pivot de l'immutabilité, puisque deux déploiements successifs sans changement de code réutilisent rigoureusement la même image tandis qu'un changement produit une étiquette nouvelle et univoque.

L'étape `terraform` applique la logique GitOps de convergence. La commande `terraform plan -detailed-exitcode` distingue trois issues par son code de sortie, à savoir aucun changement, un changement à appliquer, ou une erreur. L'application (`apply`) n'a lieu que dans le deuxième cas, et le résultat de cette décision (la présence ou non de changements d'infrastructure) est propagé aux étapes ultérieures. Ce détail conditionne toute l'économie du pipeline, car l'étape `provision`

(le playbook Ansible qui reconfigure les nœuds et le cluster Swarm) n'est déclenchée que si Terraform a effectivement modifié la topologie. Reconfigurer des serveurs inchangés serait à la fois inutile et risqué. La provision conditionnelle sert donc autant l'efficacité (donc la maîtrise des coûts et l'axe *éco-responsabilité*) que la stabilité.

L'étape build illustre une seconde optimisation, l'idempotence de la construction. Menée en matrice sur les services frontend et backend, elle interroge d'abord le registre GHCR (docker manifest inspect). Si l'image portant l'étiquette SHA existe déjà, la construction est purement et simplement sautée. Dans le cas contraire, l'image est bâtie avec le cache de couches de GitHub Actions (cache-from/cache-to de portée type=gha) puis poussée sous deux étiquettes, le SHA immuable et latest. Le gain est double, avec un temps de pipeline réduit et une consommation de calcul évitée, ce qui rejoint là encore la sobriété recherchée. L'étape deploy, enfin, ne s'exécute qu'une fois les versions résolues, les images disponibles et la provision réussie ou légitimement ignorée. Elle déploie la pile puis exécute les migrations TypeORM *après* le déploiement, dans un conteneur éphémère raccroché au réseau interne de la base. Ce séquençage (rollout d'abord, migration ensuite, sur un schéma additif) évite qu'une migration ne s'applique à une infrastructure qui ne serait pas encore prête à la recevoir. La migration n'est d'ailleurs lancée qu'après une sonde de disponibilité de la base, une boucle pg_isready répétée sur le réseau overlay interne s'assurant que PostgreSQL répond avant toute tentative. L'exécution est en outre traitée comme idempotente, le playbook ne rapportant un changement que lorsque des migrations étaient effectivement en attente. Le déploiement peut ainsi être rejoué sans effet de bord, conformément à l'exigence de reproductibilité.

L'analyse de sécurité ne s'arrête pas au dépôt, elle suit l'artefact. Chaque image construite est soumise, avant toute publication effective vers l'environnement de production, à une analyse de son contenu portant sur les vulnérabilités de ses paquets système et applicatifs ainsi que sur d'éventuels secrets embarqués, avec un seuil de blocage fixé aux sévérités critiques et élevées et une restriction aux vulnérabilités effectivement corrigeables en amont, afin que la porte reste actionnable plutôt que systématiquement rouge. La chaîne produit en outre, pour chaque image, une nomenclature logicielle au format CycloneDX qui recense l'intégralité des composants embarqués et de leurs versions. Cette nomenclature

constitue l'analyse de composition logicielle attendue d'une démarche DevSecOps et fournit, en cas de divulgation d'une vulnérabilité majeure, le moyen de déterminer immédiatement si un artefact déployé est concerné. Les résultats de l'ensemble de ces analyses sont convertis au format SARIF et publiés dans le service d'analyse de code de la forge, ce qui les rend consultables, historisés et rattachés aux lignes de code concernées, tandis que les rapports détaillés au format JSON sont archivés en artefacts pendant quatre-vingt-dix jours et qu'une synthèse par niveau de sévérité est portée dans le compte rendu d'exécution. Les exceptions éventuelles sont enfin centralisées dans un fichier dédié versionné, chaque dérogation devant porter son identifiant, sa justification et sa date de revue, de sorte qu'une acceptation de risque demeure explicite, tracée et temporaire (cf. Annexe S).

Les portes de qualité ne se limitent pas au SaaS. Le pipeline du site statique, `website-deploy`, déclenché sur modification des chemins concernés, introduit une vérification spécifique et déterminante. Il contrôle que les variables publiques `NEXT_PUBLIC_*` sont bien des URL HTTPS absolues et qu'elles ont été *inlinées* dans le bundle exporté. Concrètement, après le build il recherche la présence littérale des URL attendues dans les fichiers produits et échoue si elles sont absentes, ce qui prévient la mise en ligne d'un site pointant vers une API indéfinie. Le pipeline `saas-cdn`, quant à lui, est un exemple de garde-fou explicite. Restreint par `-target` au seul périmètre CDN et sauvegardes, il refuse tout plan qui toucherait une ressource Hetzner (hors périmètre) ou qui détruirait un bucket ou une distribution critique, en inspectant le plan au format JSON avant application. Ces garde-fous matérialisent l'axe *sécurité* au niveau de la chaîne elle-même, le pipeline se protégeant contre ses propres dérives.

Les pratiques GitOps irriguent l'ensemble. Git est la source de vérité de l'état désiré (infrastructure, pile applicative, configurations d'observabilité, règles d'alerte, tableaux de bord). L'immutabilité s'étend au-delà des images, puisque les configs et secrets de Swarm sont versionnés par empreinte de leur contenu. Chaque configuration de supervision est nommée à partir d'un condensé SHA-1 de son fichier (par exemple `dropicture_prometheus_rules_<hash>`). Une modification produit un objet de nom différent, ce qui autorise une rotation sûre sans écrasement en place d'une ressource montée par un conteneur en cours d'exécution. Le playbook va jusqu'à refuser explicitement de remplacer en place un certificat d'origine déjà

stocké dans le cluster, imposant une rotation par changement de nom versionné. Cette discipline de l'immuabilité, appliquée aussi bien au code qu'à la configuration, est ce qui rend les déploiements prévisibles et réversibles.

La sécurité de la chaîne repose enfin sur trois garanties concrètes. Les secrets sont injectés depuis les environnements GitHub (jetons Hetzner et Cloudflare, identifiants AWS, clé SSH, mots de passe de base et de supervision) et n'apparaissent jamais dans le dépôt, le playbook Ansible vérifiant d'ailleurs leur présence par des assertions avant tout déploiement, tandis que l'analyse de détection de secrets décrite plus haut agit comme un filet de sécurité automatique contre une fuite accidentelle. Les permissions accordées aux jobs sont minimales, avec une lecture seule du contenu par défaut, une écriture sur les paquets réservée à la seule étape de construction et une écriture sur les événements de sécurité limitée aux étapes qui publient les rapports d'analyse. La sérialisation des déploiements est assurée par un groupe de concurrence qui interdit deux exécutions simultanées du même pipeline. La chaîne de livraison est ainsi automatisée de bout en bout, vérifiée et analysée à chaque étape, économe en ressources et défendue contre les manipulations dangereuses, ce qui constitue une réponse directe et vérifiable aux axes automatiser, qualité et sécurité. Reste à examiner la cible de ces déploiements, à savoir l'environnement conteneurisé orchestré.

17.3 Orchestration des environnements conteneurisés

L'orchestration est l'échelon où les images immuables produites par les pipelines deviennent un service en fonctionnement, résilient et dimensionné. Elle répond à l'axe *qualité* de la problématique par la reproductibilité des environnements, à l'axe *coûts* par un placement et un dimensionnement fins, et à l'axe *sécurité* par un cloisonnement réseau strict. La plateforme dropicture s'appuie sur Docker Swarm, dont la topologie complète et la segmentation réseau sont détaillées en annexe (cf. Annexes B et C).

Le choix de Docker Swarm plutôt que de Kubernetes procède d'une analyse de proportionnalité. Pour une plateforme opérée par une équipe restreinte, sur un parc de quelques nœuds, Kubernetes introduirait une complexité opérationnelle (plan de contrôle à maintenir, écosystème d'objets et de contrôleurs, surcoût de calcul permanent) sans contrepartie fonctionnelle décisive à cette échelle. Swarm, intégré

au moteur Docker, offre nativement les primitives réellement nécessaires, à savoir le placement par contraintes, la mise à jour progressive avec retour arrière, les réseaux overlay chiffrables et les configs et secrets gérés. Ce choix illustre une décision d'architecture guidée par la sobriété, qui retient l'outil dont la puissance correspond au besoin réel plutôt que le standard le plus lourd, ce qui réduit d'autant la surface d'exploitation et la consommation de ressources. La portabilité reste préservée, la description de la pile demeurant un fichier Compose déclaratif et les images étant conformes à l'OCI.

Le placement des services obéit à des labels de rôle posés par Ansible sur chaque nœud (`node.labels.dropicture.role`), avec quatre rôles que sont proxy, db, backend et frontend. Chaque service est contraint sur le rôle qui lui convient, à savoir la base de données et son pooler sur le nœud db, le reverse proxy et la pile d'observabilité sur le nœud proxy, et les instances applicatives sur leurs nœuds respectifs. Ce placement explicite sépare les charges par nature, l'état persistant restant isolé, la charge applicative étant répartie, et l'infrastructure de supervision ne concurrençant pas les services qu'elle observe. Il conditionne aussi la résilience, en évitant qu'une défaillance de nœud n'emporte des rôles hétérogènes.

La constitution même du cluster obéit à la même exigence de rigueur. Le Swarm est initialisé sur le réseau privé Hetzner, le manager annonçant et écoutant son trafic de contrôle sur son adresse IP privée, avec un espace d'adressage overlay dédié, si bien que le plan de contrôle du cluster (port 2377) n'est jamais exposé sur l'interface publique. Les workers rejoignent le cluster par cette même adresse privée et l'automatisation attend que chacun rapporte effectivement un état actif avant de poursuivre. Un garde-fou vérifie que l'adresse réellement annoncée par le Swarm coïncide avec l'adresse privée déclarée par Terraform, et échoue explicitement si un changement d'IP a rendu cette annonce obsolète (une situation qui, silencieusement, empêcherait tout worker de joindre le manager). La cohérence entre l'inventaire Terraform et l'appartenance au cluster est enfin réconciliée automatiquement. Les nœuds encore connus du Swarm mais disparus de la description Terraform sont retirés, mais seulement après plusieurs sécurités, à savoir un refus de réconcilier tant que chaque nœud géré n'a pas rapporté son identifiant et un avertissement avant le retrait d'un nœud à état (base ou proxy) qui devrait être vidé au préalable. Cette

réconciliation prudente évite qu'une indisponibilité passagère ne soit interprétée à tort comme une suppression, et illustre le soin porté à la résilience de l'orchestration.

La segmentation réseau constitue le cœur de la posture de sécurité au niveau de l'orchestration. Cinq réseaux overlay isolent les flux selon le principe du moindre privilège. Le réseau frontend porte le trafic entrant depuis le proxy vers les applications, backend relie l'API à la base et au cache, socket isole l'accès filtré à l'API Docker, egress porte les sorties vers Internet et AWS, et monitoring transporte la télémétrie. Deux de ces réseaux sont déclarés internal, donc dépourvus de toute route vers l'extérieur. Il s'agit de backend, de sorte que la base PostgreSQL et le cache Redis ne sont jamais joignables depuis Internet, et de socket, de sorte que l'API Docker demeure inatteignable hors du cluster. Le reverse proxy Traefik n'accède d'ailleurs jamais directement au socket `/var/run/docker.sock`, puisqu'il passe par un socket-proxy en lecture seule qui n'expose que les points d'accès strictement nécessaires à la découverte des services. Ce cloisonnement, combiné à une MTU réduite adaptée au réseau privé Hetzner, réalise concrètement une défense en profondeur au niveau applicatif, en complément des protections de périphérie décrites ailleurs dans le mémoire.

La disponibilité pendant les déploiements repose sur une politique de mise à jour progressive différenciée selon la nature du service. Les services applicatifs sans état (frontend et backend) se mettent à jour en mode start-first, la nouvelle réplique étant démarrée et jugée saine avant que l'ancienne ne soit retirée, ce qui évite toute interruption de service. Les services à état (base, pooler, cache) utilisent au contraire un mode stop-first, l'ancienne instance étant arrêtée avant la nouvelle pour préserver la cohérence des données. Dans les deux cas, un intervalle de surveillance encadre le rollout et déclenche un retour arrière automatique en cas d'échec (la clé `failure_action` réglée sur `rollback`), la configuration de rollback réappliquant l'ordre approprié. Cette réversibilité automatique est une garantie de qualité opérationnelle, puisqu'un déploiement défaillant se rétracte de lui-même vers le dernier état sain, sans intervention manuelle. La mise à l'arrêt gracieuse des instances applicatives (avec un délai de grâce prolongé sur le backend) laisse par ailleurs aux requêtes en cours le temps de se terminer.

Chaque service déclare des réservations et des limites de CPU et de mémoire. Les réservations garantissent le placement et matérialisent l'exigence d'éco-conception,

en réservant exactement les ressources nécessaires. Les limites protègent le cluster contre l'emballlement d'un conteneur en plafonnant sa consommation. Ce dimensionnement au plus juste, calibré service par service, est directement relié à l'axe *maîtrise des coûts*, car il maximise la densité utile du parc sans sur-provisionnement. À cela s'ajoutent des sondes de santé (healthcheck) sur l'ensemble des services, qui conditionnent le succès des rollouts et alimentent la répartition de charge active de Traefik selon une stratégie *power-of-two-choices*.

La montée en charge horizontale, enfin, est pilotée par l'infrastructure elle-même. Le nombre de répliques des services frontend et backend n'est pas figé dans le fichier Compose, car il est *dérivé* du nombre de nœuds correspondants déclarés dans Terraform. Le playbook de déploiement télécharge l'état Terraform, y dénombre les nœuds de chaque rôle et injecte ces comptes dans le rendu du Compose, avec des assertions de cohérence qui refusent de déployer si la topologie attendue n'est pas réunie. Ajouter un nœud backend dans la description Terraform suffit ainsi à augmenter le nombre de répliques au déploiement suivant, le scaling devenant une propriété déclarative de l'infrastructure, cohérente avec l'approche *tout en code*. Les images, construites en multi-étapes sur une base Bun/Alpine, restent légères, ce qui accélère les rollouts et réduit l'empreinte de stockage du registre. Portabilité, scalabilité, résilience et sobriété de ressources sont ainsi traitées comme les facettes d'une même conception. Une infrastructure ainsi orchestrée n'a toutefois de valeur opérationnelle que si son comportement est observable, ce qui fait l'objet du dernier volet.

17.4 Mise en œuvre d'une stratégie de monitoring et d'observabilité

L'observabilité est ce qui transforme une infrastructure automatisée en une infrastructure *pilotable*. Elle répond à l'axe *observabilité* de la problématique en rendant visibles l'état et les performances du système, à l'axe *qualité* en permettant d'anticiper les incidents plutôt que de les subir, et (trait distinctif de la plateforme dropicture) à l'axe *éco-responsabilité* en instrumentant la consommation énergétique du cluster. Les tableaux de bord et l'ensemble des règles sont présentés en annexe (cf. Annexe P).

La chaîne d'observabilité repose sur une association d'outils libres, versionnés comme le reste de l'infrastructure. Prometheus collecte les métriques toutes les

quinze secondes, avec une rétention bornée à quinze jours ou dix gigaoctets. Ses cibles couvrent Prometheus lui-même, Alertmanager, le reverse proxy Traefik (entrypoints, services, routeurs), le collecteur node-exporter par nœud et cAdvisor par conteneur. Ces deux derniers sont déployés en mode global, c'est-à-dire présents sur chaque nœud du cluster, afin de garantir une couverture homogène du parc. Les journaux, eux, sont pris en charge par Loki alimenté par Alloy, également en mode global. Alloy découvre les conteneurs via le socket Docker, réétiquette les flux selon le service, la pile, le conteneur et le nœud, puis les pousse vers Loki. La collecte des journaux est d'autant plus exploitable que le backend émet des journaux d'accès HTTP structurés en JSON (méthode, route, statut, adresse IP, identifiant d'utilisateur, quota et durée), ce qui autorise des recherches et des agrégations fines plutôt qu'un simple flux textuel. Grafana, enfin, restitue l'ensemble à travers trois tableaux de bord provisionnés par code, à savoir une vue d'ensemble (nœuds actifs, conteneurs, requêtes par seconde, taux de 5xx, ressources par nœud et par service, latence au 95^e percentile), une vue dédiée aux journaux et une vue *énergie*.

L'alerting, opéré par Alertmanager, formalise dix règles qui couvrent les principaux modes de défaillance. La disponibilité est surveillée par deux règles critiques déclenchées lorsqu'une cible ou un nœud cesse de répondre pendant cinq minutes. La santé des nœuds est couverte par quatre règles, à savoir une charge CPU soutenue au-delà de 90 %, une mémoire disponible passée sous 10 %, une saturation disque imminente détectée par projection linéaire sur vingt-quatre heures, et une saturation disque critique sous 10 % d'espace libre. La santé applicative est suivie via Traefik et cAdvisor par trois règles, à savoir un taux d'erreurs serveur (5xx) supérieur à 5 %, une latence au 95^e percentile dépassant une seconde, et des redémarrages répétés d'un conteneur trahissant une boucle d'échec. À ces neuf règles s'ajoute une dixième, propre à la démarche d'éco-responsabilité, portant sur la consommation électrique estimée. Alertmanager regroupe, déduplique et route ces alertes vers un récepteur configurable (courriel ou messagerie d'équipe), et celles-ci restent consultables dans Grafana. La combinaison de métriques, de journaux structurés et d'alertes constitue une observabilité complète, orientée vers l'anticipation des incidents et l'optimisation continue des performances. Les seuils de latence et d'erreur signalent une dégradation avant qu'elle ne devienne une panne,

tandis que les métriques de ressources guident le réajustement des réservations et limites évoquées au volet précédent.

Le volet énergétique constitue la contribution la plus singulière de cette stratégie d'observabilité et matérialise concrètement l'ambition green IT de la problématique. Dans un environnement de machines virtuelles louées chez un fournisseur cloud, les compteurs matériels RAPL (/sys/class/powercap) demeurent propres au bare-metal, ce qui appelle une approche logicielle de la mesure. La plateforme met en œuvre à cet effet un dispositif d'estimation documenté et pleinement opérationnel, dérivé de l'utilisation CPU relevée par node-exporter selon un modèle linéaire explicite. La puissance d'un nœud est modélisée comme le produit de son nombre de vCPU par une puissance unitaire interpolée entre un plancher au repos et un plafond à pleine charge, selon la formule $P(\text{nœud}) = \text{nb_vCPU} \times (P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) \times \text{utilisation})$, avec $P_{\text{idle}} = 1,5 \text{ W}$ et $P_{\text{max}} = 6 \text{ W}$ par vCPU. Ce modèle est implémenté sous forme de règles d'enregistrement Prometheus qui calculent la puissance par nœud, l'agrègent à l'échelle du cluster, en projettent une énergie journalière en kilowattheures, et en déduisent une empreinte carbone horaire en grammes de CO₂ équivalent, sur la base d'une intensité de 380 gCO₂eq/kWh correspondant au mix électrique allemand du centre de données. Une règle d'alerte dédiée, *ConsommationEnergetiqueElevee*, se déclenche lorsque la puissance estimée du cluster dépasse un seuil soutenu de 200 W pendant quinze minutes, invitant à rechercher une charge anormale (boucle CPU, scraping excessif, traitement par lots) et à envisager un redimensionnement.

Cette instrumentation énergétique relie explicitement plusieurs axes de la problématique. Elle sert l'*éco-responsabilité* en donnant une visibilité chiffrée sur une consommation jusqu'alors non mesurée. Elle sert la *maîtrise des coûts*, la puissance consommée étant un proxy direct de la facture d'exploitation. Elle sert enfin l'*observabilité*, en transformant une préoccupation environnementale en une métrique de premier ordre, surveillée et alertée au même titre que la latence ou le taux d'erreur. Ses paramètres restent pleinement ajustables selon le parc réel, et la documentation du projet trace le palier d'évolution naturel du dispositif. Sur un déploiement *bare-metal*, des agents tels que Scaphandre ou Kepler viendraient enrichir cette instrumentation d'une mesure RAPL matérielle, prolongeant l'estimation

par une lecture directe des compteurs. Cette transparence sur les hypothèses du modèle constitue elle-même un gage de rigueur.

Cette instrumentation est en outre exploitée par une assistance d'intelligence artificielle dédiée à l'optimisation des ressources. Les séries de consommation processeur, mémoire et énergétique collectées par Prometheus sont analysées par un outil d'IA qui met en évidence les services sur-dimensionnés ou sous-dimensionnés et recommande des ajustements de réservations et de limites, ainsi que des créneaux propices aux traitements par lots. Validées par l'ingénieur avant application, ces recommandations ont permis d'affiner le dimensionnement au plus juste service par service et de contenir la consommation énergétique estimée du cluster, reliant concrètement l'intelligence artificielle, l'éco-responsabilité et la maîtrise des coûts.

Au terme de ces quatre volets, la plateforme dropicture illustre une infrastructure DevOps où l'automatisation, la sécurité, la qualité, l'observabilité, la maîtrise des coûts et l'éco-responsabilité ne sont pas des objectifs concurrents mais les facettes d'une même conception cohérente. L'analyse a fixé une stratégie tout en code et GitOps, les pipelines l'ont rendue exécutable, vérifiée et analysée jusqu'à la chaîne d'approvisionnement logicielle, l'orchestration l'a matérialisée en un service résilient et cloisonné, et l'observabilité l'a rendue pilotable, jusqu'à sa propre empreinte énergétique. L'ensemble apporte une réponse complète et démontrable à la problématique, tout en traçant lui-même ses perspectives d'évolution continue (couverture de tests mesurée en intégration continue, haute disponibilité de la supervision, contrôle d'accessibilité automatisé, mesure énergétique matérielle), qui sont examinées dans la section consacrée aux perspectives.

18. Analyses et Perspectives

18.1 Analyse des résultats et des performances

18.1.1 Évaluation des objectifs atteints

L'évaluation des résultats s'opère au regard des cinq objectifs opérationnels formulés à partir de la problématique. Le premier objectif, décrire l'intégralité de l'infrastructure sous forme de code reproductible, est pleinement atteint. L'ensemble des ressources, serveurs, réseau, pare-feu, DNS, distribution de contenu, stockage et sauvegardes, est déclaré en Terraform, tandis que la configuration du cluster et le déploiement de la stack sont pilotés par Ansible de manière idempotente. Aucune opération manuelle n'est requise pour reconstruire l'environnement, et l'état distant chiffré et verrouillé garantit la cohérence des exécutions.

Le deuxième objectif, la mise en place d'une chaîne de livraison continue intégrant des portes de qualité et de sécurité, est également satisfait. Le pipeline exécute la suite de tests, valide la syntaxe de l'infrastructure, soumet les dépendances, les plans d'infrastructure et les images produites à des analyses de sécurité bloquantes assorties d'une nomenclature logicielle par artefact, construit des images immuables et déclenche le déploiement, avec des garde-fous protégeant les ressources critiques. Le troisième objectif, l'orchestration d'environnements conteneurisés, se concrétise par un cluster Docker Swarm assurant le placement par rôle, la mise à l'échelle, les mises à jour progressives et le repli automatique. Le quatrième objectif, l'observabilité, est couvert par une stack collectant métriques et journaux, exposant des tableaux de bord et déclenchant des alertes. Le cinquième objectif, transversal, se traduit par une défense en profondeur prolongée jusqu'à la chaîne d'approvisionnement logicielle, une conformité réglementaire documentée, un dispositif de sauvegarde et de restauration automatisé et une supervision de la consommation énergétique estimée. L'ensemble des objectifs est ainsi atteint.

18.1.2 Analyse des performances

Sur le plan de la performance de livraison, l'automatisation supprime les délais et les erreurs des opérations manuelles. La construction des images est mise en cache et évitée lorsque l'artefact existe déjà, ce qui réduit la durée et l'empreinte des exécutions. Les mises à jour progressives, assorties d'un repli automatique,

permettent de déployer sans interruption perceptible et de contenir l'impact d'une évolution. Du point de vue de la performance applicative, le recours à un mutualiseur de connexions devant la base, l'indexation partielle des requêtes du fil public, la diffusion des médias par le réseau de distribution avec un cache immuable et la compression contribuent à des temps de réponse maîtrisés.

La performance d'exploitation bénéficie de l'observabilité, puisque les indicateurs de latence, de taux d'erreurs et de consommation de ressources sont disponibles en continu et que les règles d'alerte détectent les dérives avant qu'elles n'affectent le service. Le dimensionnement au plus juste des services, par la définition de réservations et de limites, améliore la densité et la prévisibilité tout en servant l'objectif de sobriété. Ces résultats, établis en conditions de validation, ouvrent la voie à une montée en charge réelle et prolongée qui constituera l'étape naturelle de consolidation.

18.2 Retour d'expérience et leçons apprises

18.2.1 Identification des succès et des défis

Le principal succès du projet réside dans la cohérence de la chaîne obtenue. De la description de l'infrastructure jusqu'à l'observation de la production, chaque maillon est automatisé, versionné et reproductible. L'intégration précoce de la sécurité et la mise en place d'une supervision énergétique, souvent absente des projets comparables, comptent parmi les apports les plus distinctifs. Le choix d'une orchestration adaptée à la taille du service s'est révélé particulièrement judicieux, en conservant les fonctions essentielles avec un excellent rapport entre les capacités et la simplicité d'exploitation.

Plusieurs points de vigilance sont d'ores et déjà maîtrisés et balisent une trajectoire d'évolution ambitieuse. Le nœud assurant le rôle de proxy regroupe le reverse proxy, le cache et la supervision au sein d'une architecture consolidée dont la mise en haute disponibilité constitue un palier d'évolution naturel, préparé par la persistance des données sur disque et une rétention de quinze jours. L'estimation de la consommation énergétique, adaptée aux machines virtuelles du cloud, s'appuie sur un modèle dérivé de l'utilisation processeur et fournit un indicateur de pilotage fiable et exploitable au quotidien, qu'une mesure matérielle sur un environnement dédié viendra affiner. La chaîne d'alerting et les analyses de sécurité automatisées sont

opérationnelles, l'enrichissement des canaux de notification, la mesure publiée de la couverture de tests et l'automatisation des contrôles d'accessibilité s'inscrivant dans une feuille de route d'amélioration continue déjà tracée et chiffrée.

18.2.2 Capitalisation des connaissances

La capitalisation des connaissances a été traitée comme un livrable à part entière. La documentation technique, structurée et versionnée avec le code, décrit l'architecture, les procédures d'automatisation et les décisions d'exploitation. Les décisions d'architecture significatives sont consignées sous forme de registres explicitant le contexte, la décision, les alternatives et les conséquences, ce qui préserve la mémoire des arbitrages et facilite l'intégration de nouveaux intervenants. Cette documentation, pensée pour différents profils de lecteurs, constitue un actif réutilisable au-delà du présent projet.

Cette documentation est centralisée dans le dépôt Git lui-même, versionnée au même titre que le code, et rassemble l'architecture, les procédures d'automatisation, les runbooks d'exploitation et les décisions d'architecture. Elle est organisée par profil de lecteur, l'exploitant y trouvant les procédures d'intervention et de reprise, le développeur les conventions de contribution et le contrat d'interface, et le décideur la synthèse des choix et de leurs conséquences. Sa forme sert cette lisibilité autant que son accessibilité, puisqu'elle privilégie le texte structuré au format Markdown plutôt que les captures d'écran, double chaque schéma d'une description textuelle et conserve un balisage hiérarchique exploitable par un lecteur d'écran. Ce format, consultable indifféremment dans la forge, dans un éditeur ou en ligne de commande, adapte la documentation des processus d'automatisation aux différents utilisateurs, y compris en situation de handicap, et constitue le point d'entrée unique de la connaissance projet.

18.3 Perspectives et recommandations

Les perspectives d'évolution s'organisent selon une logique d'amélioration continue, priorisée par le rapport entre la valeur et l'effort et chiffrée à vingt-neuf jours-homme au total. À court terme, l'enrichissement des canaux de notification d'alerte et la publication de la mesure de couverture des tests, assortie d'un seuil bloquant, renforceront encore la robustesse d'un socle déjà solide. À moyen terme, la mise en haute disponibilité de la supervision, la formalisation d'un exercice périodique de

reprise conduit en conditions réelles, l'intégration d'un contrôle d'accessibilité automatisé aux portes de qualité et le passage à une mesure énergétique matérielle sur un environnement dédié consolideront la résilience, l'inclusion et la précision.

À plus long terme, l'adoption de stratégies de déploiement plus sophistiquées, telles que le déploiement canari, et l'exploitation de l'intelligence artificielle pour la prédiction d'incidents et l'optimisation des ressources ouvriront une nouvelle étape de maturité. Ces recommandations prolongent la logique de l'architecture retenue et confirment que la valeur d'une infrastructure DevOps réside autant dans sa capacité à évoluer que dans la qualité de son état actuel.

19. Conclusion et remerciements

19.1 Synthèse des principaux résultats et impacts

Ce mémoire a traité de la conception et du déploiement d'une infrastructure DevOps entièrement automatisée pour dropicture, plateforme SaaS de partage de médias fondée sur la confidentialité par défaut. La problématique posée interrogeait la possibilité de concilier automatisation, sécurité, éco-responsabilité, qualité, observabilité et maîtrise des coûts au sein d'un même système. Le cheminement suivi apporte une réponse claire et argumentée, car cette conciliation est atteinte dès lors que l'automatisation est pensée de bout en bout, que la sécurité et la sobriété sont intégrées dès la conception et que l'observabilité rend l'ensemble pilotable.

Les hypothèses de travail se trouvent confirmées. La description intégrale de l'infrastructure en code, couplée à une approche GitOps, améliore simultanément la fiabilité et la vitesse des déploiements en éliminant les opérations manuelles et les divergences d'environnement. Une orchestration adaptée à la taille du service garantit portabilité, scalabilité et résilience avec un excellent rapport entre les capacités et la simplicité d'exploitation. Enfin, une stratégie d'observabilité intégrée, complétée d'une estimation de la consommation énergétique, rend l'éco-responsabilité mesurable et compatible avec la maîtrise des coûts. L'impact de cette démarche est à la fois technique, par la fiabilisation et l'accélération de la livraison, économique, par la réduction du gaspillage, et organisationnel, par la capitalisation des connaissances.

Le dossier valorise les résultats obtenus et prolonge chaque réalisation par des perspectives d'évolution ambitieuses et réalistes, dans une logique d'amélioration continue. L'infrastructure décrite constitue une base solide et évolutive, dont la valeur tient autant à sa capacité d'adaptation qu'à la qualité de son état actuel. Elle illustre la responsabilité centrale d'un expert en systèmes d'information, qui consiste non seulement à produire un logiciel, mais à construire et à maîtriser la chaîne industrielle, sûre et sobre, qui le porte jusqu'à ses utilisateurs.

19.2 Remerciements

Au terme de ce dossier, la reconnaissance s'adresse à l'équipe pédagogique de l'École LaPlateforme et à Joris VERGULDEZOONE, référent de formation, pour leur

accompagnement, à l'entreprise d'accueil FDJ United et à Philippe AMERIO, référent en entreprise, pour l'environnement d'apprentissage offert, ainsi qu'à l'ensemble des personnes ayant contribué à l'aboutissement de ce projet.

20. Bibliographie / Webographie

Les schémas figurant en annexe sont des productions originales de l'auteur et ne nécessitent donc pas de source externe. Les références ci-dessous documentent les technologies, les méthodes et les cadres normatifs mobilisés.

Référentiel et cadre de certification

- France Compétences, *Référentiel RNCP40573, Expert en Informatique et Systèmes d'Information*, 3W Academy, 2024.
- Guide du candidat et Modèle de dossier du candidat, Titre RNCP « Expert en Informatique et Systèmes d'Information ».

Ouvrages et publications de référence

- N. Forsgren, J. Humble, G. Kim, *Accelerate, The Science of Lean Software and DevOps*, IT Revolution Press, 2018 (métriques DORA).
- G. Kim, J. Humble, P. Debois, J. Willis, *The DevOps Handbook*, IT Revolution Press, 2016.
- OWASP Foundation, *OWASP Top 10 et Application Security Verification Standard (ASVS)*.

Webographie technique

- Terraform, <https://developer.hashicorp.com/terraform/docs>
- Ansible, <https://docs.ansible.com>
- Docker et Docker Swarm, <https://docs.docker.com/engine/swarm/>
- Traefik Proxy, <https://doc.traefik.io/traefik/>
- NestJS, <https://docs.nestjs.com>
- Next.js, <https://nextjs.org/docs>
- TypeORM, <https://typeorm.io>
- PostgreSQL, <https://www.postgresql.org/docs/>
- Prometheus et Alertmanager, <https://prometheus.io/docs/>
- Grafana, Loki et Alloy, <https://grafana.com/docs/>
- Hetzner Cloud, <https://docs.hetzner.com>
- Cloudflare, <https://developers.cloudflare.com>
- Amazon Web Services, <https://docs.aws.amazon.com>

- GitHub Actions, <https://docs.github.com/actions>

Cadre normatif et réglementaire

- ISO/IEC 27001 version 2022, *Systèmes de management de la sécurité de l'information*.
- Règlement (UE) 2016/679 (RGPD) et CNIL, <https://www.cnil.fr>
- Référentiel Général d'Amélioration de l'Accessibilité (RGAA) et WCAG 2.1 (W3C).

21. Annexes

Les annexes regroupent les schémas et documents détaillés appelés dans le corps du mémoire par la mention « cf. Annexe X ». Conformément aux consignes, tous les schémas sont des productions originales de l'auteur et ne requièrent donc aucune source externe. Chaque annexe renvoie à son fichier source, versionné avec le code du projet. Les schémas essentiels à la validation des blocs sont reproduits en images dans la section « Annexes illustrées » qui suit.

Justification des choix de représentation

Les représentations retenues sont justifiées au regard du besoin. Le diagramme de Gantt (Annexe N) a été choisi pour visualiser le séquençage temporel et les jalons, tandis que le PERT (Annexe M) met en évidence les dépendances et le chemin critique, information que le Gantt seul ne restitue pas. La matrice probabilité et impact (Annexe O) a été préférée à une simple liste afin de hiérarchiser visuellement les risques. Les schémas d'architecture privilégient la lisibilité, car ils sont séparés par niveau d'abstraction plutôt que réunis en une vue unique surchargée.

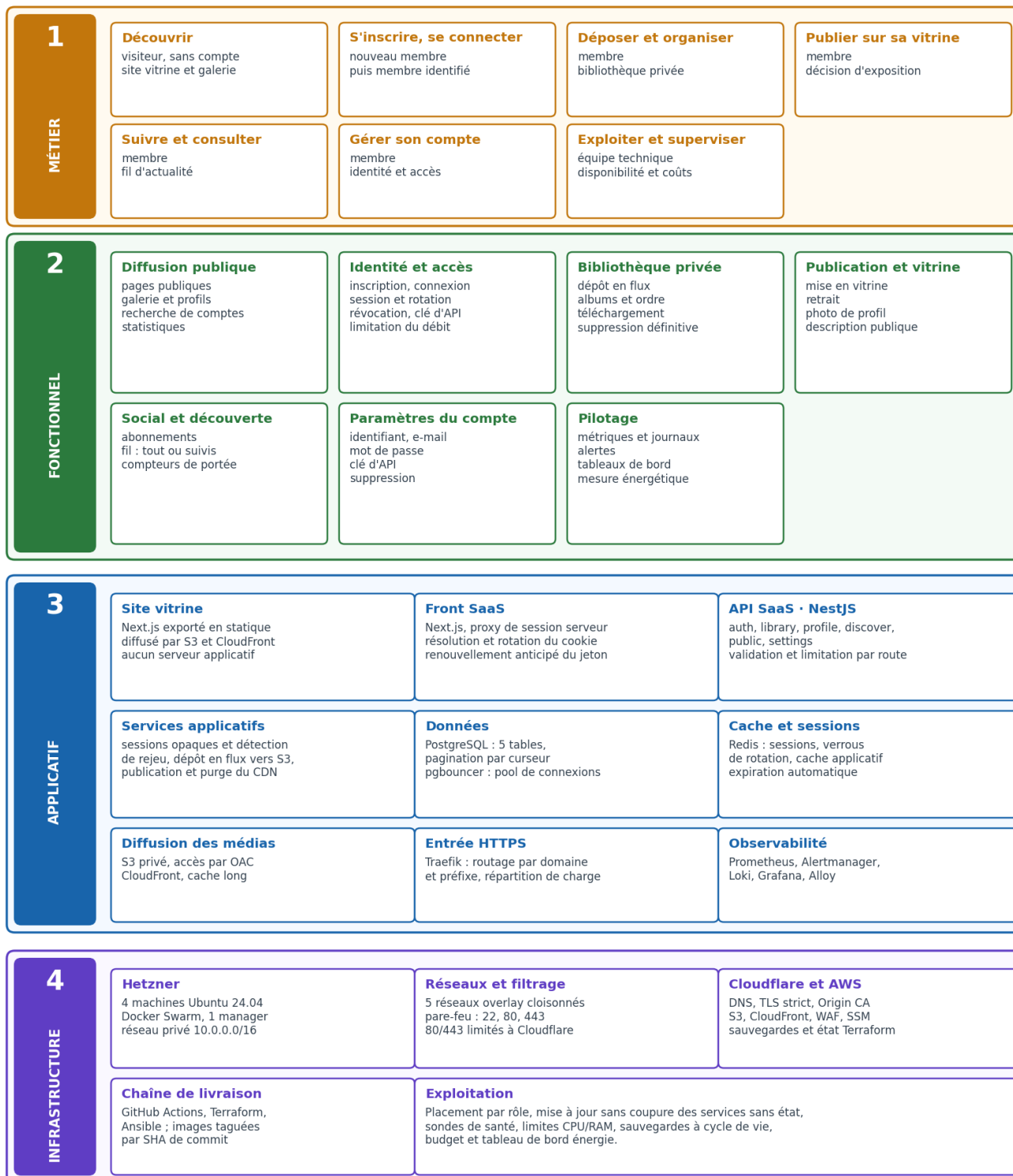
Annexes illustrées

Schémas originaux de l'auteur, appelés dans le corps du mémoire par la mention « cf. Annexe X ». Une illustration par page.

Annexe A, Cartographie du SI, 4 niveaux (métier, fonctionnel, applicatif, infrastructure) (BC1)

Cartographie du système d'information

Quatre niveaux : ce que fait l'entreprise, ce que doit faire le SI, comment il est réalisé, sur quoi il s'exécute



Lecture verticale de la carte

Le processus « Publier sur sa vitrine » s'appuie sur le bloc fonctionnel Publication, réalisé par l'API SaaS (route de publication), qui écrit dans PostgreSQL et sert les fichiers via CloudFront, le tout exécuté sur le nœud backend du cluster Swarm.

À QUOI SERT CE DÉCOUPAGE

- Le niveau métier ne dépend d'aucune technologie : il resterait valable après un changement d'implémentation.
- Le niveau fonctionnel sert de contrat : chaque bloc est testable indépendamment de son support technique.
- Les niveaux 3 et 4 sont les seuls à changer souvent.

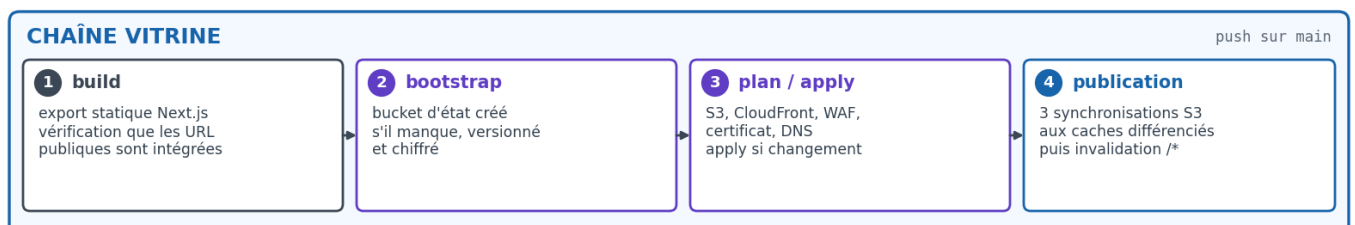
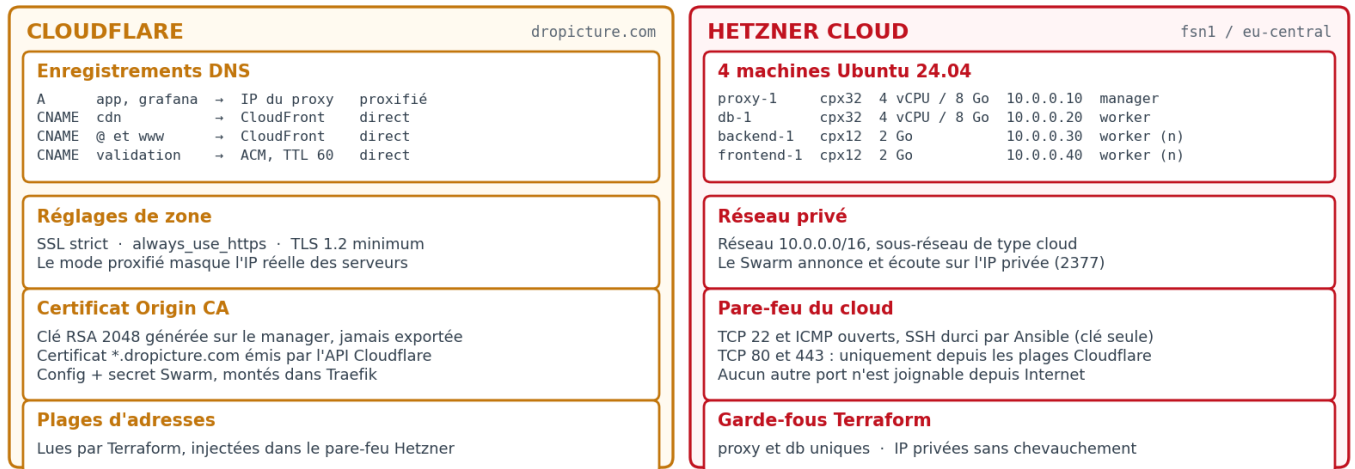
PRINCIPE STRUCTURANT DU PRODUIT

- Le privé est l'état par défaut : un média déposé n'est visible de personne tant qu'il n'est pas publié.
- Ce choix métier se retrouve à chaque niveau : une fonction Publication distincte, une date de publication en base, un CDN qui ne sert que les fichiers exposés.

Annexe B, Topologie de l'infrastructure et du déploiement (BC4C)

Topologie de l'infrastructure et du déploiement

3 fournisseurs · 4 machines · Terraform décrit, Ansible configure, GitHub Actions livre



ÉTAT TERRAFORM ET INVENTAIRE DYNAMIQUE

- L'état Terraform est la source de vérité : noms des machines, IP publiques et privées, plages Cloudflare, identifiants CDN.
- Aucun fichier d'inventaire n'est maintenu à la main. Ansible télécharge l'état depuis S3, en déduit les hôtes, leur rôle et le nombre de réplicas, puis supprime la copie locale.
- Conséquence : ajouter un nœud backend se résume à une ligne dans terraform.tfvars. Le label Swarm, le nombre de réplicas et les cibles Prometheus suivent automatiquement.
- playbook.yml prépare les nœuds : durcissement SSH, Docker, initialisation du Swarm sur le réseau privé, réconciliation des nœuds retirés, labels de rôle, certificat Origin CA.
- deploy.yml ne cible que le manager : il refuse de déployer sur un Swarm non provisionné, rend les configurations, lance le déploiement, attend Postgres puis exécute les migrations.
- Les deux playbooks sont rejouables sans effet de bord.

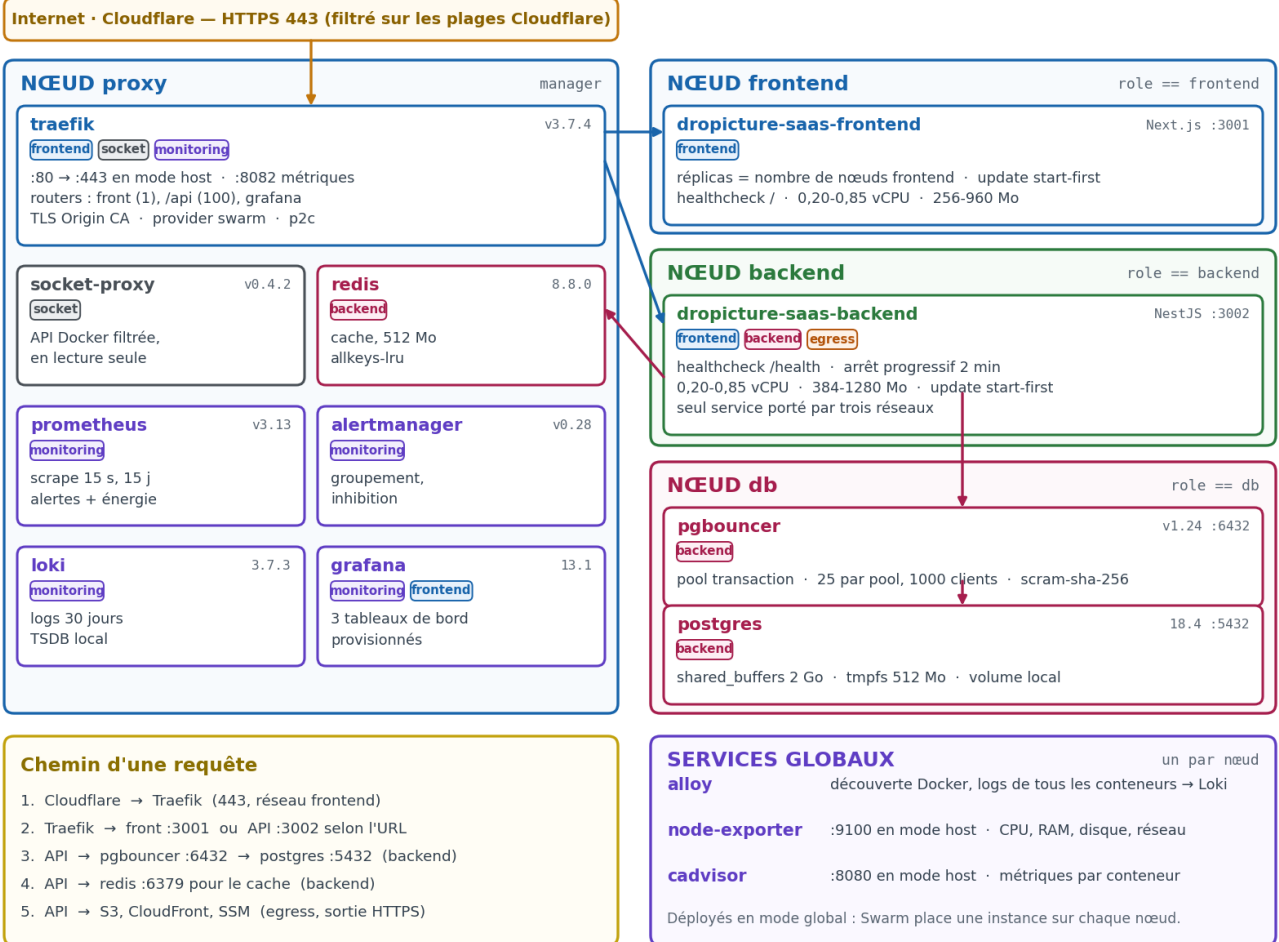
SÉCURITÉ, DIMENSIONNEMENT ET COÛTS

- Trois ports ouverts seulement, et 80/443 sont restreints aux plages Cloudflare : l'origine n'est pas joignable en direct.
- SSH par clé uniquement, mot de passe et clavier désactivés.
- Aucun secret dans le dépôt : GitHub Secrets, puis variables d'environnement, puis secret Swarm ou fichier temporaire en 0600 supprimé en fin d'exécution.
- La clé privée Origin CA ne quitte jamais le manager.
- Buckets S3 privés, servis par CloudFront via OAC, refusant explicitement le transport non chiffré.
- Quatre machines : deux pour les rôles avec état, deux pour les rôles applicatifs, qui s'ajoutent horizontalement.
- CloudFront en PriceClass_100, cycle de vie S3 vers les classes froides, budget AWS de 20 \$/mois avec alertes.
- Réservations et limites CPU/RAM sur les 15 services, et un tableau de bord énergie : la consommation est mesurée.

Annexe C, Conteneurisation, services et segmentation réseau (Docker Swarm) (BC4C)

Conteneurisation, services et segmentation réseau (Docker Swarm)

1 stack · 15 services · 4 nœuds · 5 réseaux overlay · placement par label dropicture.role



SEGMENTATION RÉSEAU

5 overlays · MTU 1400 · pool 10.100.0.0/16 en /24

frontend	exposé	Traefik vers le frontend, l'API et Grafana. Seul réseau atteint depuis l'extérieur.
backend	internal	API ↔ pgbouncer ↔ postgres, et le cache Redis. Aucune route vers Internet.
socket	internal	Traefik ↔ socket-proxy uniquement, pour interroger l'API Docker sans la monter.
egress	exposé	Sorties HTTPS du backend vers AWS : dépôt des médias, CDN, paramètres SSM.
monitoring	interne	Prometheus, Alertmanager, Loki, Grafana et les sondes présentes sur les 4 nœuds.

PLACEMENT ET CYCLE DE VIE

- Placement par label : dropicture.role == proxy | frontend | backend | db. socket-proxy est en plus contraint sur node.role == manager.
- Les labels sont posés par Ansible depuis les sorties Terraform, par identifiant de nœud, de façon idempotente.
- Services sans état (frontend, backend, socket-proxy) : update order start-first, parallelism 1, monitor 30 s, donc mise à jour sans coupure de service.
- Services avec état (traefik, postgres, pgbouncer, redis, observabilité) : order stop-first, pour ne jamais avoir deux instances sur le même volume.
- failure_action: rollback et rollback_config sur les services applicatifs ; restart_policy condition any.
- Double contrôle de santé : healthcheck du conteneur et healthcheck du load-balancer Traefik (/ , /health, /api/health), toutes les 10 secondes.
- Arrêt progressif : 2 min pour le backend, 1 min pour Postgres, afin de laisser finir les requêtes en cours.
- reservations et limits CPU/RAM sur les 15 services.

CONFIGS, SECRETS ET PERSISTANCE

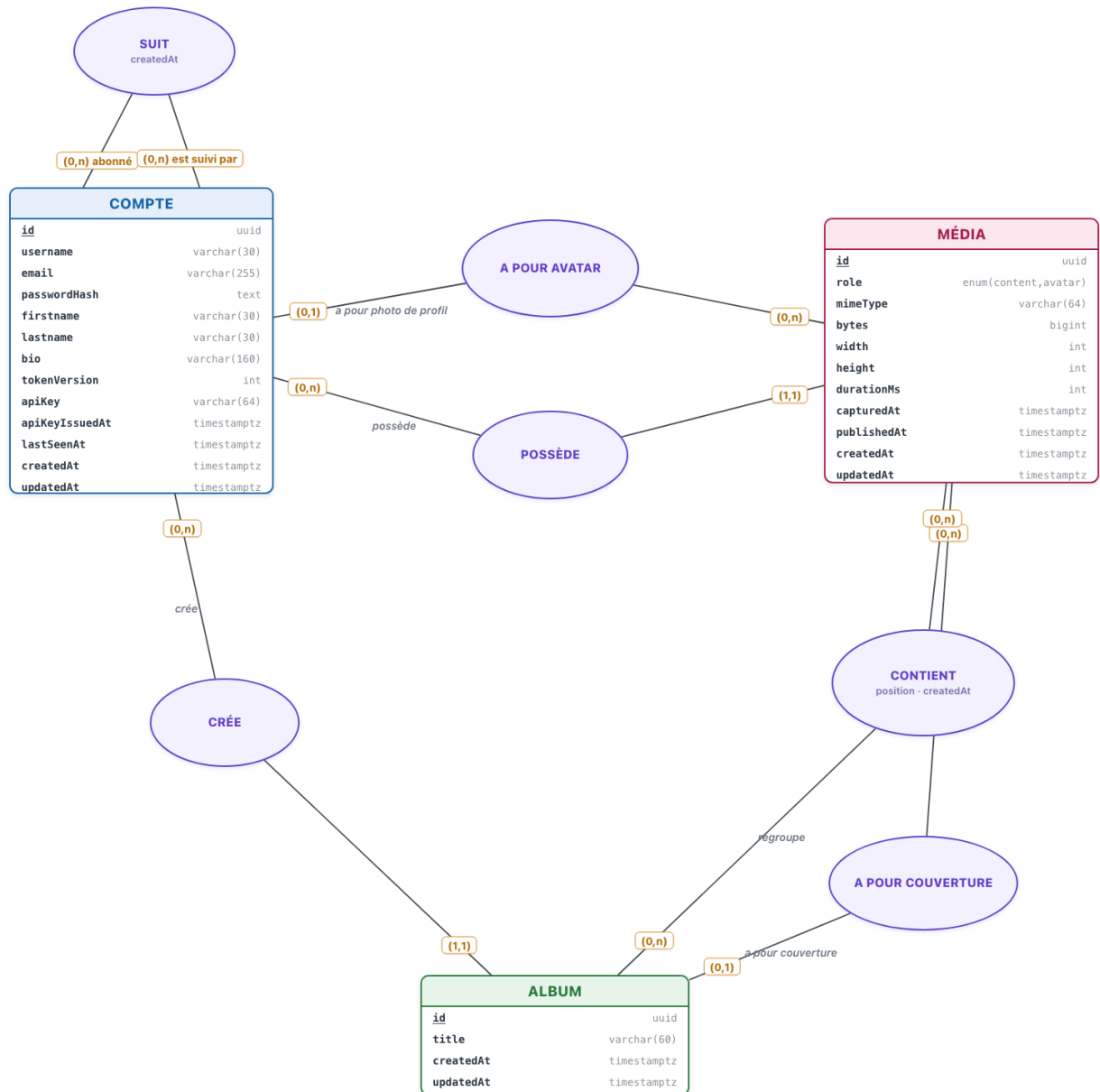
- Configs externes au stack : dropicture_traefik_dynamic_v1 et dropicture_origin_cert (certificat Origin CA).
- Secret externe : dropicture_origin_key, la clé privée, montée dans Traefik sous /run/secrets/origin.key. Elle n'est ni dans l'image, ni dans une variable, ni dans le dépôt : elle est générée sur le manager et y reste.
- Les configs Swarm étant immuables, les fichiers d'observabilité sont nommés avec un hash de contenu (dropicture_prometheus_<hash>), ce qui force la recreation du service quand le contenu change.
- Persistance par bind mount sur le disque local du nœud : /opt/dropicture/data/{postgres, redis, prometheus, alertmanager, loki, grafana, alloy}, avec les UID/GID attendus par chaque image (999, 65534, 10001, 472).
- tmpfs /dev/shm de 512 Mo pour Postgres.
- Images épinglées à une version précise : traefik 3.7.4, postgres 18.4, redis 8.8.0, loki 3.7.3, grafana 13.1.0. Aucun tag latest en production ; les images applicatives sont adressées par SHA de commit depuis ghcr.io.

Annexe D, Modèle Conceptuel de Données (MCD) (BC3)

Modèle conceptuel de données (MCD)

Merise · 3 entités · 6 associations · identifiant souligné · (min, max) : cardinalités lues depuis l'entité

3	6	31	uuid
ENTITÉS	ASSOCIATIONS	PROPRIÉTÉS	CLÉS PRIMAIRES



FORMALISME MERISE

Entité — rectangle, nom en majuscules, identifiant souligné. Association — ellipse, verbe + ses propriétés propres. (x,n) Cardinalités minimales / maximales lues depuis l'entité.

RÈGLES DE GESTION

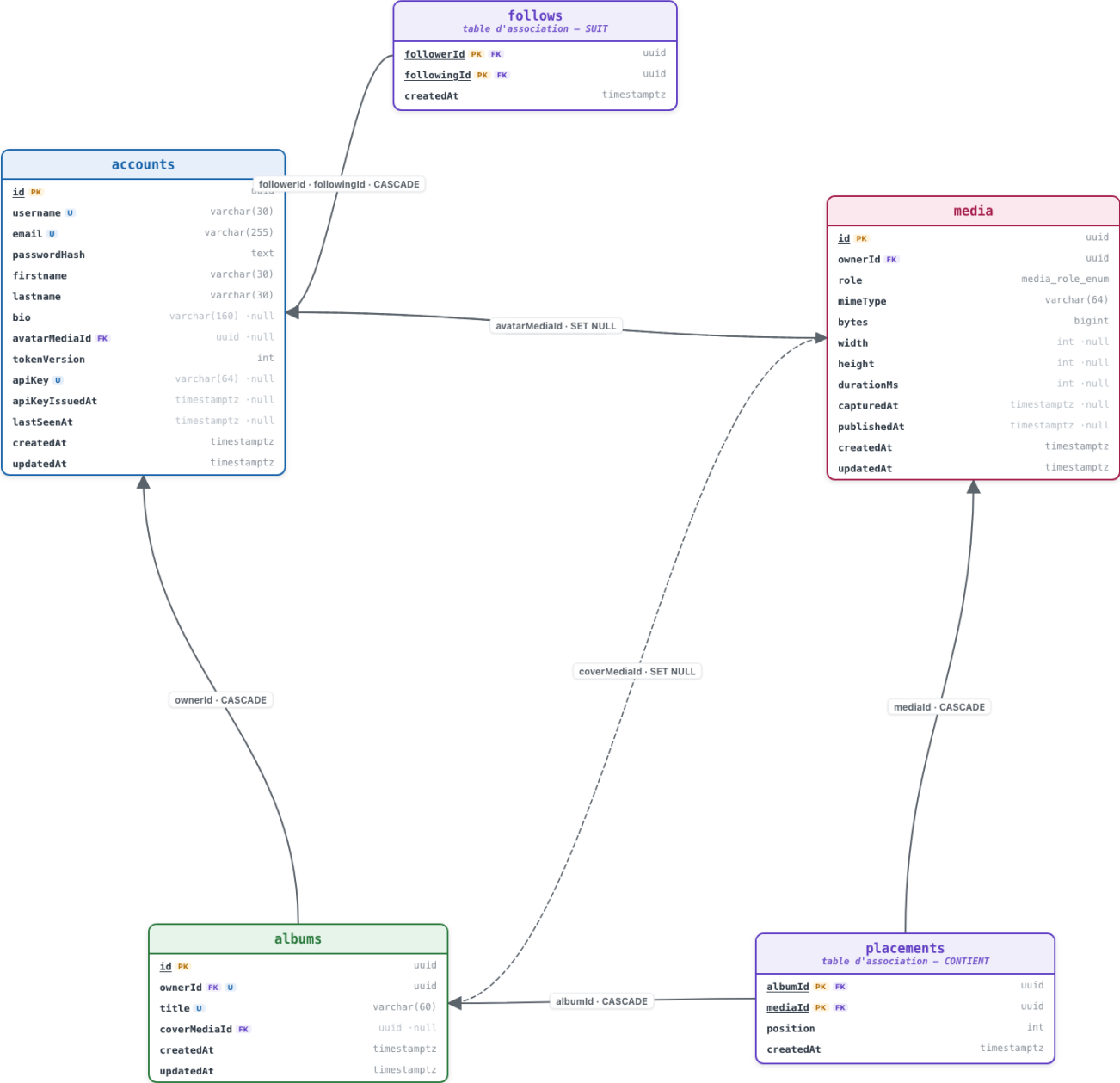
- Un compte **possède** 0..n médias ; un média appartient à exactement 1 compte.
- Un compte peut choisir **0 ou 1** média comme photo de profil (avatar).
- Un compte **crée** 0..n albums ; un album appartient à exactement 1 compte.
- Un album **regroupe** 0..n médias ; un média figure dans 0..n albums (position ordonnée).
- Un album peut avoir **0 ou 1** média de couverture.
- Un compte **suit** 0..n comptes et est suivi par 0..n comptes ; il ne peut pas se suivre lui-même.
- Unicité : username, email, apiKey, et (propriétaire + title) d'album.

Annexe E, Modèle Logique de Données (MLD) (BC3)

Modèle logique de données (MLD)

Traduction relationnelle du MCD · PostgreSQL · clés primaires soulignées, clés étrangères (→ pointe vers la PK référencée) et contraintes d'intégrité

5	8	39	uuid
TABLES	CLÉS ÉTRANGÈRES	COLONNES	CLÉS · TIMESTAMPTZ



NOTATION

PK clé primaire (nom souligné) FK clé étrangère → PK référencée U contrainte d'unicité · null colonne nullable (sinon NOT NULL) □ table d'association (relation n:n)

INTÉGRITÉ RÉFÉRENTIELLE

- **ON DELETE CASCADE** — supprimer un compte efface ses médias, albums, abonnements ; vider un album ou un média retire ses placements.
- **ON DELETE SET NULL** — supprimer un média retire seulement l'avatar (accounts.avatarMediaId) ou la couverture (albums.coverMediaId), sans casser la ligne.
- **CHECK** followerId <> followingId — un compte ne peut pas se suivre lui-même.

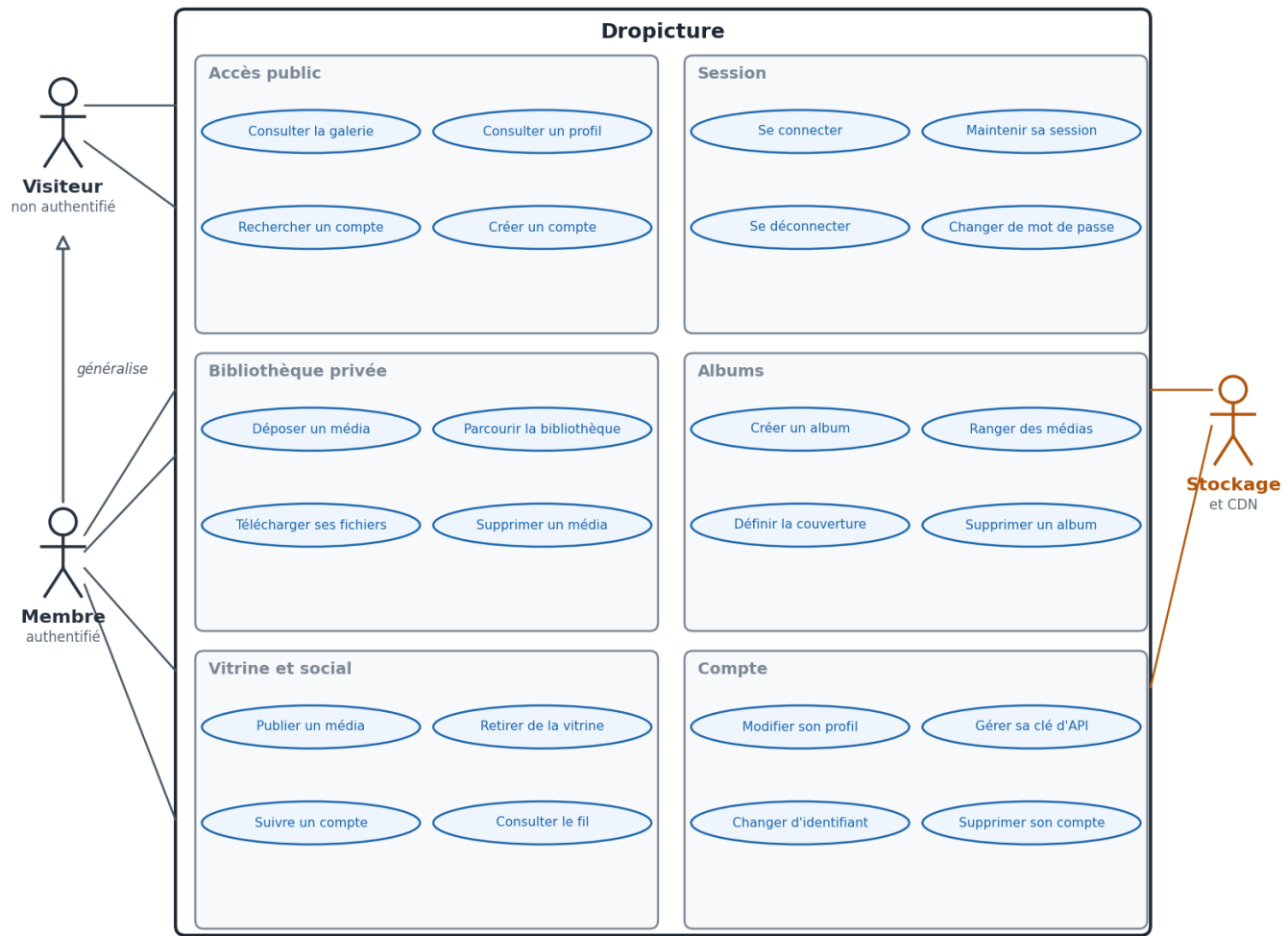
INDEX, UNICITÉS & DÉFAUTS

- **Unicités** : username, email, apiKey (partiel : WHERE apiKey IS NOT NULL), (ownerId, title) d'album.
- **Index** : media(ownerId, capturedAt) ; media(publishedAt) partiel role=content ; follows(followingId) ; placements(albumId, position) ; placements(mediaId).
- **Défauts** : id = uuid_generate_v4() ; createdAt / updatedAt = now() ; PostgreSQL 18.

Annexe G, Diagramme de cas d'utilisation (BC3)

Diagramme de cas d'utilisation

2 acteurs principaux, 1 acteur secondaire, 22 cas d'utilisation regroupés par domaine



RELATIONS ENTRE CAS

- « Déposer un média » inclut « Envoyer le fichier au stockage », et étend « Ranger des médias » lorsqu'un album est choisi au moment du dépôt.
- « Publier un média » inclut « Rendre le fichier accessible au CDN ».
- « Supprimer un média » inclut « Purger le cache ».
- « Changer de mot de passe » inclut « Révoquer toutes les sessions » : les autres appareils sont déconnectés.
- Tous les cas du membre incluent « Contrôler la session ».
- Le membre hérite du visiteur : il conserve l'accès à l'ensemble des cas publics.

RÈGLES COMMUNES

- Chaque entrée est validée avant traitement : format, longueur, type, caractères autorisés.
- Chaque route est limitée en débit, avec un plafond adapté au risque : 5 inscriptions par heure, 10 connexions par minute, 240 lectures de fil.
- Un membre n'agit que sur ses propres médias : la propriété est vérifiée en base, pas dans l'interface.
- Un média déposé est privé. Le publier est un cas distinct, déclenché explicitement par le membre.
- Les identifiants réservés et les noms déjà pris sont refusés dès la saisie, puis revérifiés côté serveur.

CORRESPONDANCE CAS D'UTILISATION → SERVICE RENDU PAR L'API

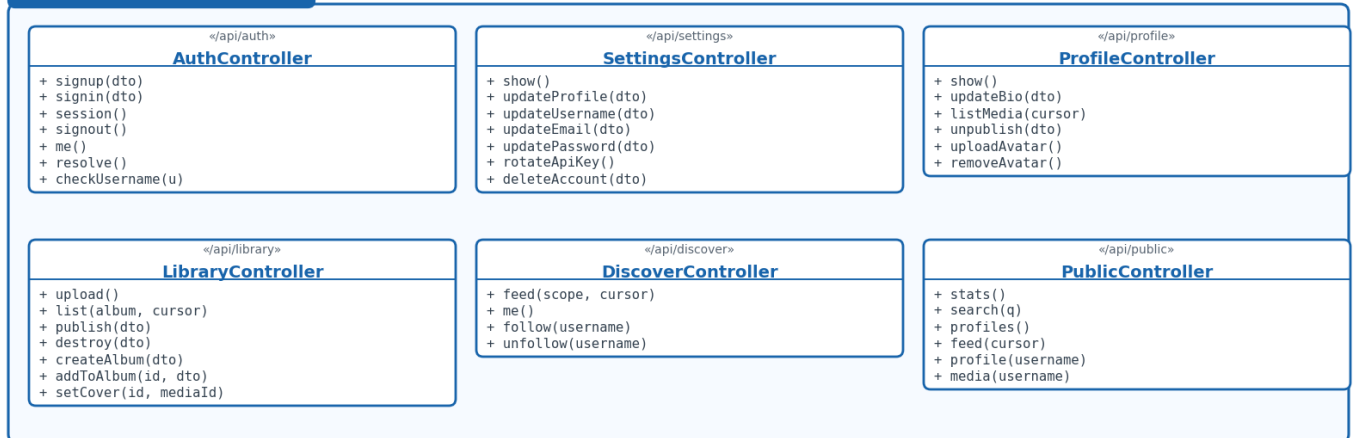
Cas d'utilisation	Service rendu	Acteur
Consulter la galerie	lecture publique, pagination par curseur	visiteur
Consulter un profil	profil public, compteurs, premiers médias	visiteur
Rechercher un compte	recherche par identifiant, nom ou prénom	visiteur
Créer un compte	contrôle du format, unicité, mot de passe haché	visiteur
Se connecter	vérification puis ouverture de session	visiteur
Maintenir sa session	rotation du jeton, détection de rejeu	membre
Déposer un média	envoi en flux vers le stockage, taille contrôlée	membre
Parcourir la bibliothèque	liste privée, regroupement par mois	membre
Publier un média	date de publication posée, fichier exposé	membre
Retirer de la vitrine	date effacée, le fichier redevient privé	membre
Supprimer un média	effacement du stockage puis de la base	membre
Ranger des médias	ajout à un album avec position ordonnée	membre
Suivre un compte	abonnement, sans doublon possible	membre
Consulter le fil	tout public, ou limité aux comptes suivis	membre
Gérer sa clé d'API	génération, affichage, révocation	membre
Supprimer son compte	médias effacés puis compte supprimé	membre

Annexe H, Diagramme de classes (backend) (BC3)

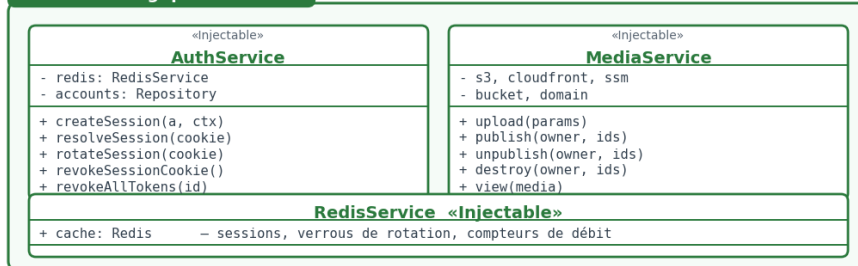
Diagramme de classes — backend

NestJS · 6 contrôleurs, 3 services, 2 stratégies d'authentification, 5 entités persistantes

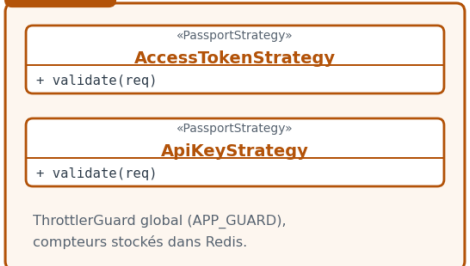
Contrôleurs · couche HTTP



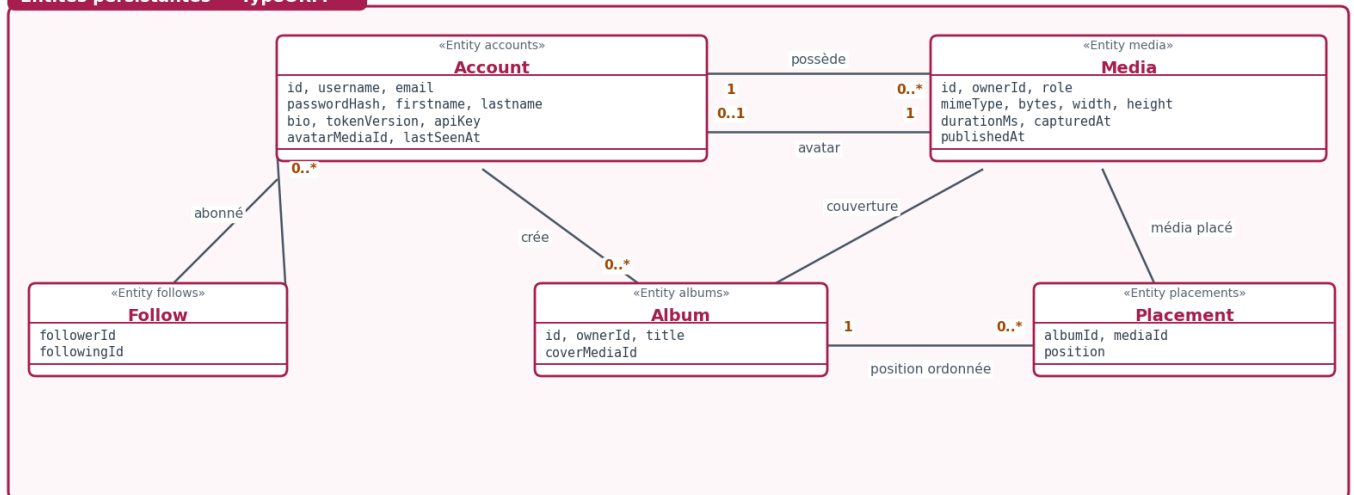
Services · logique métier



Sécurité



Entités persistantes · TypeORM



PRINCIPES DE CONCEPTION

- Injection de dépendances : chaque contrôleur reçoit ses services et ses dépôts par constructeur, ce qui permet de les remplacer par des doubles dans les tests.
- Aucune requête SQL n'est écrite dans un contrôleur.
- Les entités ne portent aucune logique métier : ce sont des structures de données annotées.
- Les entrées sont validées avant d'atteindre le contrôleur, et les champs inconnus sont rejetés.

DEUX POINTS À RETENIR

- AuthService concentre toute la gestion de session : création, résolution, rotation et révocation. Aucun contrôleur ne manipule le cookie ni Redis directement, donc la détection de rejet s'applique partout de la même façon.
- MediaService est le seul point de contact avec le stockage objet et le CDN. Publier, dépublier et supprimer sont des opérations de lot qui renvoient ce qui a réussi, à charge des contrôleurs d'expliquer les échecs au client.

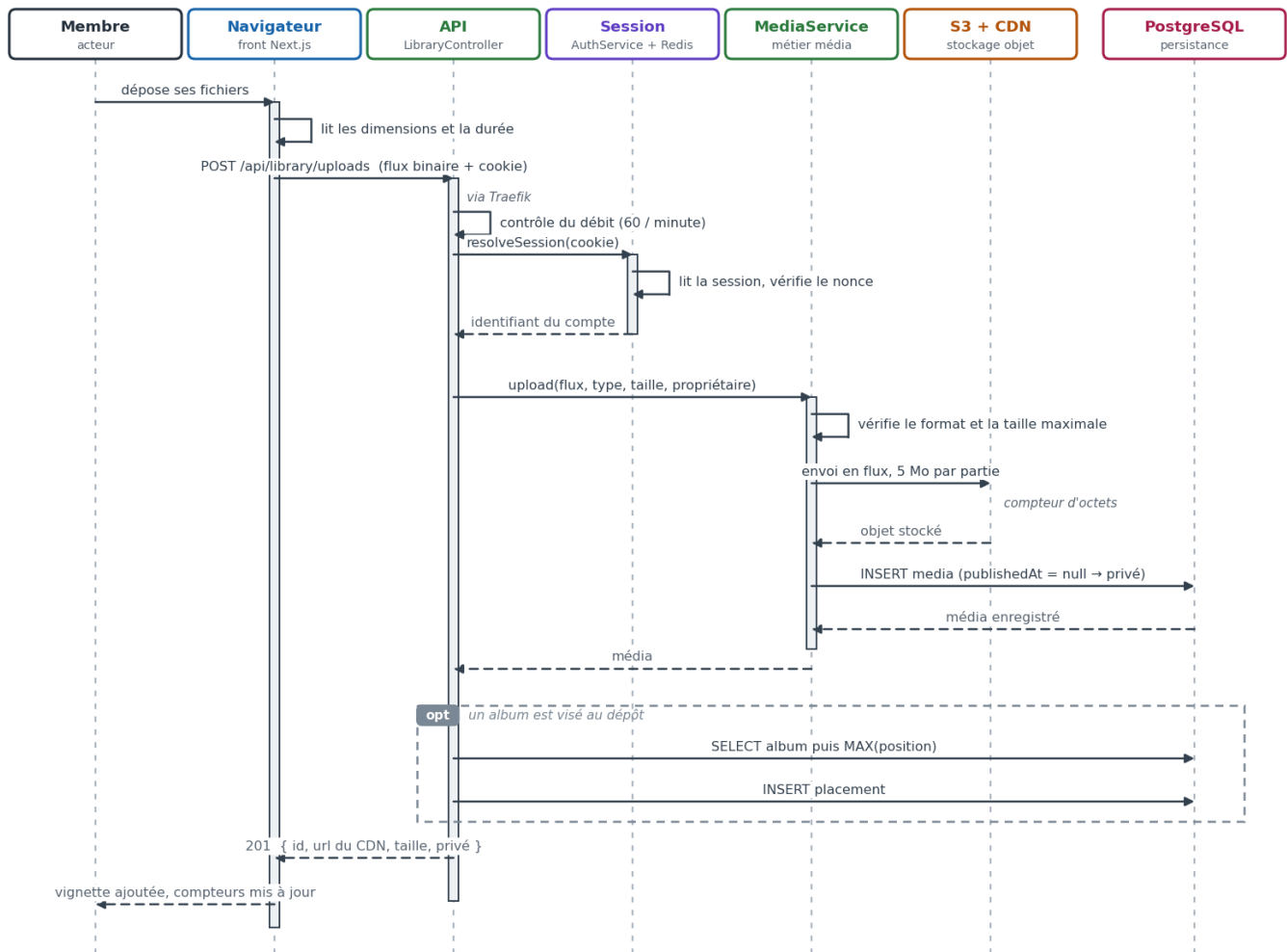
DÉPENDANCES ENTRE CLASSES

Classe	Dépend de
AuthController	AuthService, Repository<Account>
SettingsController	AuthService, MediaService, Repository<Account>, Repository<Media>
ProfileController	MediaService, Repository<Account>, Repository<Media>
LibraryController	MediaService, Repository<Media> Album Placement>
DiscoverController	MediaService, Repository<Account> Media Follow>
PublicController	MediaService, Repository<Account> Media Follow>
AuthService	RedisService, Repository<Account>
MediaService	Repository<Media>, S3Client, CloudFrontClient, SSMClient
AccessTokenStrategy	AuthService
ApiKeyStrategy	Repository<Account>

Annexe I.1, Séquence, téléversement d'un média (BC3)

Diagramme de séquence — téléversement d'un média

Du dépôt du fichier jusqu'à son enregistrement privé · le média reste invisible tant qu'il n'est pas publié



CHEMINS D'ERREUR

- 429 — débit dépassé, avant tout traitement.
- 401 — session absente, expirée, ou cookie rejoué hors de la fenêtre de grâce : toutes les sessions sont alors révoquées.
- 400 — format non accepté (huit types autorisés).
- 400 — fichier trop lourd. Deux contrôles : la taille annoncée avant transfert, puis un compteur d'octets pendant le flux ; l'envoi est alors interrompu et l'objet partiel supprimé.
- 400 — fichier vide : l'objet est supprimé, rien n'est écrit.

POINTS CLÉS

- Le fichier ne touche jamais le disque de l'API : il est relayé en flux vers le stockage objet.
- Aucune écriture en base tant que le fichier n'est pas stocké : pas de ligne orpheline.
- Le média est créé sans date de publication, donc privé.

DÉTAIL DES ÉTAPES

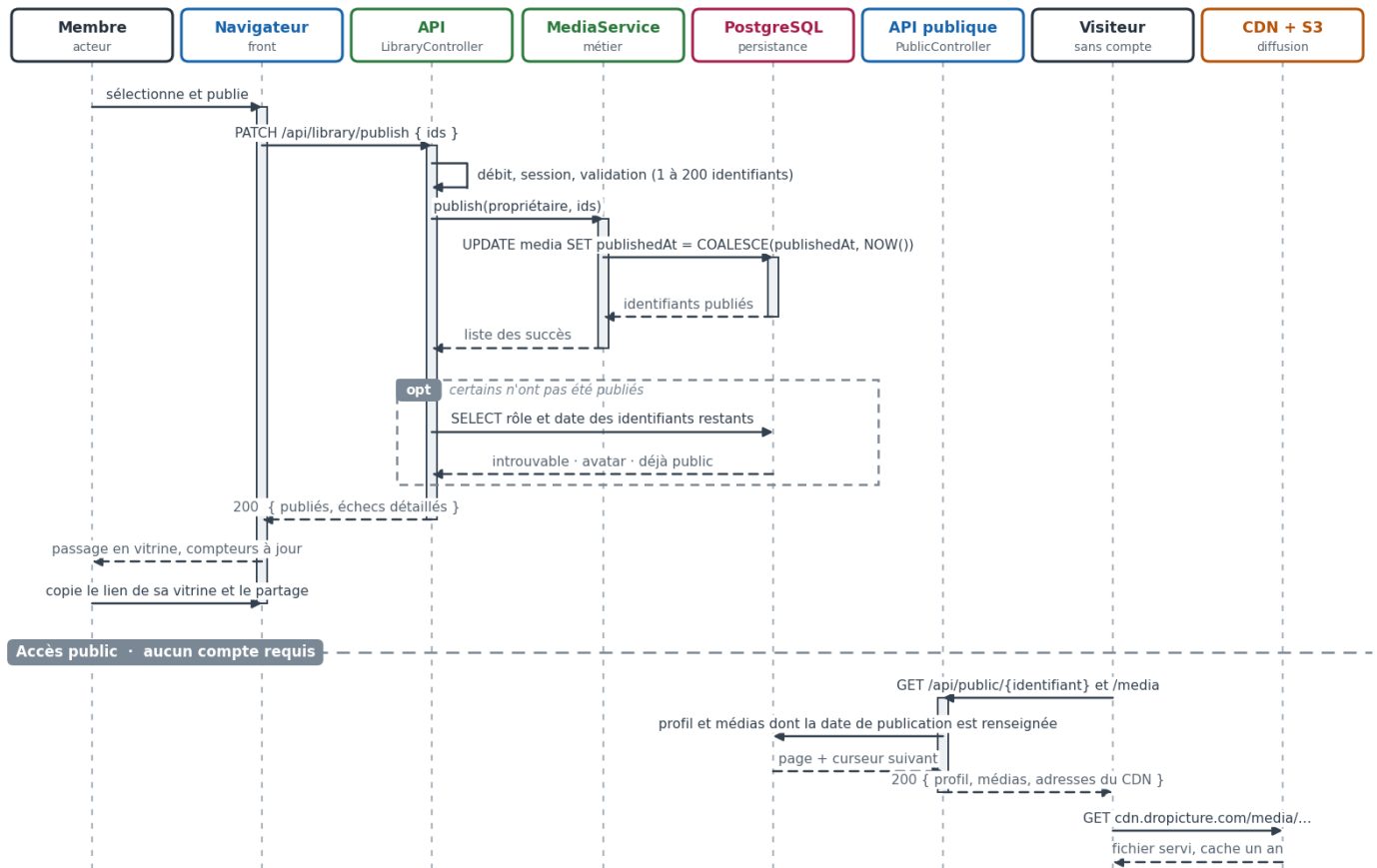
Étape	Acteur	Traitement
1	Navigateur	Lit les métadonnées du fichier côté client (dimensions, durée, date de prise de vue)
2	Navigateur	Envoie le corps brut avec le type MIME en en-tête ; la progression est suivie localement
3	API	Le garde de débit s'applique avant le contrôleur : 60 téléversements par minute
4	Session	Le cookie opaque est résolu en identifiant de compte ; le nonce est vérifié
5	MediaService	Refuse les types non listés ; refuse au-delà de 8 Mo (image) ou 100 Mo (vidéo)
6	MediaService	Génère un identifiant et une clé : media/{propriétaire}/{identifiant}.{extension}
7	S3	Envoi en plusieurs parties de 5 Mo, deux en parallèle, cache immuable d'un an
8	MediaService	Interrompt et supprime l'objet si le compteur dépasse la limite en cours de route
9	PostgreSQL	Insère la ligne média : propriétaire, type, taille réelle, dimensions, dates
10	API	Si un album est visé : vérifie qu'il appartient au membre, calcule la position
11	API	Insère le placement en ignorant les doublons (le média peut déjà y figurer)
12	Navigateur	Ajoute la vignette en tête de liste et rafraîchit les compteurs de stockage

Le téléversement s'arrête au stockage privé. La publication, la mise en album et la suppression sont des cas d'utilisation distincts, chacun avec sa propre séquence : c'est ce découpage qui permet de garantir qu'un fichier déposé n'est jamais exposé par accident.

Annexe I.2, Séquence, publication / partage (BC3)

Diagramme de séquence — publication et partage

Le membre expose un média déjà déposé, puis un visiteur y accède sans compte · publier ne déplace aucun fichier



CE QUE FAIT LA PUBLICATION

- Une date est posée sur la ligne du média. Aucun fichier n'est copié, déplacé ni ré-encodé : l'objet est dans le stockage depuis le téléversement, seule sa visibilité change.
- L'opération est idempotente : republier un média déjà public conserve sa date d'origine.
- Un avatar est refusé : il est public par nature.
- Le traitement est partiel par conception : les identifiants valides passent, les autres reviennent expliqués.

LIMITE ASSUMÉE DU RETRAIT

- Dépublier remet la date à vide : le média quitte le fil, la galerie publique et le profil.
- L'adresse directe du fichier reste servie par le cache du CDN jusqu'à expiration. Elle n'est devinable que par son identifiant aléatoire, mais quiconque l'a reçue peut encore l'ouvrir pendant ce délai.
- Seule la suppression efface l'objet du stockage et demande la purge du cache : c'est la seule action irréversible.

DÉTAIL DES ÉTAPES

Étape	Acteur	Traitement
1	Navigateur	Envoie la sélection en un seul appel : un lot plutôt qu'une requête par média
2	API	Débit, session, puis validation du lot : de 1 à 200 identifiants bien formés
3	MediaService	Une seule mise à jour, filtrée sur le propriétaire : personne ne publie chez autrui
4	PostgreSQL	Renvoie les identifiants réellement modifiés, ce qui distingue succès et échecs
5	API	Explique chaque échec : média introuvable, avatar interdit, ou déjà public
6	Navigateur	Retire les éléments de la bibliothèque privée et met à jour les compteurs
7	Visiteur	Ouvre la vitrine partagée : aucune authentification n'est demandée
8	API publique	Ne renvoie que les médias dont la date de publication est renseignée
9	CDN	Sert le fichier depuis le cache, avec un en-tête d'immuabilité d'un an

Le même mécanisme sert au retrait : la route de dépublication remet la date à vide et refuse les avatars, avec le même compte rendu détaillé. Publication et retrait sont donc symétriques, réversibles, et sans effet sur le fichier stocké.

Vue d'ensemble du cycle de vie d'un média :

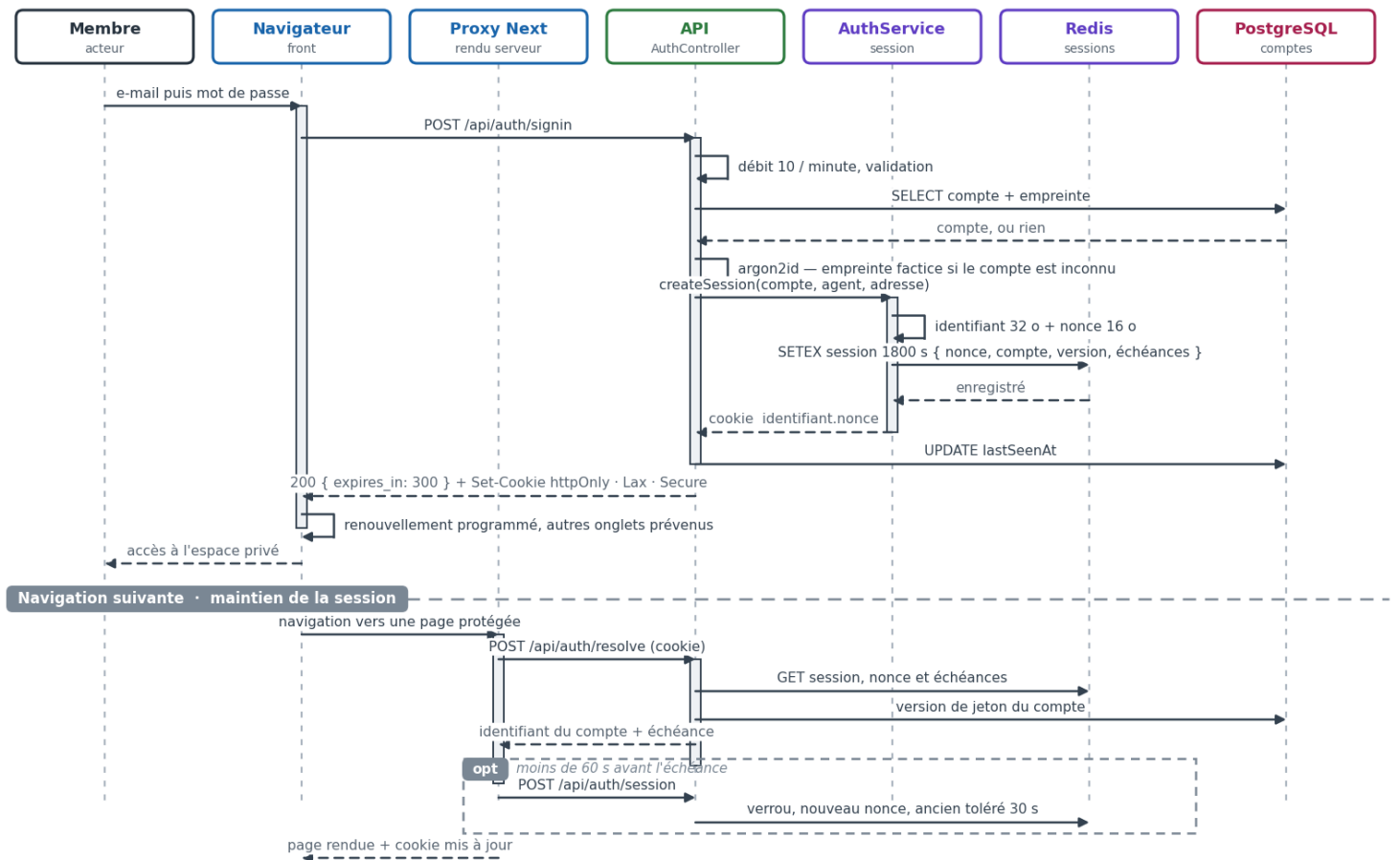


Seule la dernière étape est irréversible. Le passage de « publié » à « dépublié » peut se faire dans les deux sens, autant de fois que voulu.

Annexe I.3, Séquence, session sécurisée (connexion) (BC3)

Diagramme de séquence — session sécurisée

Connexion puis maintien par rotation · cookie opaque adossé à Redis, aucun jeton lisible côté client



CE QUI PROTÈGE LA SESSION

- Le cookie ne contient aucune donnée : c'est une référence opaque vers un enregistrement Redis. Rien n'est lisible ni falsifiable côté client.
- Double échéance : glissante de 30 minutes, absolue de 8 h.
- Chaque rotation change le nonce et garde l'identifiant, ce qui permet de reconnaître un cookie ancien.
- Un ancien nonce présenté hors de la fenêtre de grâce est traité comme un vol : la session est détruite et la version de jeton du compte incrémentée, ce qui déconnecte tous les appareils.

DÉTAILS QUI COMPTENT

- Un e-mail inconnu et un mot de passe faux renvoient la même erreur, après le même temps de calcul : une empreinte factice est vérifiée pour ne pas révéler l'existence du compte.
- Les rotations concurrentes sont sérialisées par un verrou : deux onglets qui renouvellent en même temps reçoivent le même cookie au lieu de se révoquer mutuellement.
- Le renouvellement est anticipé d'une minute, donc l'échéance n'est jamais atteinte pendant une navigation active.
- La déconnexion supprime l'enregistrement, pas seulement le cookie.

PARAMÈTRES DE LA SESSION

Cookie	opaque, httpOnly, SameSite=Lax Secure en production, Path=/ 32 octets aléatoires
Identifiant de session	32 octets aléatoires
Nonce	16 octets, renouvelé à chaque rotation
Expiration glissante	30 minutes d'inactivité
Expiration absolue	8 heures, non prolongeable
Jeton d'accès applicatif	5 minutes
Renouvellement	60 secondes avant l'échéance
Fenêtre de grâce	30 secondes après rotation
Verrou de rotation	5 s, 6 tentatives espacées de 25 ms
Hachage	argon2id, 19 456 Kio, 2 passes, 1 fil
Débit	10 connexions et 30 rotations / minute

CE QUI FERME UNE SESSION

Événement	Mécanisme	Portée
Déconnexion	l'enregistrement Redis est supprimé	cette session
Inactivité 30 min	la clé expire d'elle-même	cette session
8 heures écoulées	échéance absolue atteinte	cette session
Mot de passe changé	version de jeton incrémentée	tous les appareils
Rejeu détecté	version de jeton incrémentée	tous les appareils
Compte supprimé	session détruite, compte effacé	tous les appareils

La version de jeton est le levier de révocation globale : elle est comparée à chaque résolution de session, donc un simple incrément invalide instantanément toutes les sessions ouvertes du compte, sans avoir à les énumérer ni à parcourir Redis.

Choix de conception : ne pas utiliser de jeton signé lisible par le client. Un jeton signé ne peut pas être révoqué avant son échéance sans tenir malgré tout une liste de révocation côté serveur. Puisqu'une telle liste est nécessaire, autant garder l'état complet en mémoire et n'exposer qu'une référence opaque : la révocation devient immédiate et le contenu de la session n'est jamais visible du client.

Annexe J, Journal des décisions d'architecture (ADR) (BC1)

Journal des décisions d'architecture (ADR)

Registre des décisions structurantes de la plateforme dropicture, chacune formalisée au format Contexte / Décision / Alternatives / Conséquences — mémoire des arbitrages et support de réévaluation.

10
DÉCISIONS

C-D-A-C
FORMAT

2
REPLIS
LICENCE

100%
TRACÉES

FORMAT C-D-A-C

Contexte → Décision → Alternatives écartées → Conséquences. Chaque décision structurante est tracée, datée et réévaluable ; deux décisions portent un **repli de licence** documenté (OpenTofu, Valkey).

ADR-01 ORCHESTRATION

Adoptée

Docker Swarm

Cluster de quelques nœuds, équipe réduite.

ALTERNATIVES ÉCARTÉES

Kubernetes k3s Nomad

Conséquence — Primitives natives (placement, rolling update, secrets, overlay) à surcharge minimale ; K8s conservé comme cible.

ADR-02 INFRA AS CODE

Repli OpenTofu

Terraform + Ansible

Multi-fournisseurs à décrire déclarativement.

ALTERNATIVES ÉCARTÉES

Pulumi CloudFormation tout-Ansible

Conséquence — Séparation provisioning / configuration, deux cycles idempotents.

ADR-03 BACKEND

Adoptée

NestJS 11 sous Bun

Besoin d'une ossature structurante et performante.

ALTERNATIVES ÉCARTÉES

Express nu Fastify seul

Conséquence — DI, modularité par domaine, pipes de validation ; hautes perfs sous Bun.

ADR-04 PERSISTANCE

Adoptée

PostgreSQL 18 + TypeORM

Domaine relationnel, intégrité forte.

ALTERNATIVES ÉCARTÉES

NoSQL à schéma implicite

Conséquence — Index partiels, migrations explicites, cascade = droit à l'effacement.

ADR-05 CACHE & SESSIONS

Repli Valkey

Redis 8

Sessions opaques, cache, verrous, throttling.

ALTERNATIVES ÉCARTÉES

Memcached

Conséquence — Interface unique multi-usage.

ADR-06 FRONTEND

Adoptée

Next.js 16 — SSR + export statique

App riche + site public à très bas coût.

ALTERNATIVES ÉCARTÉES

SPA React serveur SSR dédié

Conséquence — Profils publics servis sans serveur (S3+CDN) : coût et surface d'attaque réduits.

ADR-07 OBSERVABILITÉ

Adoptée

Prometheus · Loki · Grafana

Souveraineté et coût maîtrisé exigés.

ALTERNATIVES ÉCARTÉES

Datadog / New Relic ELK / EFK

Conséquence — Auto-hébergé, sans licence, config versionnée.

ADR-08 HÉBERGEMENT

Adoptée

Multi-fournisseurs

Meilleur rapport coût / souveraineté par besoin.

ALTERNATIVES ÉCARTÉES

hyperscaler unique PaaS managé

Conséquence — Hetzner (calcul) · Cloudflare (bord) · AWS eu-west-3 (stockage) ; secrets répartis à surveiller.

ADR-09 AUTHENTIFICATION

Adoptée

Sessions opaques Redis

Révocation immédiate impérative.

ALTERNATIVES ÉCARTÉES

JWT stateless

Conséquence — tokenVersion (révocation globale), rotation nonce, détection rejeu ; rien d'exploitable côté client.

ADR-10 MESURE ÉNERGIE

A affiner

Modèle CPU → watts

RAPL indisponible sur VM cloud.

ALTERNATIVES ÉCARTÉES

Scaphandre / Kepler (bare-metal)

Conséquence — Indicateur pilotable kWh/j & gCO₂/h ; mesure matérielle en perspective.

RÉVERSIBILITÉ & TRAJECTOIRE D'ÉVOLUTION

Terraform → OpenTofu

licence BSL 2023 — fork Linux Foundation, config compatible

Redis → Valkey

changement de licence 2024 — interface compatible

Swarm → k3s / Kubernetes

si le trafic ou l'équipe croissent (scaling, écosystème)

Estimation → RAPL matériel

Scaphandre / Kepler sur bare-metal dédié

Rolling → canari / bleu-vert

stratégies de déploiement plus fines

Sécurité → DevSecOps

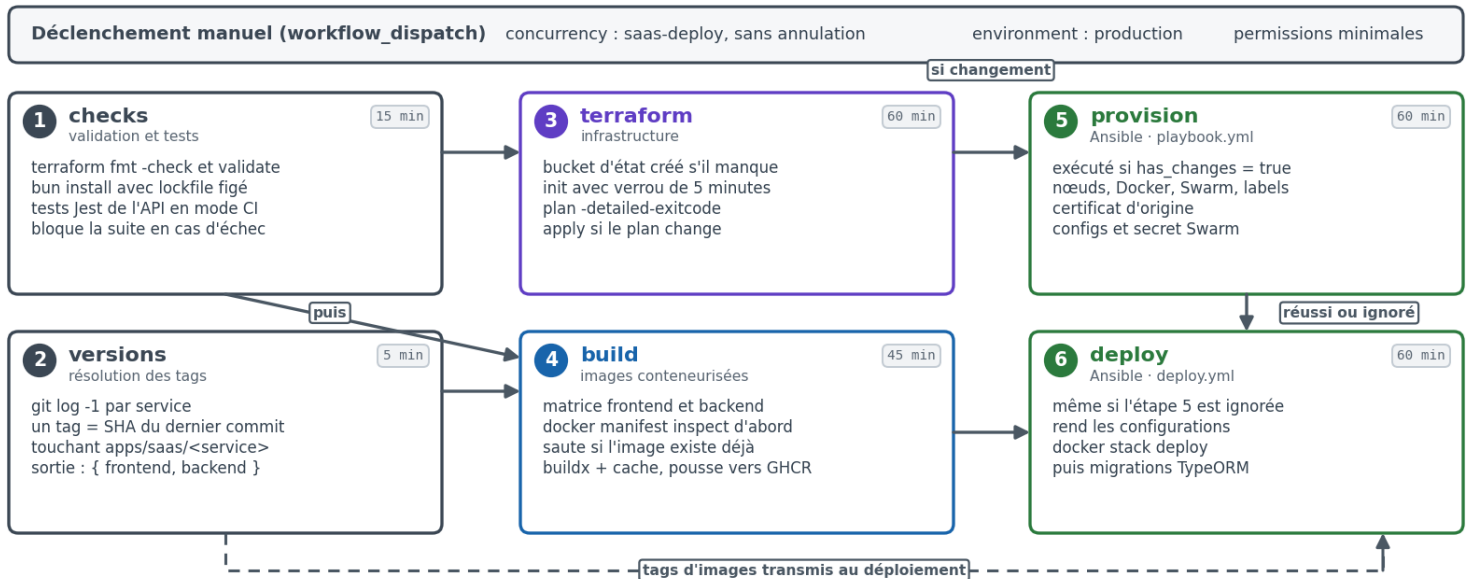
scans images / SCA / SAST / IaC en portes bloquantes

Chaque décision préserve une **trajectoire de repli** sans reprise majeure : la réversibilité est un critère d'architecture à part entière, tracé au même titre que la décision elle-même.

Annexe K, Pipeline CI/CD (chaîne de déploiement SaaS) (BC4C)

Chaîne de déploiement SaaS — GitHub Actions

6 jobs, 2 exécutés en parallèle, 1 conditionnel · du test au cluster en une seule exécution reproductible



DÉTAIL DES JOBS

Job	Dépend de	Condition	Ce qu'il produit
checks	–	toujours	feu vert : format, types, tests unitaires de l'API
versions	–	toujours	un identifiant d'image par service, dérivé du dernier commit
terraform	checks	toujours	infrastructure à jour et indicateur de changement
build	checks, versions	toujours	images poussées vers le registre, ou rien à faire
provision	terraform	si l'infra a changé	nœuds préparés, Swarm initialisé, objets de configuration
deploy	versions, build, provision	provision réussi ou ignoré	stack déployée et migrations appliquées

SECRETS UTILISÉS

Fournisseurs : jeton Hetzner, clé publique SSH, jeton Cloudflare, identifiants AWS
Application : base de données (nom, utilisateur, mot de passe), URL publique de l'API, mot de passe Grafana
Registre : jeton GHCR pour pousser les images, puis les distribuer aux nœuds au moment du déploiement
Accès aux machines : clé privée SSH encodée, matérialisée en fichier temporaire 0600 puis supprimée
Aucun de ces secrets n'apparaît dans le dépôt, dans une image, ni dans les journaux : les tâches sensibles sont marquées no_log.

GARDE-FOUS

- Un seul déploiement à la fois : les exécutions s'attendent au lieu de s'annuler, donc aucune ne laisse le cluster à moitié mis à jour.
- Les secrets sont portés par l'environnement « production », ce qui trace chaque exécution.
- Chaque job a son propre plafond de durée, de 5 à 60 min.
- Les images sont adressées par SHA de commit : rejouer la chaîne redéploie exactement le même artefact.
- Le déploiement refuse de s'exécuter sur un Swarm qui n'a pas été provisionné : il vérifie d'abord les objets attendus.

CE QUI ÉVITE LE TRAVAIL INUTILE

- Une image n'est reconstruite que si son tag est absent du registre : un service inchangé n'est pas rebâti, même si l'autre l'est.
- Le provisionnement ne tourne que si le plan Terraform annonce des changements réels.
- Le code de sortie détaillé du plan distingue trois cas : aucun changement, erreur, changements à appliquer.
- Caches séparés pour les dépendances et pour les couches d'image, avec une portée par service.
- Les migrations ne s'appliquent que si elles sont en attente.

DEUX SCÉNARIOS D'EXÉCUTION

Livraison applicative seule — le code change, l'infrastructure non

checks et versions en parallèle, terraform ne trouve aucun changement, provision est ignoré, build reconstruit le seul service modifié, puis deploy met à jour la stack sans coupure et applique les migrations en attente.

Changement d'infrastructure — un nœud est ajouté dans terraform.tfvars

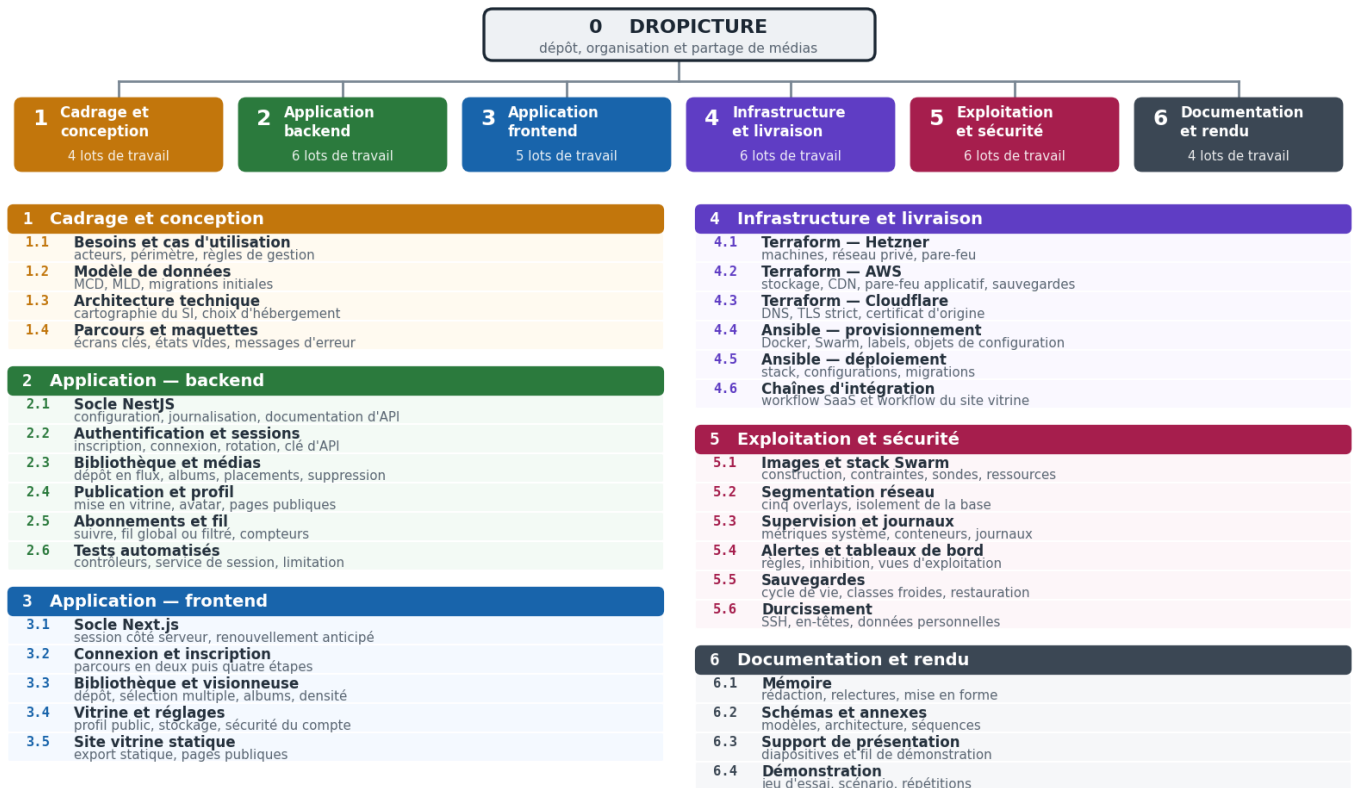
terraform crée la machine et met à jour l'état, provision s'exécute : la nouvelle machine est préparée, rejoint le Swarm et reçoit son label de rôle. deploy recalcule alors le nombre de réplicas depuis l'état et redéploie la stack en conséquence.

Le même workflow couvre les deux cas : c'est la conditionnalité entre les jobs qui adapte le travail réellement effectué, et non deux chaînes séparées qu'il faudrait maintenir en parallèle.

Annexe L, Work Breakdown Structure (WBS) (BC2)

Organigramme des tâches (WBS)

6 lots, 31 lots de travail · décomposition arrêtée au niveau où une charge est estimable et un livrable identifiable



RÈGLE DE DÉCOUPAGE

- Un lot de dernier niveau est livrable, testable et attribuable à une personne.
- La décomposition s'arrête dès que la charge est estimable à la demi-journée près : plus bas, on produirait une liste de tâches.
- Chaque lot porte un identifiant stable, réutilisé dans le planning et le suivi d'avancement.

DÉPENDANCES STRUCTURANTES

- 1.2 conditionne tout le lot 2 : sans modèle de données figé, ni entités ni migrations.
- 4.2 conditionne 2.3 : le dépôt suppose un stockage objet et un CDN déjà en place.
- 4.4 conditionne 4.5 : on ne déploie pas sur un cluster qui n'a pas été préparé.
- Les lots 3 et 5 avancent en parallèle du lot 2.

JALONS

J1	Modèle de données validé	clôture 1.2	le schéma est migré et rejouable
J2	API validée par les tests	clôture 2.6	la suite de tests passe en intégration
J3	Parcours complet en local	clôture 3.4	dépôt, publication et consultation enchaînés
J4	Infrastructure rejouable	clôture 4.5	une destruction suivie d'une recreation aboutit
J5	Mise en production	clôture 4.6 · 5.2	le service répond sur son domaine, supervisé
J6	Mémoire et soutenance	clôture 6.4	document rendu et démonstration répétée

Un jalon n'est pas une date mais un état vérifiable : chacun se constate en exécutant quelque chose, pas en consultant un document.

CE QUI NE FIGURE PAS DANS CE DÉCOUPAGE

L'exploitation courante après la mise en service : surveillance quotidienne, réponse aux incidents, montées de version des composants. Ces activités sont continues et sans fin déterminée, elles relèvent du maintien en condition opérationnelle et non du projet. Elles sont néanmoins préparées par les lots 5.3 à 5.5, qui livrent les outils nécessaires : tableaux de bord, règles d'alerte et procédure de restauration.

RÉPARTITION INDICATIVE DE LA CHARGE

Proportion du temps consacré à chaque lot, à titre de repère plutôt que de mesure exacte.

1 Cadrage et conception		10 %
2 Application — backend		26 %
3 Application — frontend		22 %
4 Infrastructure et livraison		20 %
5 Exploitation et sécurité		14 %
6 Documentation et rendu		8 %

Le poids du lot 4 s'explique par le choix d'une infrastructure entièrement décrite en code plutôt que configurée à la main : l'effort est concentré en amont, mais chaque redéploiement ultérieur devient gratuit et reproductible.

Annexe M, Diagramme PERT et chemin critique (BC2)

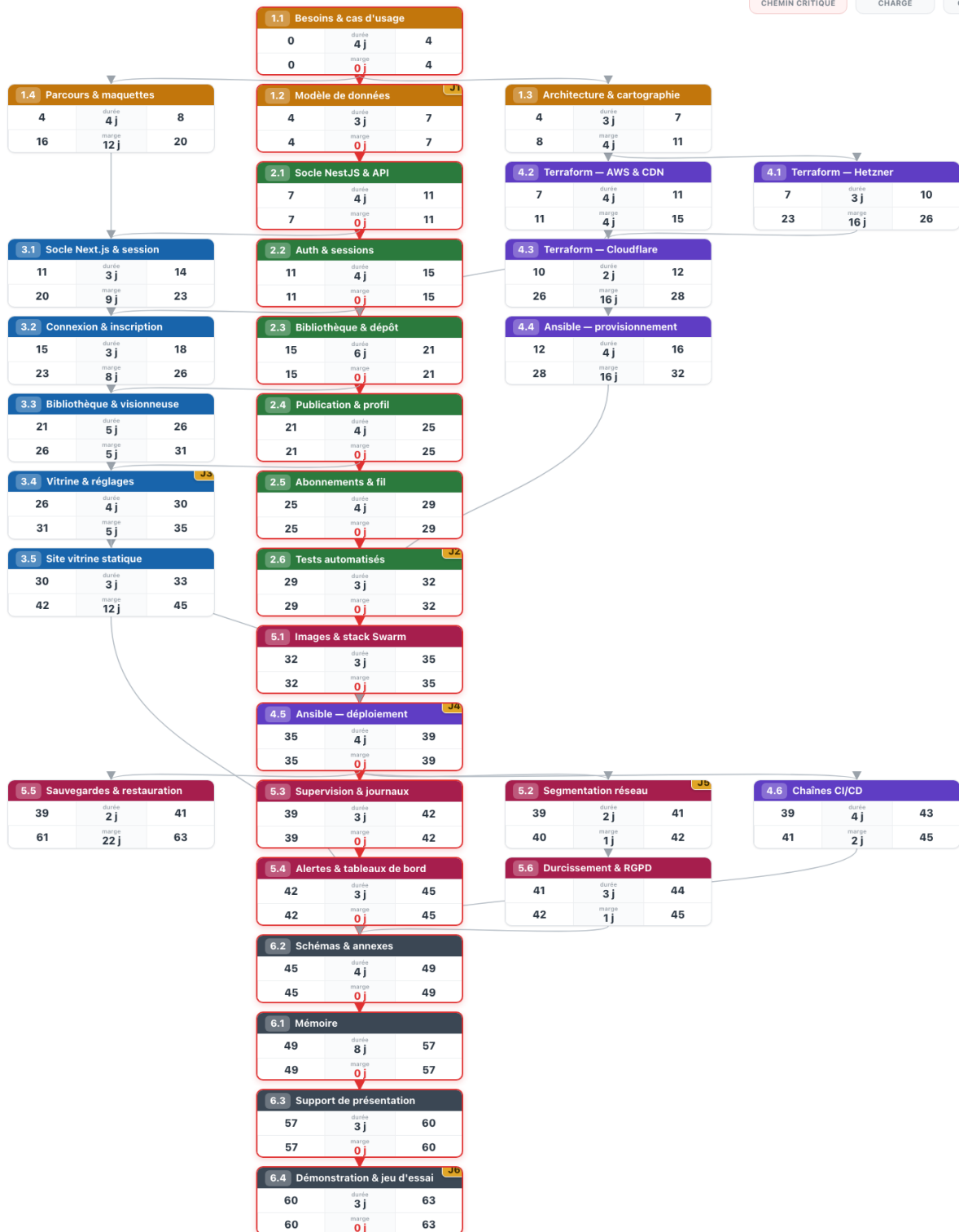
Diagramme de PERT & chemin critique

Séquençage des 31 tâches du WBS en réseau à antécédents (méthode des potentiels-tâches). Chaque nœud porte ses dates au plus tôt et au plus tard ; la marge nulle dessine le chemin critique.

63 j
CHEMIN CRITIQUE

112 j·p
CHARGE

16/31
CRITIQUES



LECTURE D'UN NŒUD

2.3 Tâche				
début	durée	fin		
15	6 j	21		
début	marge	fin		
15	0 j	21		

Coins : dates au plus tôt (haut) et au plus tard (bas).
Milieu : durée et marge totale.
— chemin critique
— antériorité (marge)

LES 6 LOTS — COULEUR DES NŒUDS

- Lot 1 : Cadrage & conception
- Lot 2 : Application — backend
- Lot 3 : Application — frontend
- Lot 4 : Infrastructure & livraison
- Lot 5 : Exploitation & sécurité
- Lot 6 : Documentation & rendu

JALONS & MARGES

- J1 Modèle de données validé — 1.2
- J2 API validée par les tests — 2.6
- J3 Parcours complet en local — 3.4
- J4 Infrastructure reposable — 4.5
- J5 Mise en production — 4.6 - 5.2
- J6 Mémoire & soutenance — 6.4

Frontend et infrastructure gardent 5 à 16 j de marge. Menée à une personne, la charge (112 j·p) est réalisée en série ; le PERT fixe l'ordre et la borne minimale de 63 j ouvrés.

Annexe N, Planning (diagramme de Gantt) (BC2)

Planning — diagramme de Gantt

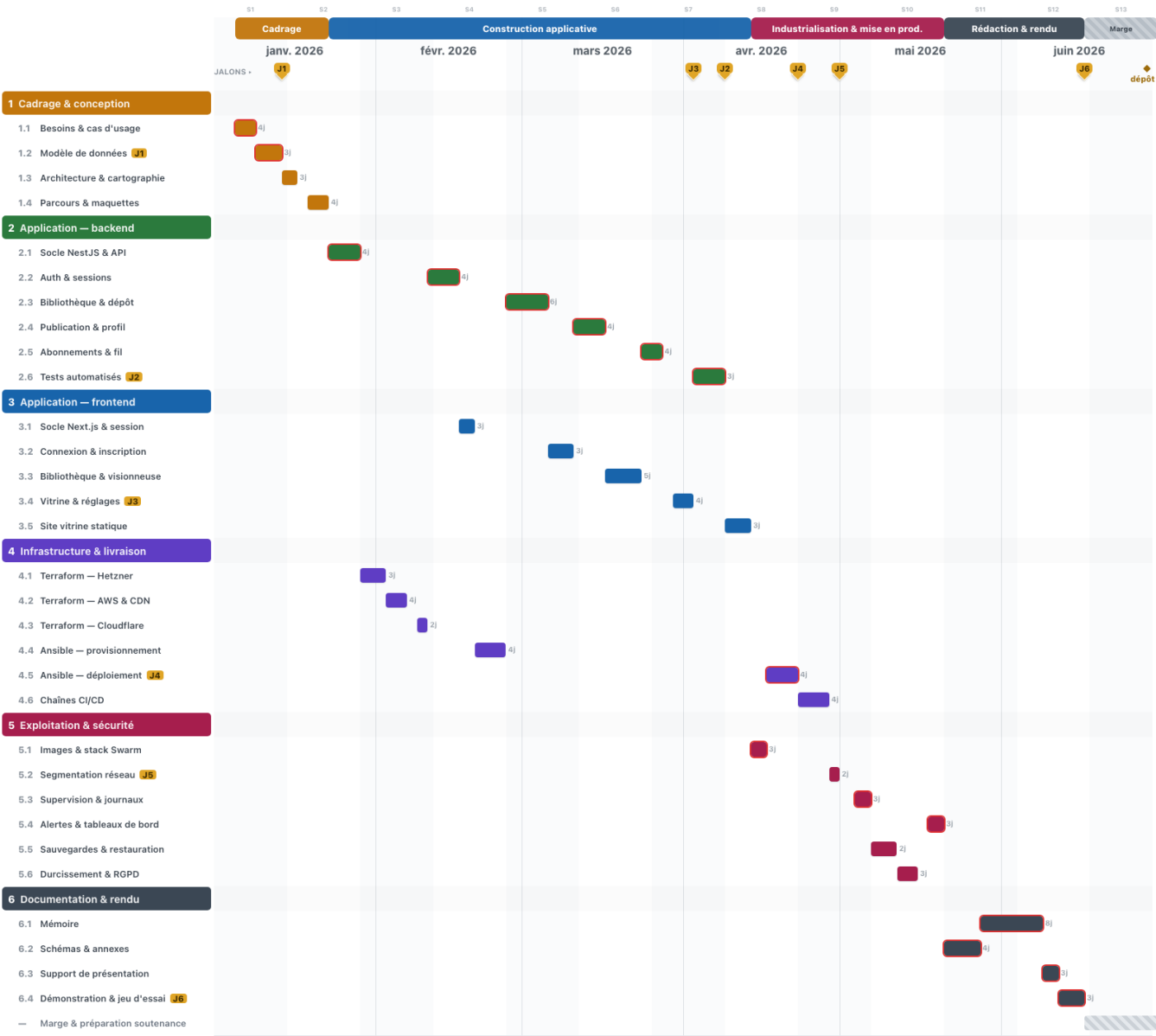
1^{er} janvier → 30 juin 2026 · 26 semaines · 13 sprints de 2 semaines · développement itératif à une personne · chemin critique en rouge

6 mois
1 JAN → 30 JUIN

112 j-p
CHARGE · 1 RESSOURCE

13
SPRINTS (2 SEM.)

J1–J6
6 JALONS



LÉGENDE

Lot 1 · Cadrage & conception

Lot 2 · Application — backend

Lot 3 · Application — frontend

Lot 4 · Infrastructure & livraison

Lot 5 · Exploitation & sécurité

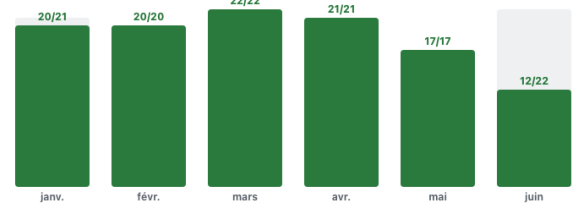
Lot 6 · Documentation & rendu

🔴 tâche du chemin critique

📌 jalon Jx

📅 marge / préparation soutenance

PLAN DE CHARGE — JOURS OUVRÉS UTILISÉS / DISPONIBLES PAR MOIS



La charge (vert) reste sous la capacité mensuelle (gris) : ordonnancement en série, week-ends et jours fériés exclus.

RESSOURCES & BUDGET

Ressource humaine	1 développeur · 112 j-p
Serveur Hetzner (CAX11)	≈ 15 €/mois
Stockage S3 + CDN CloudFront	≈ 10 €/mois
DNS / TLS Cloudflare	0 € — gratuit
Nom de domaine	≈ 12 €/an
Total infrastructure	≈ 26 €/mois · ~155 €

Développement itératif, une personne. La marge du 16 au 30 juin couvre les aléas et la préparation de soutenance ; le chemin critique (rouge, cf. PERT) conditionne la date de fin.

Annexe O, Matrice des risques (probabilité / impact) (BC2)

Matrice des risques — probabilité / impact

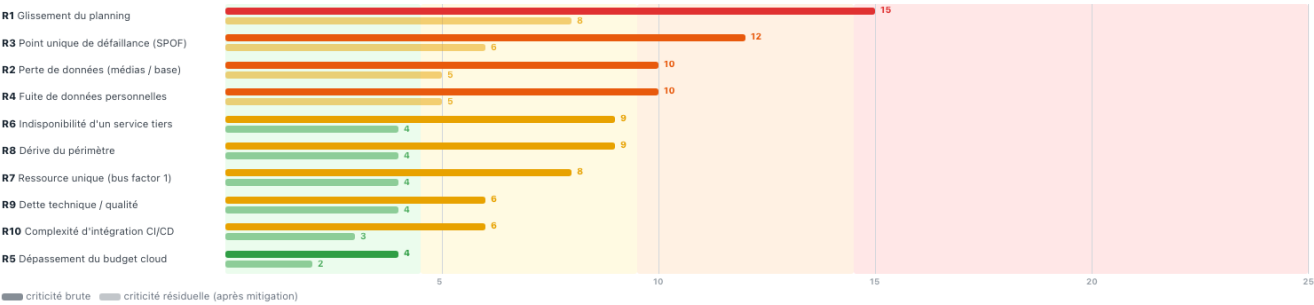
Échelle 5x5 avec codes couleurs significatifs et hiérarchisation par criticité (P×I). 10 risques du projet, stratégie et plan de mitigation ; la flèche indique la criticité résiduelle visée.



REGISTRE DES RISQUES — HIÉRARCHISÉ PAR CRITICITÉ DÉCROISSANTE

#	RISQUE	CATÉGORIE	CAUSE PRINCIPALE	P	I	CRITICITÉ	STRATÉGIE	PLAN D'ACTION DE MITIGATION	RÉSIDUEL
R1	Glissement du planning	Gestion de projet	Sous-estimation de la charge, ressource unique	3	5	15 · Critique	Réduire	Marge de 2 semaines (16→30 juin), jalons hebdomadaires, priorité au chemin critique, périmètre MVP figé	8 · Modéré
R3	Point unique de défaillance (SPOF)	Infrastructure	Cluster mono-nœud, proxy Traefik unique	3	4	12 · Élevé	Réduire	Healthchecks + rollback, infrastructure rejoinable en IaC (Terraform/Ansible), runbook de restauration	6 · Modéré
R2	Perte de données (médias / base)	Données	Suppression accidentelle, panne de stockage	2	5	10 · Élevé	Réduire / Transférer	Sauvegardes automatisées + test de restauration (lot 5.5), versioning S3, snapshots Postgres (PITR)	6 · Modéré
R4	Fuite de données personnelles	Sécurité / RGPD	API exposée, mauvaise gestion des secrets	2	5	10 · Élevé	Réduire	Helmet, throttler Redis, ValidationPipe, CORS strict, durcissement (5.6), scans Trivy / SAST en CI	5 · Modéré
R6	Indisponibilité d'un service tiers	Fournisseur	Panne Hetzner / AWS / Cloudflare / GHCR	3	3	9 · Modéré	Transférer / Accepter	SLA fournisseurs, IaC pour recréer ailleurs, images publiées sur GHCR, stockage multi-AZ	4 · Faible
R8	Dérive du périmètre	Périmètre	Retours utilisateurs, ambition fonctionnelle	3	3	9 · Modéré	Éviter / Réduire	Périmètre figé dans la note de cadrage, backlog priorisé, MVP, arbitrage à chaque sprint	4 · Faible
R7	Ressource unique (bus factor 1)	Organisation	Projet mené par une seule personne	2	4	8 · Modéré	Réduire	Documentation continue (lot 6), automatisation CI/CD + IaC (rejoinabilité), marges au planning	4 · Faible
R9	Dette technique / qualité	Qualité	Pression du planning sur les tests	3	2	6 · Modéré	Réduire	Tests automatisés (2.6), seuil de couverture, lint + revues en CI, refactoring par sprint	4 · Faible
R10	Complexité d'intégration CI/CD	Technique	Chaîne d'outils multiple (build → déploiement)	2	3	6 · Modéré	Réduire	Pipeline reproductible, déploiement idempotent (Ansible), environnement de test, cache d'images	3 · Faible
R5	Dépassement du budget cloud	Financier	Volume de médias, egress du CDN	2	2	4 · Faible	Accepter / Réduire	Alertes de coût, quotas, offres gratuites Cloudflare, classes de stockage froides S3	2 · Faible

CRITICITÉ PAR RISQUE — AVANT / APRÈS MITIGATION



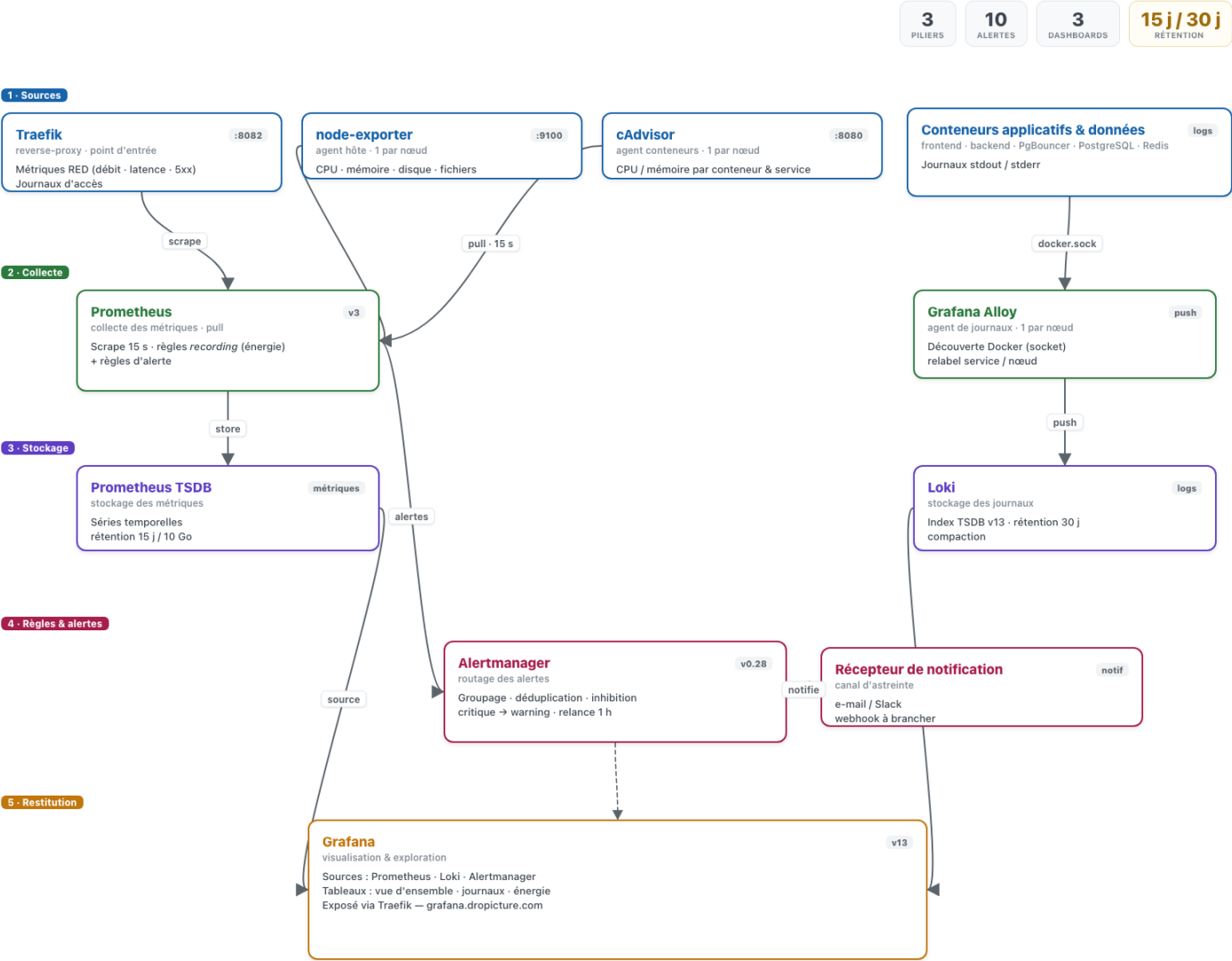
PILOTAGE DES RISQUES

Le registre est **revu à chaque sprint** : réévaluation de la probabilité et de l'impact, suivi des actions et de la criticité résiduelle. Après mitigation, **aucun risque ne demeure en zone critique ou élevée** (0 sur 10) — R1 (planning), R2 et R4 (données, RGPD) restent les points de vigilance prioritaires. La matrice complète les livrables de pilotage **WBS · PERT · planning** et alimente la stratégie de sécurité et de continuité du projet.

Annexe P, Architecture de la supervision et de l'observabilité (BC4C)

Architecture de la supervision & de l'observabilité

Pile auto-hébergée déployée sur le réseau overlay **monitoring** du cluster Swarm : métriques (Prometheus), journaux (Loki via Alloy) et estimation énergétique, alertes routées par Alertmanager, restitution dans Grafana.



TROIS PILIERS DE L'OBSERVABILITÉ

Métriques — Prometheus

Hôte (node-exporter), conteneurs (cAdvisor) et applicatif RED (Traefik). Scrape pull 15 s.

Journaux — Loki

Agent Alloy sur chaque nœud, découverte Docker via socket, relabel par service et nœud.

Énergie — Green IT

Règles recording : puissance estimée (W), kWh/jour, gCO₂/h. Traces distribuées : perspective.

RÈGLES D'ALERTE & ROUTAGE

- **Disponibilité** — cible / nœud injoignable (up == 0).
- **Ressources** — CPU > 90 %, RAM < 10 %, disque saturé ou prédictif (24 h).
- **Application** — 5xx > 5 %, latence p95 > 1 s, redémarrages en boucle.
- **Énergie** — puissance estimée du cluster > 200 W.

Alertmanager : groupage par *alertname* / sévérité, déduplication, inhibition *critique* → *warning* sur un même nœud, relance 1 h pour les critiques. Récepteur (e-mail / Slack) à brancher.

CHOIX D'ARCHITECTURE & SOBRIÉTÉ

- **Pull-based** et **auto-hébergé** — aucune dépendance à un SaaS tiers (souveraineté des données, coût maîtrisé).
- Réseau overlay **monitoring** isolé, séparé des flux applicatifs (frontend / backend / egress).
- Agents **globaux** et **légers** (node-exporter, cAdvisor, Alloy ≈ 1 par nœud) sous *limits* Swarm.
- **Rétention bornée** — 15 j métriques / 30 j journaux, empreinte disque contenue.
- Énergie estimée **sans RAPL** (VM cloud) via un modèle CPU → watts.
- Provisionné en **IaC** (Ansible) et versionné : pile rejouable à l'identique.

Annexe Q, Contrat d'interface (OpenAPI / Swagger) (BC3)

Contrat d'interface — API REST dropicture

Surface exposée par le backend NestJS, spécifiée en **OpenAPI 3** (/api/docs, JSON /api/docs-json) — générée depuis le code et les annotations de validation, donc toujours synchronisée avec l'implémentation.

6

DOMAINES

46

ROUTES REST

2

MODES D'AUTH

3.0

OPENAPI

AUTHENTIFICATION

Session — cookie session opaque httpOnly adossé à Redis, glissante 30 min / absolue 8 h, rotation par nonce, détection de rejeu. · Clé d'API — en-tête x-api-key (ou ?appid=).

CONVENTIONS

Erreurs { code } (ex. INVALID_CREDENTIALS) · débit limité par IP/route (x-ratelimit=*, stockage Redis) · pagination par curseur opaque cursor / nextCursor.

Bibliothèque /api/library session

POST	/api/library/uploads	60/min
GET	/api/library/summary	120/min
GET	/api/library	240/min
POST	/api/library/download	30/min
PATCH	/api/library/publish	60/min
PATCH	/api/library/unpublish	60/min
DELETE	/api/library/media	30/min
GET	/api/library/albums	120/min
POST	/api/library/albums	30/min
PATCH	/api/library/albums/:id	60/min
POST	/api/library/albums/:id/media	60/min
DELETE	/api/library/albums/:id/media	60/min
PATCH	/api/library/albums/:id/cover:m	60/min
DELETE	/api/library/albums/:id	30/min

Paramètres /api/settings session

GET	/api/settings	120/min
PATCH	/api/settings/me	20/min
PATCH	/api/settings/username	10/min
PATCH	/api/settings/email	10/min
PATCH	/api/settings/password	10/min
GET	/api/settings/apikey	20/min
POST	/api/settings/apikey	5/min
DELETE	/api/settings/apikey	5/min
DELETE	/api/settings/account	5/min

Authentification /api/auth session · clé

GET	/api/auth/me	60/min
GET	/api/auth/username/:u	60/min
POST	/api/auth/resolve	120/min
POST	/api/auth/signin	10/min
POST	/api/auth/signup	5/h
POST	/api/auth/session	30/min
POST	/api/auth/signout	20/min

API publique /api/public ouvert

GET	/api/public/stats	120/min
GET	/api/public/search	240/min
GET	/api/public/profiles	120/min
GET	/api/public/feed	240/min
GET	/api/public/:username	120/min
GET	/api/public/:username/media	240/min

Profil /api/profile session

GET	/api/profile	120/min
PATCH	/api/profile	30/min
GET	/api/profile/media	240/min
PATCH	/api/profile/media/unpublish	60/min
POST	/api/profile/avatar	20/min
DELETE	/api/profile/avatar	20/min

Découverte /api/discover session

GET	/api/discover/feed	240/min
GET	/api/discover/me	120/min
POST	/api/discover/follows/:u	60/min
DELETE	/api/discover/follows/:u	60/min

MODÈLE D'ERREUR & PAGINATION

Erreurs normalisées { code: string } — INVALID_CREDENTIALS, ACCOUNT_NOT_FOUND, USERNAME_TAKEN, QUOTA_EXCEEDED, REPLAY_DETECTED.
Pagination par **curseur opaque** — paramètre cursor, réponse nextCursor : temps de réponse constant, robuste aux insertions concurrentes.
En-têtes de débit x-ratelimit-limit / -remaining / -reset.

CONTRÔLES TRANSVERSES

· Validation **DTO en liste blanche** (ValidationPipe global : rejet des champs non déclarés, transformation de types).
· **Throttler** @nestjs adossé à Redis, différencié par route.
· **Helmet** + HSTS, **CORS** en liste blanche, corps de requête plafonné.
· Champs secrets (empreinte, clé d'API) exclus par défaut (select: false).
· Guards Passport : access-token (session) et api-key.

EXEMPLE D'ÉCHANGE — CONNEXION

```
POST /api/auth/signin      Content-Type: application/json
{ "login": "amir", "password": "*****" }

200 OK Set-Cookie: session=sid.nonce; HttpOnly; Secure; SameSite=Lax
{ "username": "amir", "createdAt": "2026-07-21T..." }

429 (quota dépassé - 10/min) { "code": "THROTTLED" }
```

ENTITÉS EXPOSÉES PAR L'API

Account
identité, empreinte mot de passe, clé d'API

Media
fichier, rôle, date de publication

Album
regroupement, média de couverture

Placement
association ordonnée album↔média

Follow
relation réflexive entre comptes

Annexe R, Cartographie ISO/IEC 27001 & RGPD (BC4C)

Cartographie de la sécurité — défense en profondeur

Contrôles de sécurité de la plateforme organisés en quatre couches, du bord vers la donnée, et rattachés aux mesures de l'Annexe A d'ISO/IEC 27001:2022 et aux articles du RGPD.

4
COUCHES
DÉFENSE

27
CONTRÔLES

ISO
27001
V. 2022

RGPD
CONFORME

1 - Périphérie Cloudflare · AWS	
WAF Cloudflare — filtrage du trafic entrant A.8.20 A.8.23	TLS strict · HSTS 2 ans preload A.8.24
Protection volumétrique (anti-DDoS) A.8.6	AWS WAFv2 — rate-based · IP reputation · common rules A.8.20
CloudFront — redirection HTTPS forcée A.8.24	
2 - Réseau Hetzner · Swarm	
Pare-feu Hetzner — 80/443 depuis Cloudflare uniquement A.8.20 A.8.22	Accès SSH par clé · durcissement système A.8.5 A.8.9
5 réseaux overlay segmentés (moindre privilège) A.8.22	backend & socket en <i>internal</i> — sans route Internet A.8.22
socket-proxy Docker en lecture seule A.8.2 A.8.18	Plan de contrôle Swarm sur IP privée A.8.20
3 - Application NestJS · Bun	
Validation DTO en liste blanche A.8.28	Argon2id — mem 19 456 · t=2 · p=1 A.8.24
Sessions opaques · rotation nonce · détection rejeu A.8.5 A.8.15	tokenVersion — révocation globale immédiate A.8.5
Rate limiting Redis différencié par route A.8.6 A.8.20	Helmet · en-têtes de sécurité · HSTS A.8.24
CORS liste blanche · corps de requête plafonné A.8.28	Journaux d'accès structurés JSON A.8.15 A.8.16
4 - Données PostgreSQL · S3	
Chiffrement au repos AES-256 (SSE) — tous les buckets A.8.24 RGPD Art.32	TLS en transit · refus de connexion non sûr A.8.24 RGPD Art.32
Versioning S3 + Object Lock (GOVERNANCE) A.8.13	Cycle de vie STANDARD_IA → Glacier A.8.13
État Terraform chiffré · verrouillé · versionné A.8.9	Suppression en cascade = droit à l'effacement RGPD Art.17
Minimisation des données collectées RGPD Art.5	Hébergement UE — Hetzner DE · AWS eu-west-3 RGPD Art.44+

ISO/IEC 27001:2022 — ANNEXE A

A.5 Organisationnel (37)
politiques · gestion des secrets · fournisseurs

A.6 Personnes (8)
moindre privilège · gestion des accès

A.7 Physique (14)
délégué aux hébergeurs certifiés (Hetzner, AWS)

A.8 Technologique (34)
chiffrement · réseau · journalisation · dev. sécurisé

RGPD — POINTS DE CONFORMITÉ

· Art. 5 — minimisation & limitation des finalités

· Art. 17 — droit à l'effacement (cascade base + objets)

· Art. 32 — sécurité du traitement (chiffrement, sessions)

· Art. 44+ — pas de transfert hors UE (hébergement souverain)

GESTION DES SECRETS

Injectés depuis les environnements GitHub — jetons Hetzner/Cloudflare, clés AWS, mots de passe. Jamais dans le dépôt (assertions Ansible avant déploiement). Configs & secrets Swarm versionnés par empreinte, moindre privilège.

JOURNALISATION & RÉPONSE À INCIDENT

Journaux d'accès JSON → Loki, 9 règles d'alerte Prometheus. Retour arrière automatique des déploiements, révocation de sessions (tokenVersion) et détection de rejeu. Runbook pour les cas non automatisés.

SAUVEGARDE & REPRISE (PRA)

Dumps S3 versionnés + Object Lock, chiffrés AES-256. État Terraform distant, chiffré, verrouillé. Reprise = reconstruction par le code (Terraform + Ansible) puis restauration ; test de restauration inscrit au plan d'amélioration.

Annexe S, Chaîne DevSecOps et plan de reprise d'activité (BC4C)

Chaîne DevSecOps et plan de reprise d'activité

3
NIVEAUX
DE SCAN

4
SAUVEGARDES
PAR JOUR

CRITICAL
HIGH
SEUIL
BLOQUANT

Portes de sécurité automatiques du pipeline de livraison, de l'analyse du dépôt à l'analyse de l'artefact publié, et dispositif de **sauvegarde automatisée** et de **restauration outillée** de la base de production. Toutes les étapes sont décrites en code et versionnées avec le projet.

1 - Chaîne DevSecOps

GitHub Actions · `saas-deploy.yml` · Trivy v0.58.2

ÉTAPE CHECKS

Tests & validation IaC

218 tests Jest via `bun run test --ci`, puis `terraform fmt -check` et `terraform validate`.

BLOQUANT

ÉTAPE CHECKS

Dépendances & secrets

`trivy fs` — vulnérabilités des dépendances et détection de secrets versionnés, `ignore-unfixed`.

BLOQUANT

ÉTAPE CHECKS

Plans d'infrastructure

`trivy config` — analyse statique des fichiers Terraform, des playbooks Ansible et des Dockerfile.

BLOQUANT

ÉTAPE BUILD

Image de conteneur

`trivy image` sur l'artefact étiqueté par SHA, avant toute publication effective.

BLOQUANT

ÉTAPE BUILD

SBOM CycloneDX

Nomenclature logicielle par image : analyse de composition et inventaire des composants embarqués.

TRAÇABILITÉ

RESTITUTION ET TRAÇABILITÉ

- Rapports convertis au format **SARIF** et publiés dans **GitHub Code Scanning** — historisés et rattachés aux lignes de code concernées.
- Rapports détaillés **JSON** et SARIF archivés en artefacts pendant **90 jours**.
- Synthèse du nombre de vulnérabilités **par niveau de sévérité** portée dans le compte rendu d'exécution du pipeline.
- Base de vulnérabilités mise en cache entre exécutions (sobriété réseau et durée de pipeline).

GESTION DES EXCEPTIONS — `.TRIVYIGNORE`

- Toute dérogation est **centralisée dans un fichier versionné**, revue en même temps que le code.
- Chaque ligne porte obligatoirement **l'identifiant** (CVE ou règle), **la justification** et **la date de revue**.
- Une acceptation de risque demeure ainsi **explicite, tracée et temporaire**, jamais silencieuse.
- Permissions des jobs au moindre privilège : lecture seule par défaut, écriture sur les paquets et les événements de sécurité limitée aux seules étapes concernées.

2 - Sauvegarde automatisée

`saas-backup.yml` — cron `0 */6 * * *` · Ansible `backup.yml`

01 Découverte de la cible

Lecture de l'état Terraform distant pour localiser le nœud `db`. Sans état ni nœud, la chaîne se termine sans erreur.

02 Cliché de la base

`pg_dump` format custom, `compress=6`, sans propriétaire ni privilèges, depuis le conteneur en fonctionnement.

03 Gardes d'intégrité

Empreinte **SHA-256** calculée et refus explicite de tout cliché de taille invraisemblable (< 1 Ko).

04 Dépôt sur S3

Téléversement du cliché et de son empreinte sous le préfixe `daily/`, puis **vérification de la taille de l'objet déposé**.

05 Durcissement du stockage

Versioning, **Object Lock GOVERNANCE**, chiffrement AES-256, cycle de vie `STANDARD_IA` puis Glacier.

3 - Restauration outillée (PRA)

`saas-recovery.yml` — déclenchement manuel · Ansible `restore.yml`

01 Confirmation explicite

Saisie obligatoire de `RESTORE`. Choix du préfixe `daily/` ou `monthly/`, ou d'une clé S3 précise.

02 Gel des écritures

Backend descendu à **0 réplica** le temps de l'opération, afin qu'aucune écriture concurrente ne survienne.

03 Vérification

Téléchargement du cliché et de son empreinte, puis **contrôle SHA-256** avant toute action destructrice.

04 Cliché de sécurité

Sauvegarde de la base **courante** avant écrasement — la restauration elle-même est réversible.

05 Restauration & reprise

Transaction unique (annulation intégrale sur erreur), contrôle du schéma restauré, puis rétablissement du nombre initial de répliques.

Objectifs de reprise et amélioration continue

- RPO** — la planification toutes les six heures borne la perte de données maximale à **6 heures**.
- RTO** — reconstruction complète de l'infrastructure par le code (Terraform puis Ansible), suivie de la restauration de la base ; les médias restent disponibles depuis le stockage objet, qui n'est jamais détruit.
- Prochain palier** — formalisation d'un exercice de reprise périodique conduit en conditions réelles et suivi documenté de l'objectif de point de reprise, inscrits au plan d'amélioration continue.

Annexe F, Maquettes des interfaces et flux de données (BC3)

Maquettes des interfaces & flux de données

Interfaces réelles des principaux modules de la plateforme, chacune rattachée aux points d'accès de l'API qui l'alimentent (cf.

Annexe Q) et aux cas d'utilisation correspondants (cf. **Annexe G**). La frontière **privé** / **public** qui structure le produit est visible d'un écran à l'autre.

5
ÉCRANS

8
CAS
D'USAGE

Connexion / inscription

AUTH

BIBLIOTHÈQUE PHOTO ET VIDÉO

Contenu de te revoir.

Ta bibliothèque et ta vitrine t'attendent.

E-mail

1 / 2

loi@dropicture.com

Continuer

OU

Créer un compte

Flux : [POST /api/auth/resolve](#) (étape 1/2) puis [POST /api/auth/signin](#) → session opaque, cookie [sid.nonce](#), rotation par [POST /api/auth/session](#).

Bibliothèque privée

LIBRARY

dropicture

Fil Bibliothèque Vitrine

Chercher un compte

7

A

PRIVE

Ta bibliothèque

Tu seul vois ces fichiers. Publier en envoies un sur ta vitrine.

6 éléments privés · 1,8 Mo

Dépose tes photos et tes vidéos

Arrive en privé · jusqu'à 100 Mo par fichier

Parcourir

ALBUMS · RANGEMENT PRIVE

Nom de l'album...

Créer Annuler

2026

6 éléments

Favorites

6 éléments

0

Juillet 2026

6 éléments

Flux : [GET /api/library](#) (pagination par curseur) · [GET /api/library/summary](#) · [GET-POST /api/library/albums](#). Tout y est **privé** par défaut.

Téléversement d'un média

UPLOAD

dropicture

Fil Bibliothèque Vitrine

Chercher un compte

7

A

PRIVE

Ta bibliothèque

Tu seul vois ces fichiers. Publier en envoies un sur ta vitrine.

6 éléments privés · 1,8 Mo

Dépose tes photos et tes vidéos

Arrive en privé · jusqu'à 100 Mo par fichier

Parcourir

LIBRARY · RANGEMENT PRIVE

Nom de l'album...

Créer Annuler

2026

6 éléments

Favorites

6 éléments

0

Juillet 2026

6 éléments

Flux : [POST /api/library/uploads](#) → flux multipart vers S3 par tranches de 5 Mo, garde de taille 8 Mo (image) / 100 Mo (vidéo), sans mise en mémoire du fichier.

Vitrine & publication

PUBLISH

Fil Bibliothèque Vitrine

Chercher un compte

7

A

PUBLIC

Ta vitrine

Tout ce qui est sur cette page est visible de tous. Le reste attend dans ta bibliothèque.

@amir.redjem

Aucune description.

1 en vitrine · 6 en bibliothèque · depuis le 22 juillet 2026

Modifier

[http://localhost:3000/~/amir.redjem](#)

Copier Voir la page

Publié

Tout sélectionner

Publier depuis ma bibliothèque

Flux : [GET /api/profile/media](#) · [PATCH /api/library/publish](#) (datation idempotente → invalidation CDN) · [PATCH /api/profile/media/unpublish](#).

Fil & découverte

DISCOVER

dropicture

Fil Bibliothèque Vitrine

Chercher un compte

7

A

PUBLIC

Le fil commence avec toi

Tout ce que les comptes ont mis en vitrine, du plus récent au plus ancien.

6 médias · 1 compte · tu suis 0

Tout

Suivre

Flux : [GET /api/discover/feed](#) (fil global ou restreint aux abonnements) · [POST-DELETE /api/discover/follows/:username](#) · [GET /api/public/search](#). Le profil public exporté statiquement est servi par [GET /api/public/:username](#) depuis S3 et CloudFront.

Convention. Ces captures documentent l'état livré des interfaces plutôt qu'une intention graphique. Elles ont été précédées, en phase de conception, de maquettes basse fidélité ayant servi à valider l'agencement des écrans et les parcours utilisateurs avec les parties prenantes avant tout développement. Chaque écran est rattaché à un ou plusieurs points d'accès de l'API documentée en OpenAPI (cf. Annexe Q) et aux cas d'utilisation (cf. Annexe G), ce qui garantit la traçabilité du besoin jusqu'à l'implémentation. Les surfaces respectent les référentiels **WCAG 2.2** et **RGAA 4** (structure sémantique, hiérarchie de titres, champs étiquetés, navigation au clavier avec focus visible, contrastes et langue déclarée).

dropicture — Annexe F · Maquettes des interfaces et flux de données (BC3)

Captures originales de l'auteur

Annexe L bis, Matrice RACI (BC2)

Matrice RACI — rôles & responsabilités

8

LOTS

6

ACTEURS

1 A

PAR LOT

Répartition Responsible / Accountable / Consulted / Informed par lot de la WBS et par partie prenante. Un seul décideur (A) par lot, distribution équilibrée des rôles sur l'équipe projet reconstituée.

AFFECTATION DES RÔLES PAR LOT DE TRAVAIL						
LOT DE TRAVAIL (WBS)	CHEF DE PROJET / DEVOPS	DÉVELOP-PEURS	RÉFÉRENT SÉCURITÉ	SPONSOR	RÉFÉRENT PÉDAGO.	RÉFÉRENT ENTREPRISE
Cadrage & conception veille, cartographie, choix d'archi.	R	C	I	A	C	C
Infrastructure IaC Terraform, Ansible, Swarm	R	I	C	A	I	I
Backend API NestJS, TypeORM, sécurité	A	R	C	I	I	C
Frontend & site Next.js, export statique	A	R	I	I	I	I
Chaîne CI/CD GitHub Actions, GitOps	A	R	C	I	I	C
Observabilité Prometheus, Loki, Grafana	A	R	C	I	I	I
Sécurité & conformité ISO 27001, RGPD, durcissement	R	C	A	I	C	C
Documentation & soutenance mémoire, runbooks, REX	R	C	C	I	A	C

SIGNIFICATION DES RÔLES

R

Responsible — réalise concrètement la tâche

A

Accountable — décide et rend compte, un seul par lot

C

Consulted — consulté avant décision (2 voies)

I

Informed — informé du résultat (1 voie)

DISTRIBUTION DES RÔLES PAR ACTEUR

ACTEUR	R	A	C	I
Chef de projet / DevOps	4	4	0	0
Développeurs	4	0	4	0
Référent sécurité	0	1	5	2
Sponsor	0	2	0	6
Référent pédagogique	0	1	2	5
Référent entreprise	0	0	5	3

Lecture. Chaque lot dispose d'un unique **Accountable** pour éviter la dilution de la décision : le sponsor sur le cadrage et l'infrastructure (enjeux budget et délais), le chef de projet / ingénieur DevOps sur les lots techniques, le référent sécurité sur la sécurité & conformité et le référent pédagogique sur la documentation. La communication à double sens (R ↔ C) est réservée aux acteurs dont l'avis conditionne la réalisation, la simple information (I) aux autres, ce qui fluidifie les arbitrages sans réunion superflue.

Annexe O bis, Matrice d'engagement des parties prenantes (BC2)

Matrice d'engagement des parties prenantes

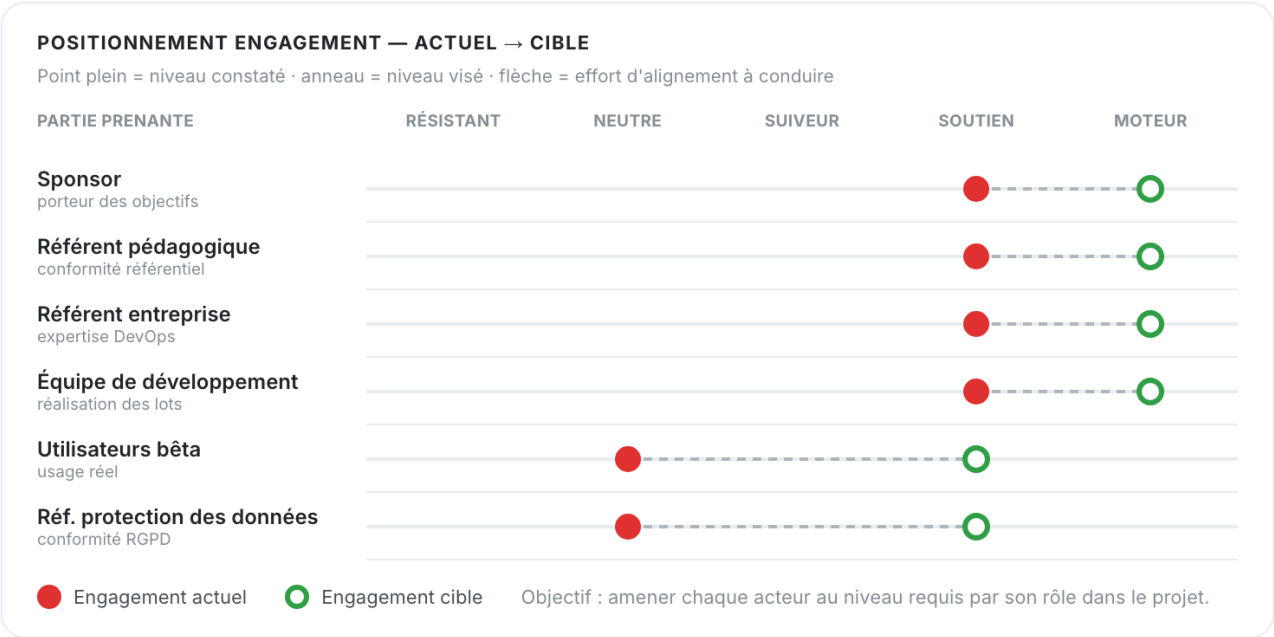
6

PARTIES
PRENANTES

5

NIVEAUX

Niveau d'engagement actuel et cible de chaque acteur, causes identifiées et actions d'amélioration, incluant les mesures d'accessibilité pour les personnes en situation de handicap (PSH).



CAUSES & ACTIONS D'AMÉLIORATION DE L'ENGAGEMENT				
PARTIE PRENANTE	ACTUEL	CIBLE	CAUSES DU NIVEAU D'ENGAGEMENT	ACTIONS À ENTREPRENDRE
Sponsor porteur des objectifs	Soutien	Moteur	Attentes fortes sur le budget et les délais ; besoin de visibilité chiffrée.	Reporting synthétique aux jalons, tableaux de bord de coûts et d'avancement partagés.
Référent pédagogique conformité référentiel	Soutien	Moteur	Garant de l'alignement au RNCP ; disponibilité limitée.	Revue régulière, traçabilité des livrables par bloc de compétences.
Référent entreprise expertise DevOps	Soutien	Moteur	Intérêt technique réel, mais sollicité par ses propres missions.	Points ciblés sur les choix d'architecture, partage des décisions (ADR).
Équipe de développement réalisation des lots	Soutien	Moteur	Motivée par le produit concret ; charge sur les lots du chemin critique.	Autonomie encadrée, célébration des jalons, filets de sécurité (tests, rollback).
Utilisateurs bêta usage réel	Neutre	Soutien	Curieux mais peu impliqués ; retours ponctuels et informels.	Canal de retour structuré, prise en compte visible des remontées.
Référent protection des données conformité RGPD	Neutre	Soutien	Vigilance réglementaire ; intervient en contrôle plus qu'en accompagnement.	Revue de conformité en amont, cartographie ISO 27001 / RGPD partagée.

PRISE EN COMPTE DES SITUATIONS DE HANDICAP (PSH)

L'engagement inclusif est traité comme une exigence transverse. Les **utilisateurs bêta en situation de handicap** testent les surfaces publiques rendues conformes aux référentiels **WCAG 2.2** et **RGAA 4** (structure sémantique, navigation clavier, contrastes, attributs ARIA) et disposent d'un canal de retour accessible. La **documentation projet** et les supports de communication sont produits dans des formats lisibles par lecteur d'écran, et les réunions prévoient au besoin sous-titrage et comptes rendus écrits. Cette attention garantit que la contribution de chaque acteur, quel que soit son handicap, est effectivement recueillie et valorisée.

Annexe T, Outils de gestion de projet ()

